

时空可组合性的一种编程范式

A Programming Paradigm for Spatiotemporal Composability

Yifan Shi^{1,2} Wei Zhang¹ Tianyi Cui²

¹ 北京大学 ²DeepSeek-AI

中文译本

目录

1 引言	2
1.1 可组合性的维度	2
1.2 动机示例	2
1.2.1 插件系统	2
1.2.2 自我进化的智能体底座	3
1.2.3 粗粒度的权宜之计	3
1.3 贡献	3
2 预备知识	5
2.1 副作用	5
2.2 副作用	5
2.3 与动态可组合性的关系	6
3 可逆副作用与反应式副作用	7
3.1 可逆副作用	7
3.1.1 副作用上下文	7
3.1.2 可逆副作用函数	10
3.1.3 副作用的独立性	13
3.2 反应式副作用	15
3.3 副作用上下文	15
3.4 规约与通知	17
3.5 隔离与拦截	18
3.6 上下文范式	20
3.6.1 统一上下文	20
3.6.2 观测等价	20
3.6.3 上下文范式的定位	24
4 动态组合演算	25
4.1 组件与线程	25
4.2 基础演算	27
4.3 进行中的转换	29
4.3.1 撤离	30
4.3.2 迭代	31

4.3.3	异步性	32
4.3.4	失败	33
4.4	元理论	34
4.5	保持	37
4.6	时间性可组合	38
4.7	空间性可组合	40
4.8	进展	42
4.9	合流性	44
5	实现与案例研究	49
5.1	核心库	49
5.1.1	副作用追踪	49
5.1.2	副作用操作	51
5.1.3	组件生命周期	53
5.1.4	上下文访问	55
5.2	组件加载器	56
5.2.1	声明式配置	56
5.2.2	热模块替换	58
5.3	案例研究: Koishi	61
6	讨论	62
6.1	系统边界	62
6.2	服务多路复用	62
6.3	访问控制与沙箱化	63
6.4	语言无关性与语言选择	64
6.5	互依赖与组件粒度	65
6.6	依赖类型与版本化	65
6.7	与语言和操作系统的协同设计	66
7	相关工作	68
7.1	效应与协效应系统	68
7.2	编程范式	69
7.3	时间性可组合	69
7.4	空间性可组合	71
8	结论	73

参考文献

74

摘要

现代软件——从插件系统 (plugin system) 到自我进化的智能体底座 (agent harness) ——日益需要动态组合 (dynamic composition)，然而其形式基础仍不完善。我们识别出这一问题的两个正交维度：时间性可组合 (temporal composability)，即在移除组件时完全撤销其对环境所致修改的能力；空间性可组合 (spatial composability)，即声明组件间依赖并对其施以反应式管理的能力。我们通过将经典的副作用 (effect) 与协副作用 (coeffect) 概念提升为运行时机制来解决这两个维度。特别地，我们形式化了可逆副作用 (reversible effects)：每一次上下文 (context) 变换都携带一个由运行时跟踪的逆变换。我们形式化了反应式协副作用 (reactive coeffects)：上下文的每一次变更都依据组件的协副作用规约通知该组件。我们将副作用上下文 (effect context) 与协副作用上下文 (coeffect context) 统一为单一的上下文类型，这构成了一种编程范式 (programming paradigm)。在此之后，我们把这些机制组合为组件 (component) 的概念，并给出一个动态组合演算 (calculus of dynamic composition)，其元理论 (metatheory) 将时空可组合性 (spatiotemporal composability) 从单个组件推广到由交错组件构成的整个系统。我们在 Cordis 中实现了这些思想——Cordis 是一个时空可组合性的元框架 (meta-framework)，它提供了一个具备副作用追踪与协副作用解析的核心库，以及一个具备配置调和 (configuration reconciliation) 与热模块替换 (hot module replacement, HMR) 的声明式组件加载器 (component loader)。

第 1 章 引言

组合 (composition) ——从较简单的部分装配出复杂系统——是软件工程的一项基础原则 [1]。传统上, 组合是静态的: 函数调用、模块导入与类继承在编译时解析, 并在整个执行过程中保持不变。然而, 现代软件日益需要动态组合 (dynamic composition): 组件 (component) 在运行时被加载、卸载与重配置。插件架构 (plugin architecture) [2] 与自我进化的智能体底座 (agent harness) 都要求系统能够在运行中安全地增删功能, 而当前的实际做法依赖粗粒度机制 [3]——只能通过重启来重配置, 并丢弃运行时状态。尽管动态组合的实际重要性日益增长, 与静态组合所拥有的丰富形式框架相比, 其理论基础仍不完善。

1.1 可组合性的维度

为刻画动态组合的需求, 我们在已被充分研究的组合之代数方面之外, 识别出两个正交维度:

- 时间性可组合 (temporal composability) 处理时间维度: 组件被移除时, 它对共享环境所做的修改必须被完全且安全地逆转。这要求跟踪组件进行的每一项资源分配、事件注册与状态变更, 并保证这些内容在移除时被有序回收。
- 空间性可组合 (spatial composability) 处理空间维度: 组件必须能够以结构化且可验证的方式声明、发现并解析彼此之间的依赖。这要求管理依赖拓扑, 并随依赖变化协调各组件的生命周期 (lifecycle)。

在静态场景下, 时间性可组合归约为词法作用域 (如 RAII [4]、bracket 模式 [5]), 空间性可组合归约为模块导入解析 [6]。而在组件于运行时到来与离开的动态场景下, 两个维度都显著变得更难: 时间性可组合必须处理作用域不受词法约束的、长寿命的有状态副作用 (effect); 空间性可组合则必须处理在执行过程中出现、消失或改变身份的依赖。

1.2 动机示例

1.2.1 插件系统

插件系统 (plugin system) 是动态组合的一个典型实例。我们以 Visual Studio Code (VSCode) ——使用最广泛的可扩展 IDE 之一——作为代表性例子。

时间性局限。VSCode 在一个称为扩展宿主 (extension host) 的共享进程中运行所有扩展。虽然扩展可以被动态安装, 但该宿主没有提供在运行时卸载单个扩展代码的机制。一旦某个扩展的 `activate` 函数执行完毕, 禁用或卸载它就需要重启整个宿主, 从而波及所有已加载的扩展。诸如主题、键绑定、代码片段等纯声明式扩展不含代码, 可以自由移除; 然而在按安装量排名前 100 的扩展中, 有 87 个包含可执行代码¹, 因此在移除时都需要这样的重启。尽管 VSCode 提供了 `deactivate` 钩子, 它只是宿主进程终止时的优雅停机回调, 并不能实现活移除 (live removal)。此外, 该钩子将副作用的释放与副作用的创建 (在 `activate` 中) 分离, 违背了关注点的局部性, 使完全的清理难以验证。

空间性局限。VSCode 确实提供了 `extensionDependencies` 来声明扩展间的依赖, 但鲜有使用: 在按安装量排名前 100 的扩展中, 只有 7 个声明了对非内置扩展的 `extensionDependencies`¹。这种稀缺反映了扩展 API 的形态——它暴露的是命令、视图、语言特性等固定的表层扩展点。

扩展通过这些点向宿主做贡献，而非彼此依赖，因此扩展之间的依赖很少出现。此外，VSCode 的扩展间交互机制不提供结构性契约：它通过 `vscode.extensions.getExtension(...).exports` 把一个扩展的功能暴露给其他扩展，但返回值是无类型的（默认为 `any`），依赖方无法依赖一个经过检查的接口。简言之，VSCode 把扩展导向一组由宿主提供的固定扩展点，却没有为扩展彼此依赖提供安全、结构化的途径。

这两类局限并非 VSCode 独有；它们在各类插件系统中普遍再现 [2, 7]，只是程度不同。

1.2.2 自我进化的智能体底座

现代 AI 智能体 (agent) 依赖运行时 agent harness [8, 9, 10]。这类系统可能组合多样的工具集 [11] 与执行环境，管控权限与沙箱化 (sandboxing)，维护会话状态与持久化，提供上下文 (context) 管理与记忆系统 [12]，编排子智能体 (subagent) 与多智能体工作流 [13]，并向用户与自动化流程暴露接口。未来的 harness 可能在请求持续服务的同时，生成并部署对自身组件的修改。模型合成的可复用工具 [14] 是组件级自我修改的一个较窄的前奏。每一次这样的修改，本身都是动态组合的一个实例。

由于这些修改持续发生且很少或没有人工监督，动态可组合性 (dynamic composability) 变得不可或缺。没有时间性可组合，每次自我修改都迫使一次完全重启，丢弃所有进程本地的累积状态；在这样的频率下，累积的不可用时间会相当可观，正在执行的任务也会被反复打断；更糟的是，一次有缺陷的自我修改可能使恢复所需的那个进程本身瘫痪。没有空间性可组合，每个模块必须自行检测并适应其依赖的模块在出现、消失或改变身份时的变化，且只能以临时手段做到；更糟的是，朴素的代码替换策略可能悄然破坏依赖方，或引入仅在 reload 时才暴露的循环依赖。

1.2.3 粗粒度的权宜之计

动态可组合性之所以受到的形式化关注有限，原因之一是操作系统与容器编排器 (container orchestrator) 已经提供了粗粒度的替代品。操作系统以进程的粒度提供时间性可组合；容器编排器 [3] 以服务 (service) 的粒度提供空间性可组合。实践中，大多数软件通过求助于这些粗粒度机制来容忍细粒度可组合性的缺失：行为失常的模块靠重启进程处理，服务依赖靠容器编排器管理。

然而这一权宜之计代价高昂。在时间上，每次重启都丢弃所有进程本地的累积状态（如缓存、连接、部分完成的计算），重建它们需要数秒到数分钟 [15]；在此期间维持可用性需要冗余副本，为无法恢复单个组件而付出资源开销。在空间上，容器层的编排无法表达共享同一地址空间的组件之间的依赖，并为本可以是本地函数调用的交互引入网络开销。这两种机制都作用于进程与容器的边界，而现代系统日益在更细的层级上进行组合。这一粒度错配呼唤一种组合式抽象，能与与组件自身相同的层级上管理副作用与依赖。

1.3 贡献

动态可组合性的两个维度分别关乎计算如何修改环境、以及如何依赖环境。这两个方向正是副作用系统 (effect system) [16, 17] 与协副作用系统 (coeffect system) [18, 19] 所形式化的对象：副作用为推理环境修改提供了形式化词汇，协副作用为推理环境需求提供了形式化词汇。然而，现有的构造把推理限制在词法固定作用域上的编译时分析，无法延伸到组件于运行时到来与离开的动态场景。通过把副作用提升为可逆的运行模型、把协副作用提升为反应式的依赖解析机制，我们为动态可组合性获得了统一的形式基础——它独立于语言，适用于任何需要动态组

合的软件架构。我们做出以下贡献：

1. 我们形式化可逆副作用（revertible effects）（3.1 节）：每一次上下文变换都携带一个由运行时跟踪的显式逆变换，且跟踪与恢复都保持组合性，从而上下文在组件移除时得以恢复。这确立了局部的时间性可组合。
2. 我们形式化反应式副作用（reactive effects）（3.2 节）：组件将其所需的副作用声明为一份规约，上下文的每一次变更都依据该规约将此变更通知组件——激活、去激活或中性。这确立了局部的空间性可组合。
3. 我们把副作用上下文（effect context）与副作用上下文（effect context）统一为单一的上下文类型（3.3 节），其中副作用上的一个观测等价（observational equivalence）为副作用提供了独立性，构成时空可组合性（spatiotemporal composability）的一种编程范式。
4. 我们给出动态组合演算（calculus of dynamic composition）（第 4 章），把这两种机制组合为组件的概念，并为其生命周期配备操作语义。其元理论（metatheory）将时空可组合性从单个组件推广到由交错组件构成的整个系统。
5. 我们在 Cordis（第 5 章）中实现这些思想。Cordis 是一个时空可组合性的元框架（meta-framework），它提供了一个以副作用追踪与副作用解析实现形式模型的核心库，以及一个具备配置调和（configuration reconciliation）与热模块替换（hot module replacement, HMR）的声明式组件加载器（component loader）。

第 2 章 预备知识

本节概述副作用（effect）系统与协副作用（coeffect）系统——支撑本工作的两根理论支柱。我们假设读者熟悉基础的类型论与范畴论；本节的目标是固定记号，并引入第 3 章将操作化为运行时机制的关键抽象。

2.1 副作用

在简单类型 lambda 演算（simply typed lambda calculus, STLC）[20, 21] 中，类型判定 $\Gamma \vdash t : T$ 表明项 t 在上下文（context） Γ 下具有类型 T 。副作用系统细化类型，以描述一个计算可能产生哪些副作用，得到如下形式的判定：

$$\Gamma \vdash t : T_{\text{effect}} \quad (2.1)$$

这里，结果类型由副作用代数（effect algebra）中的一个元素标注，该元素描述计算可能产生哪些副作用，从而支持对有状态计算的组合式推理。这一方法源自 Lucassen 与 Gifford [22]，他们引入了一种区分类型、副作用与区域（region）的带种类（kinded）类型系统，以发现并行程序中的调度约束。

单子式副作用。Moggi [16] 首先通过单子（monad）以范畴论方式建模计算副作用；Wadler [23] 在 Haskell 中推广了这一方法。范畴 $\mathcal{C}$ 上的单子 (T, η, μ) 把一个含副作用的计算封装为类型 $T(A)$ 的值： $\eta : A \rightarrow T(A)$ 将纯值提升（lift）为计算， $\mu : T(T(A)) \rightarrow T(A)$ 将嵌套的计算串联起来。经典的实例包括 Maybe 单子（用于部分性）、State 单子（用于可变状态）与 IO 单子（用于外部交互）。

代数式副作用。Plotkin 与 Power [17, 24] 证明了代数运算确定单子，建立了一个将副作用接口与其实现解耦的框架。副作用签名 Σ 声明一组运算（例如，用于状态的 $\text{get} : () \rightarrow S$ 、 $\text{put} : S \rightarrow ()$ ）；程序自由地调用运算，而不必绑定于某个特定解释。Plotkin 与 Pretnar [25] 随后引入了副作用处理器（effect handler），通过提供续体语义来解释运算：

$$\text{handle } e \text{ with } \{\text{op}(v, \kappa) \mapsto \dots\} \quad (2.2)$$

处理器接收运算实参 v 与定界续体（delimited continuation） κ ，并可以零次、一次或多次调用该续体，从而在统一的框架内 [26] 支持异常、协程与非确定性。Koka [27, 28]、Eff [29] 与 OCaml 5 [30] 等语言已在不同设计取舍下采用了代数式副作用。

2.2 协副作用

与副作用对偶地，协副作用系统 [18, 31] 丰富的是上下文而非类型，得到如下形式的判定：

$$\Gamma_{\text{coeffect}} \vdash t : T \quad (2.3)$$

这里，上下文由协副作用代数（coeffect algebra）中的一个元素标注，该元素描述计算对其环境有何要求，例如要访问的资源、要持有的权限、或要依赖的服务（service）。副作用建模程序对世界的作用，协副作用则建模世界对程序的约束。

余单子式协副作用。用余单子 (comonad) 来组织依赖上下文的计算, 这一思想由 Uustalu 与 Vene [32] 首先发展, 他们提出对称 (半) 么半余单子作为 Moggi 单子式副作用框架的对偶, 捕获了数据流与属性求值等概念。Petricek 等 [18] 在此基础上提出协副作用, 作为对上下文依赖的统一静态分析。余单子 (D, ε, δ) 捕获依赖上下文的计算: $\varepsilon: D(A) \rightarrow A$ 从上下文中提取当前值, $\delta: D(A) \rightarrow D(D(A))$ 复制上下文以供嵌套访问。Environment 余单子 $D(X) = E \times X$ 建模对固定环境 E 的依赖; Stream 余单子 $D(X) = \mathbb{N} \rightarrow X$ 建模对时序数据的依赖。

分次协副作用。为进行更细粒度的追踪, 分次协副作用 (graded coeffect) 系统使用预序半环 $S = (S, \leq, +, \times, 0, 1)$ 作为协副作用代数 [33]; 这一纪律后来由 Gaboardi 等 [19] 与分次效应 (graded effect) 统一。 S 中的元素标注每个变量绑定以量化其使用: 0 表示未使用, 1 表示线性使用, n 表示有界使用, ∞ 表示无限制使用。半环运算分别按顺序 ($\times$) 与并行 ($+$) 组合协副作用, 从而在统一的代数框架 [37] 内支持精确的资源追踪、敏感度分析 [34] 与信息流控制 [35, 36]。

2.3 与动态可组合性的关系

副作用与协副作用系统沿两个互补的方向组织关于计算的推理: 副作用描述计算如何修改其环境, 协副作用描述计算如何依赖其环境。这两个方向正对应第1章识别出的动态可组合性 (dynamic composability) 的两个维度:

- 时间性可组合 (temporal composability) 要求组件对共享环境的修改在卸载时可逆。这里相关的副作用是那些持久地变换该环境的有状态副作用; 要撤销这样一次变换, 就要求它容许一个逆。
- 空间性可组合 (spatial composability) 要求组件间依赖被声明并被反应式地管理。这样的依赖恰是协副作用所捕获的对象, 而管理它们就是依据环境所供给的内容对每个依赖进行解析。

然而, 经典的副作用与协副作用系统是静态工具: 副作用在词法固定的作用域内被追踪, 并由编译时的处理器了结; 协副作用标注则对照执行前确定的上下文来验证。相比之下, 动态组合 (dynamic composition) 要求这些保证对运行时到来与离开的组件、对持续演化的上下文依然成立。部署之后才加载的插件, 没有固定的词法作用域能够限定; 运行时配置涌现出的依赖, 没有编译时的上下文能够预见。

这促使一次视角转换: 我们不再以更多标注去扩充静态类型系统, 而是把副作用与协副作用的概念结构具体化 (reify), 使运行时能够直接对其进行操作, 从而动态地建立这些系统在静态下提供的保证。

第 3 章 可逆副作用与反应式副作用

本章将第 2 章引入的副作用 (effect) 与副作用 (coeffect) 概念提升为运行时机制, 构造一套动态组合的理论。核心思想是: 把携带副作用与副作用的类型上下文转化为上下文类型 (context type), 即把上下文具体化为一等实体的、运行时可操作的类型。对副作用类型, 我们将其建模为配有逆变换的上下文变换, 从而实现局部的时间性可组合 (temporal composability); 对副作用上下文, 我们将其建模为携带依赖信息的类型, 从而实现局部的空间性可组合 (spatial composability)。进而, 副作用上的一个观测等价 (observational equivalence) 为副作用提供了独立性。同时携带副作用与副作用的统一上下文, 其本身即构成一种编程范式。

3.1 可逆副作用

时间性可组合是指在运行时加载与卸载组件 (component) 的能力, 且卸载之后, 共享环境恢复到组合之前的状态。这要求组件对环境的每一处修改都既可追踪又可恢复。因此, 我们把一个副作用建模为类型为 $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$ 的函数: 作用于当前上下文时, 它给出修改后的上下文以及一个显式的逆。提供这个逆, 使副作用可以被撤销; 把它交还给运行时, 使副作用可以被追踪。我们称这类副作用为可逆副作用 (revertible effects): 通过在执行过程中追踪并复合这些逆, 完整的环境恢复成为一项结构性保证。

3.1.1 副作用上下文

给定任意非纯函数 $f_{\text{impure}} : X \rightarrow Y$, 我们把它变换为纯化形式 $f : \Gamma \times X \rightarrow \Gamma \times Y$, 其中 Γ 是上下文 (context), 所有可能的副作用都可以表示为 Γ 上的变换。对任意固定输入 $x : X$, 诱导映射 $\gamma \mapsto \text{pr}_1(f(\gamma, x))$ 与返回值无关地刻画了 f 的副作用。于是 Γ 上的副作用生活在复合 $\circ$ 之下由变换 $\Gamma \rightarrow \Gamma$ 构成的幺半群中, 其中每条幺半群公理都可以直接读作副作用的一条性质:

- 封闭性: 两个副作用的顺序复合仍是一个副作用;
- 结合性: 复合副作用与其加括号的方式无关;
- 单位元: Γ 上的恒等函数 id_Γ 充当复合的单位。

为建模可被撤销的副作用, 我们把每个变换 f 与另一个撤销 f 的变换 g 配对, 称 g 为 f 的左逆 (left inverse), 全文简称逆。撤销是单侧的: 对一个逆所要求的是 $g \circ f$, 而绝不是 $f \circ g$ 。变换对自带一个乘法:

定义 3.1 (扭复合). 定义上下文变换对的扭复合 (twisted composition) 为

$$(f_1, g_1) \circ (f_2, g_2) := (f_1 \circ f_2, g_2 \circ g_1) \quad (3.1)$$

与 $\circ$ 本身一样, 左操作数后于右操作数生效, 而诸逆按相反的顺序累积。它使 $(\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma)$ 成为以 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 为单位元的幺半群, 即变换幺半群与其反幺半群的积, 我们称之为 Γ 上的扭复合幺半群 $\mathfrak{S}_\Gamma$ 。

为了在上下文自身内部追踪副作用, 我们引入如下定义:

定义 3.2. 给定上下文 Γ ，定义其副作用上下文 (*effect context*) 为

$$\partial\Gamma := \Gamma \times (\Gamma \rightarrow \Gamma) \quad (3.2)$$

它可以理解为一个二元组 (γ, φ) ，其中：

- $\gamma : \Gamma$ 是当前上下文状态；
- $\varphi : \Gamma \rightarrow \Gamma$ 是累加器 (accumulator)，即迄今所执行的各副作用之逆的复合，是把上下文恢复到初始状态的函数。

特别地，初始副作用上下文可以表示为 $(\gamma_0, \text{id}_\Gamma)$ 。

我们还记 $\partial^2\Gamma = \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$ ，并沿此塔依次类推。

由于累加器 φ 的存在， $\partial\Gamma$ 上执行的所有副作用都可以被追踪与恢复。下面给出追踪与恢复的具体构造。

定义 3.3. 定义上下文函数对上的变换 track 如下：

$$\begin{aligned} \text{track}_\Gamma &: (\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma) \rightarrow \partial\Gamma \rightarrow \partial\Gamma \\ \text{track}_\Gamma &= (f, g) \mapsto (\gamma, \varphi) \mapsto (f(\gamma), \varphi \circ g) \end{aligned} \quad (3.3)$$

这一变换把前向函数 f 连同候选逆 g 转换为副作用上下文 $\partial\Gamma$ 上的一个变换。把 $\text{track}_\Gamma(f, g)$ 作用于状态 (γ, φ) ，就用 f 变换 γ ，并把逆 g 复合到 φ 上，从而在上下文中追踪 f 的副作用。

定理 3.4. 对每个 $(f, g) \in (\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma)$ ，下面的图交换，即

$$\text{pr}_1 \circ \text{track}_\Gamma(f, g) = f \circ \text{pr}_1 \quad (3.4)$$

$$\begin{array}{ccc} \Gamma & \xrightarrow{f} & \Gamma \\ \text{pr}_1 \uparrow & \Downarrow \text{track} & \uparrow \text{pr}_1 \\ \partial\Gamma & \xrightarrow{f'} & \partial\Gamma \end{array}$$

证明. 对所有 $(\gamma, \varphi) \in \partial\Gamma$,

$$\begin{aligned} (\text{pr}_1 \circ \text{track}_\Gamma(f, g))(\gamma, \varphi) &= \text{pr}_1(f(\gamma), \varphi \circ g) \\ &= f(\gamma) \\ &= (f \circ \text{pr}_1)(\gamma, \varphi) \end{aligned}$$

□

定理 3.5. track 是从 $\mathfrak{T}_\Gamma$ 到 $\partial\Gamma \rightarrow \partial\Gamma$ 的么半群同态。即：

$$1. \text{track}_\Gamma(\text{id}_\Gamma, \text{id}_\Gamma) = \text{id}_{\partial\Gamma};$$

$$2. \text{对所有 } (f_1, g_1), (f_2, g_2) \in \mathfrak{T}_\Gamma,$$

$$\text{track}_\Gamma((f_1, g_1) \circ (f_2, g_2)) = \text{track}_\Gamma(f_1, g_1) \circ \text{track}_\Gamma(f_2, g_2) \quad (3.5)$$

证明. 1. 单位元被映到单位元, 因为 $\text{track}_\Gamma(\text{id}_\Gamma, \text{id}_\Gamma)(\gamma, \varphi) = (\gamma, \varphi \circ \text{id}_\Gamma) = (\gamma, \varphi)$.

2. 对乘法, 取任意 $(\gamma, \varphi) \in \partial\Gamma$:

$$\begin{aligned} (\text{track}_\Gamma(f_1, g_1) \circ \text{track}_\Gamma(f_2, g_2))(\gamma, \varphi) &= \text{track}_\Gamma(f_1, g_1)(f_2(\gamma), \varphi \circ g_2) \\ &= (f_1(f_2(\gamma)), \varphi \circ g_2 \circ g_1) \\ &= \text{track}_\Gamma(f_1 \circ f_2, g_2 \circ g_1)(\gamma, \varphi) \end{aligned}$$

□

定义 3.6. 定义 $\partial\Gamma$ 上的变换 recover 如下:

$$\begin{aligned} \text{recover}_\Gamma &: \partial\Gamma \rightarrow \partial\Gamma \\ \text{recover}_\Gamma &= (\gamma, \varphi) \mapsto (\varphi(\gamma), \text{id}_\Gamma) \end{aligned} \quad (3.6)$$

这一变换把恢复函数 φ 作用于当前状态 γ , 并把 φ 重置为恒等。下图展示了在一系列副作用 $\text{track}(f_1, g_1), \dots, \text{track}(f_n, g_n)$ 作用于 $\partial\Gamma$ 之后, recover 如何把上下文恢复到初始状态:

$$\begin{array}{ccccccc} \Gamma & \xrightarrow{f_1} & \Gamma & \xrightarrow{\dots} & \Gamma & \xrightarrow{f_n} & \Gamma \\ \downarrow \text{track} & & \downarrow \text{track} & & \downarrow \text{track} & & \\ \partial\Gamma & \xrightarrow{f'_1} & \partial\Gamma & \xrightarrow{\dots} & \partial\Gamma & \xrightarrow{f'_n} & \partial\Gamma \xrightarrow{\text{recover}} \partial\Gamma \end{array}$$

该图表明: 先执行被追踪的副作用、再执行 recover , 会把初始副作用上下文带回它自身。每个追踪步骤所保持的, 其实是从任意状态出发的恢复结果本身:

定理 3.7. 对每个 $(\gamma, \varphi) \in \partial\Gamma$ 以及每个满足 $g(f(\gamma)) = \gamma$ 的对 (f, g) ,

$$\text{recover}_\Gamma(\text{track}_\Gamma(f, g)(\gamma, \varphi)) = \text{recover}_\Gamma(\gamma, \varphi) \quad (3.7)$$

证明.

$$\begin{aligned} \text{recover}_\Gamma(\text{track}_\Gamma(f, g)(\gamma, \varphi)) &= \text{recover}_\Gamma(f(\gamma), \varphi \circ g) \\ &= (\varphi(g(f(\gamma))), \text{id}_\Gamma) \\ &= (\varphi(\gamma), \text{id}_\Gamma) = \text{recover}_\Gamma(\gamma, \varphi) \end{aligned}$$

□

对由对构成的序列无需另行论证。设 $(f_1, g_1), \dots, (f_n, g_n)$ 从 (γ, φ) 出发依次施加, 并记 $\delta_0 = \gamma$, $\delta_i := f_i(\delta_{i-1})$ 。由定理 3.5, 复合 $\text{track}_\Gamma(f_n, g_n) \circ \dots \circ \text{track}_\Gamma(f_1, g_1)$ 就是扭复合 $(f_n \circ \dots \circ f_1, g_1 \circ \dots \circ g_n)$ 的 track ; 而若对每个 i 都有 $g_i(\delta_i) = \delta_{i-1}$, 则 $(g_1 \circ \dots \circ g_n)(\delta_n) = \delta_0 = \gamma$ 。于是该对在 γ 处满足定理 3.7 的假设, 从而一次应用该定理即得

$$\text{recover}_\Gamma \left((\text{track}_\Gamma(f_n, g_n) \circ \dots \circ \text{track}_\Gamma(f_1, g_1))(\gamma, \varphi) \right) = \text{recover}_\Gamma(\gamma, \varphi) \quad (3.8)$$

取 $(\gamma, \varphi) = (\gamma_0, \text{id}_\Gamma)$ ，恢复便把以这种方式到达的每个状态都带回 $(\gamma_0, \text{id}_\Gamma)$ 。而满足 $g \circ f = \text{id}_\Gamma$ 的对在每个状态处都满足该假设。

恢复是通过量 $\varphi(\gamma)$ 来读取状态的，我们把 $\varphi(\gamma) = \gamma_0$ 称为 $\partial\Gamma$ 中状态的一个可靠性不变量 (soundness invariant)。

3.1.2 可逆副作用函数

上一节的 track / recover 模型把逆视为先验给定： $\text{track}_\Gamma(f, g)$ 在见到任何上下文状态之前就固定了 g ，因此同一个 g 必须服务于该副作用施加的所有状态。然而在实践中，每个副作用的逆并非先验已知：它必须由调用方在副作用施加之处提供。此外，recover 是全有或全无的：它无法选择性地撤销一个副作用而保留其余。针对这两个问题，我们在输入与输出两侧同时增强模型：

1. 输入侧：不仅变换 Γ ，还随其返回一个逆函数，使逆恰在副作用施加处提供： $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$ ，即 $\Gamma \rightarrow \partial\Gamma$ ；
2. 输出侧：不仅变换 $\partial\Gamma$ ，还随其返回一个逆函数，使一个副作用可被撤销而其余被保留： $\partial\Gamma \rightarrow \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$ ，即 $\partial\Gamma \rightarrow \partial^2\Gamma$ 。

这一增强保持了输入与输出之间的结构一致性，因此我们仍可定义相应的理论，使其保持 track 的数学性质。所得类型即副作用函数 $\mathfrak{E}_\Gamma$ 及其带见证的细化 $\mathfrak{E}_\Gamma^*$ ：

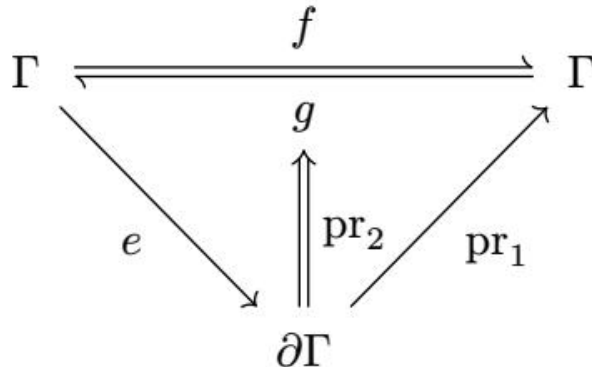
定义 3.8. 定义副作用函数 (effect function) $\mathfrak{E}_\Gamma$ 与带见证的副作用函数 (witnessed effect function) $\mathfrak{E}_\Gamma^*$ 如下：

$$\begin{aligned} \mathfrak{E}_\Gamma &:= \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \\ \mathfrak{E}_\Gamma^* &:= (e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)) \\ &\quad \times \left((\gamma : \Gamma) \rightarrow ((\delta : \Gamma) \times (g : \Gamma \rightarrow \Gamma) \times ((\delta, g) = e(\gamma) \rightarrow g(\delta) = \gamma)) \right) \end{aligned} \quad (3.9)$$

其中 $e(\gamma)$ 给出一个二元组 (δ, g) ，表示：

- $\delta : \Gamma$ 是新的上下文；
- $g : \Gamma \rightarrow \Gamma$ 是当前副作用的逆函数。

$\mathfrak{E}_\Gamma^*$ 的元素逐状态地选择其逆，而约束 $g(\delta) = \gamma$ 把这一选择限定为只在副作用施加之处撤销该副作用，在其余处处则让 g 不受约束。单个满足 $g \circ f = \text{id}_\Gamma$ 的 g 一举满足所有状态处的约束，并通过 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 诱导出 $\mathfrak{E}_\Gamma^*$ 的一个元素；定理 3.11 将证明这是一个同态。该约束可以画成如下的交换图，它保证 e 所返回的逆 g 确实逆转 e 施加之处的变换：



由于副作用函数 $\mathfrak{E}_\Gamma$ 不再是上下文上的自同态，它们无法直接复合。因此我们为副作用复合定义一个新的运算：

定义 3.9. 给定函数 $f, g \in \mathfrak{E}_\Gamma$ ，定义它们的副作用复合 $f \diamond g$ 如下：

$$\begin{aligned} f \diamond g &: \Gamma \rightarrow \partial\Gamma \\ f \diamond g &= \gamma \mapsto \mathbf{let} (\delta, s) = g(\gamma) \mathbf{in} \mathbf{let} (\varepsilon, t) = f(\delta) \mathbf{in} (\varepsilon, s \circ t) \end{aligned} \quad (3.10)$$

定理 3.10. 副作用复合把 $\mathfrak{T}_\Gamma$ 的幺半群结构搬运到 $\mathfrak{E}_\Gamma$ 上。即：

1. $(\mathfrak{E}_\Gamma, \diamond)$ 是以 $\eta_\Gamma := \gamma \mapsto (\gamma, \text{id}_\Gamma)$ 为单位元的幺半群；
2. 对应 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 是从 $\mathfrak{T}_\Gamma$ 到 $\mathfrak{E}_\Gamma$ 的幺半群同态。

证明. 1. 结合律与单位律由 $\circ$ 的相应定律逐分量得出。

2. 记 $e_i = \gamma \mapsto (f_i(\gamma), g_i)$ ；则 $(e_1 \diamond e_2)(\gamma) = (f_1(f_2(\gamma)), g_2 \circ g_1)$ ，这正是 $(f_1, g_1) \circ (f_2, g_2)$ 的像，而 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 被映到 η_Γ 。

□

定理 3.11. 见证在副作用复合下得以保持，且一个一致的逆在每个状态处都构成见证。即：

1. $\mathfrak{E}_\Gamma^*$ 是 $\mathfrak{E}_\Gamma$ 的子幺半群；
2. 定理 3.10 的同态把每个满足 $g \circ f = \text{id}_\Gamma$ 的对映入 $\mathfrak{E}_\Gamma^*$ 。

证明. 1. 单位元属于 $\mathfrak{E}_\Gamma^*$ ，因为 $\text{id}_\Gamma(\gamma) = \gamma$ 。对封闭性，取 $f, g \in \mathfrak{E}_\Gamma^*$ 及任意 $\gamma \in \Gamma$ ，令 $(\delta, s) = g(\gamma)$ 、 $(\varepsilon, t) = f(\delta)$ ，于是 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 。那么 $s(\delta) = \gamma$ 且 $t(\varepsilon) = \delta$ ，因此 $(s \circ t)(\varepsilon) = s(\delta) = \gamma$ 。

2. $g \circ f = \text{id}_\Gamma$ 在每个 γ 处给出 $g(f(\gamma)) = \gamma$ ，故这样一个对的像在每个状态处都被见证。

□

正如 track 把 Γ 上的变换对提升到 $\partial\Gamma$ ，我们定义 effect 把 $\mathfrak{E}_\Gamma$ 提升到 $\mathfrak{E}_{\partial\Gamma}$ ：

定义 3.12. 定义副作用函数变换 effect 如下：

$$\begin{aligned} \text{effect}_\Gamma &: \mathfrak{E}_\Gamma \rightarrow \partial\Gamma \rightarrow \partial^2\Gamma \\ \text{effect}_\Gamma &= e \mapsto (\gamma, \varphi) \mapsto \mathbf{let} (\delta, g) = e(\gamma) \mathbf{in} \\ &\quad ((\delta, \varphi \circ g), \text{track}_\Gamma(g, \text{pr}_1 \circ e)) \end{aligned} \quad (3.11)$$

由于 $\text{effect}_\Gamma(e)$ 本身属于 $\mathfrak{E}_{\partial\Gamma}$ ，它所返回的是把定义 3.8 上读一层之后的那个意义上的逆。那个逆本身又是把副作用的两个方向互换后所得对的 track。普通的追踪规则再次适用：撤销该副作用本身也是一个副作用，它用 g 变换状态，而撤销这一撤销的方式就是再次执行该副作用，这正是 $\text{pr}_1 \circ e$ 所做的事。因此，这个逆恰好按 track 所规定的方式复合到它所收到的累加器上。

现在我们可以为 effect 证明与 track 类似的性质。

定理 3.13. effect 保持 $\diamond$ 运算。即对全部 $f, g \in \mathfrak{E}_\Gamma$ ，

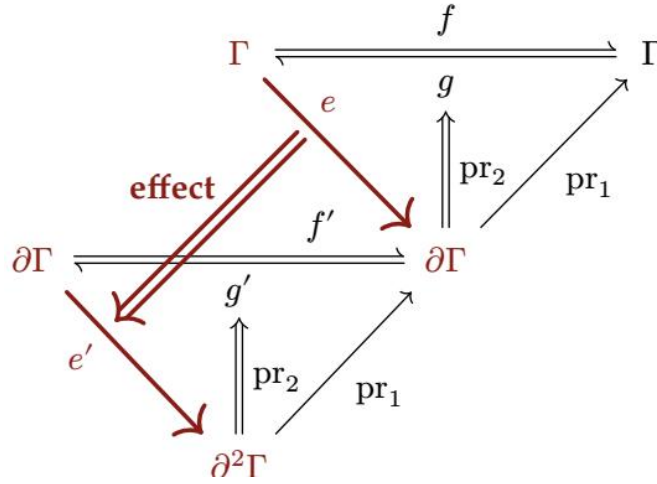
$$\text{effect}_\Gamma(f) \diamond \text{effect}_\Gamma(g) = \text{effect}_\Gamma(f \diamond g) \quad (3.12)$$

证明. 取任意 $(\gamma, \varphi) \in \partial\Gamma$, 令 $(\delta, s) = g(\gamma)$ 、 $(\varepsilon, t) = f(\delta)$, 于是 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$, 且 $\text{pr}_1 \circ (f \diamond g) = (\text{pr}_1 \circ f) \circ (\text{pr}_1 \circ g)$. 那么

$$\begin{aligned} & (\text{effect}_\Gamma(f) \diamond \text{effect}_\Gamma(g))(\gamma, \varphi) \\ &= ((\varepsilon, \varphi \circ s \circ t), \text{track}_\Gamma(s, \text{pr}_1 \circ g) \circ \text{track}_\Gamma(t, \text{pr}_1 \circ f)) \\ &= ((\varepsilon, \varphi \circ s \circ t), \text{track}_\Gamma(s \circ t, (\text{pr}_1 \circ f) \circ (\text{pr}_1 \circ g))) \\ &= \text{effect}_\Gamma(f \diamond g)(\gamma, \varphi) \end{aligned}$$

其中第一步在 (γ, φ) 与 $(\delta, \varphi \circ s)$ 处展开定义 3.12, 第二步用定理 3.5, 第三步折叠定义 3.12. $\square$

两个层次之间如何关联, 由下图展示. 其上三角是 e 的见证条件 (依定义 3.8), 其下三角则是 e' 是否以 e 的方式被见证的问题.



在两个层次之间, 投影 pr_1 把每个提升后的映射关联到被提升的那个映射, 正如定理 3.4 中对 track 所示.

定理 3.14. 设 $e \in \mathfrak{E}_\Gamma$, 记 $f := \text{pr}_1 \circ e$, 并设 $e' := \text{effect}_\Gamma(e)$, 其前向映射为 $f' := \text{pr}_1 \circ e'$. 则:

1. $\text{pr}_1 \circ f' = f \circ \text{pr}_1$;
2. 对每个 $(\gamma, \varphi) \in \partial\Gamma$, 提升后的逆 $g' := \text{pr}_2(e'(\gamma, \varphi))$ 与在该处被见证的逆 $g := \text{pr}_2(e(\gamma))$ 满足 $\text{pr}_1 \circ g' = g \circ \text{pr}_1$.

证明. 1. 由定义 3.12, $f'(\gamma, \varphi) = (f(\gamma), \varphi \circ g)$, 其状态为 $f(\gamma) = (f \circ \text{pr}_1)(\gamma, \varphi)$.

2. 这就是把定理 3.4 应用于 $g' = \text{track}_\Gamma(g, f)$.

$\square$

下三角是否闭合, 可以通过计算提升后的逆返回什么来判定:

定理 3.15. 设 $e \in \mathfrak{E}_\Gamma^*$ 并记 $f := \text{pr}_1 \circ e$. 固定 $(\gamma, \varphi) \in \partial\Gamma$, 令 $(\delta, g) = e(\gamma)$, 并记 (Δ, g') 为 $\text{effect}_\Gamma(e)$ 在 (γ, φ) 处的值. 则

$$g'(\Delta) = (\gamma, \varphi \circ g \circ f) \quad (3.13)$$

状态被精确恢复. 累加器也随之恢复——等价地, $\text{effect}_\Gamma(e) \in \mathfrak{E}_{\partial\Gamma}^*$ ——当且仅当 $g \circ f = \text{id}_\Gamma$; 且在任何情形下都有 $(\varphi \circ g \circ f)(\gamma) = \varphi(\gamma)$, 故可靠性不变量得以保持.

证明. 由定义 3.12, $\Delta = (\delta, \varphi \circ g)$ 且 $g' = \text{track}_\Gamma(g, f)$, 于是

$$g'(\Delta) = (g(\delta), \varphi \circ g \circ f) = (\gamma, \varphi \circ g \circ f)$$

其中用到 $g(\delta) = \gamma$. 属于 $\mathfrak{E}_{\partial\Gamma}^*$ 要求上式在每个输入处都等于 (γ, φ) ; 取 $\varphi = \text{id}_\Gamma$ 便把累加器的相等转化为 $g \circ f = \text{id}_\Gamma$, 而该条件反过来又对每个 φ 给出累加器的相等. 最后, $(\varphi \circ g \circ f)(\gamma) = \varphi(g(\delta)) = \varphi(\gamma)$. $\square$

因此, 下三角只有在 γ 处被见证的那个逆于每个状态都逆转 f 时才闭合, 所以 effect 并不把 $\mathfrak{E}_\Gamma^*$ 映入 $\mathfrak{E}_{\partial\Gamma}^*$. 在任何情形下都成立的是在 γ 处的一致: $\text{recover}_\Gamma(g'(\Delta)) = \text{recover}_\Gamma(\gamma, \varphi)$, 而这正是定理 3.7 对一个累加器所要求的全部, 故撤销不会触及恢复的目标。

按与施加相反的顺序撤销副作用不需要任何进一步的条件, 因为此时每个逆所遇到的, 正是其自身施加所产生的状态:

定理 3.16. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 从 $(\gamma_0, \text{id}_\Gamma)$ 出发依次施加, 并按相反顺序撤销. 则:

1. 每次撤销都恢复其施加时所针对的上下文状态;
2. 每个中间状态都满足可靠性不变量。

证明. 每一步都是一次施加或一次撤销. 一次施加把 (γ, φ) 带到 $(\delta, \varphi \circ g)$ 且 $g(\delta) = \gamma$, 故由定理 3.7 它保持 $\varphi(\gamma)$, 而该定理的假设恰是 $\mathfrak{E}_\Gamma^*$ 的见证. 按相反顺序撤销时, 每个逆所得到的的是其自身施加所产生的状态, 故由定理 3.15, 那次撤销精确地恢复前一个状态, 并同样保持 $\varphi(\gamma)$; 两个结论都不依赖于逆所收到的累加器. $\square$

3.1.3 副作用的独立性

在其自身施加所产生的状态处撤销一个副作用, 这是定理 3.16 所覆盖的情形; 在任何其他状态处撤销, 则是本小节的主题. 有两种情形需要后者. 其一, 一个逆可能在后续副作用仍然在位时运行, 这正是从一个运行中的系统撤离 (withdrawal) 一个组件的含义; 其二, 一个序列可能交错多个组件的副作用, 各自保留各自的逆, 于是一个组件的诸逆被另一个组件的施加所分隔. 在这两种情形中, 逆都会遇到一个被外来副作用移动过的状态, 而它是否仍能撤销它原本要撤销的东西, 是一个交换性问题: 必须交换的, 是一个副作用所能实施的每个变换与另一个副作用所能实施的每个变换, 前向映射与所产出的逆一律如此. 单个累加器两种情形都解决了, 因为 φ 是一个按单一顺序、一次性地运行其持有的全部逆的复合。

定义 3.17. 对一个副作用函数 $e \in \mathfrak{E}_\Gamma$, 变换幺半群 $\mathfrak{M}(e)$ 是 $\Gamma \rightarrow \Gamma$ 中由 e 的前向映射连同 e 所产出的每个逆生成的子幺半群, 而 $\mathfrak{M}(e)$ 的生成元即该生成集的元素:

$$\mathfrak{M}(e) := \langle \{ \text{pr}_1 \circ e \} \cup \{ \text{pr}_2(e(\gamma)) \mid \gamma \in \Gamma \} \rangle \quad (3.14)$$

由对 $(f, g) \in \mathfrak{T}_\Gamma$ 诱导的副作用以 f 和 g 为生成元, 它在每个状态处给出的逆都是 g 。

引理 3.18. 交换性在生成元上即可判定, 且 $\diamond$ 不会扩大任何变换幺半群. 即:

1. 若 $\mathfrak{M}(e_1)$ 的每个生成元与 $\mathfrak{M}(e_2)$ 的每个生成元交换, 则 $\mathfrak{M}(e_1)$ 的每个元素与 $\mathfrak{M}(e_2)$ 的每个元素交换;
2. $\mathfrak{M}(e_1 \diamond e_2) \subseteq \langle \mathfrak{M}(e_1) \cup \mathfrak{M}(e_2) \rangle$.

证明. 1. 与 $\mathfrak{M}(e_2)$ 的每个生成元交换的那些映射构成 $\Gamma \rightarrow \Gamma$ 的一个子幺半群, 因为 id_Γ 属于它, 且 f 与 f' 都属于时 $f \circ f'$ 也属于。由假设, 该子幺半群包含 $\mathfrak{M}(e_1)$ 的生成元, 从而包含 $\mathfrak{M}(e_1)$ 。固定 $f \in \mathfrak{M}(e_1)$, 与 f 交换的那些映射同样构成一个子幺半群, 它包含 $\mathfrak{M}(e_2)$ 的生成元, 从而包含 $\mathfrak{M}(e_2)$ 。

2. 由定义 3.9, $e_1 \diamond e_2$ 的前向映射是 $(\text{pr}_1 \circ e_1) \circ (\text{pr}_1 \circ e_2)$, 而它在任意状态处产出的逆是 $s \circ t$, 其中 s 由 e_2 产出、 t 由 e_1 产出。因此 $\mathfrak{M}(e_1 \diamond e_2)$ 的每个生成元都是二者生成元的复合。

□

定义 3.19. 副作用函数 $e_1, e_2 \in \mathfrak{E}_\Gamma$ 是独立的, 若:

1. 一个的每个变换与另一个的每个变换交换,

$$\forall f \in \mathfrak{M}(e_1), g \in \mathfrak{M}(e_2). \quad f \circ g = g \circ f \quad (3.15)$$

2. 一个的变换不打扰另一个所产出的逆,

$$\forall g \in \mathfrak{M}(e_2), \gamma \in \Gamma. \quad \text{pr}_2(e_1(g(\gamma))) = \text{pr}_2(e_1(\gamma)) \quad (3.16)$$

以及把 e_1 与 e_2 互换后的同样条件。

族 $(e_l)_{l \in L}$ 两两独立 (pairwise independent) 是指对所有 $l \neq l'$, e_l 与 $e_{l'}$ 独立。一个族可以重复同一个副作用函数, 而让一个副作用函数与自身独立, 就是要求 $\mathfrak{M}(e)$ 交换。

对于由对 (f_1, g_1) 与 (f_2, g_2) 诱导的副作用, 条款 (1) 依引理 3.18(1) 就是四对 $f_1, f_2; g_1, g_2; f_1, g_2; g_1, f_2$ 的交换性, 而条款 (2) 自动成立, 因为诱导副作用在每个状态处给出同一个逆。◇ 之下的交换则是另一种性质。 $e_1 \diamond e_2 = e_2 \diamond e_1$ 所等同的, 是两种顺序的复合前向映射彼此之间、以及两种顺序的复合逆彼此之间, 每个逆都在其自身施加所产生的状态处进入复合; 独立性则把一个副作用的每个变换与另一个副作用的每个变换相关联, 前向映射与外来逆的配对也不例外。

在独立性之下, 一个逆可以在后续副作用已经移动过的状态上运行, 而它在那里撤销的是它自己的贡献, 别无其他:

定理 3.20. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 两两独立, 并从 γ_0 出发依次施加。记 $f_i := \text{pr}_1 \circ e_i$, 令 $\delta_i := f_i(\delta_{i-1})$ 且 $\delta_0 := \gamma_0$, 并令 $g_i := \text{pr}_2(e_i(\delta_{i-1}))$ 为 e_i 在其施加处所产出的逆。固定 j , 并对去掉 e_j 的序列的状态记 $\delta'_i := (f_i \circ \dots \circ f_{j+1})(\delta_{j-1})$, 于是 $\delta'_j = \delta_{j-1}$ 。则对每个满足 $j \leq u \leq n$ 的 u :

1. $\delta_u = f_j(\delta'_u)$ 且 $g_j(\delta_u) = \delta'_u$;

2. 每个满足 $i > j$ 的 e_i 在 δ'_{i-1} 处给出与它在 δ_{i-1} 处相同的逆 g_i 。

证明. 1. 第一个等式是对 u 的归纳。在 $u = j$ 处它读作 $\delta_j = f_j(\delta_{j-1})$, 这正是 δ_j 的定义。归纳步: $\delta_{u+1} = f_{u+1}(\delta_u) = f_{u+1}(f_j(\delta'_u)) = f_j(f_{u+1}(\delta'_u)) = f_j(\delta'_{u+1})$, 其中间的等式是定义 3.19 对 e_{u+1} 与 e_j 的条款 (1), 由于 $u+1 > j$, 二者是族中互异的副作用。对第二个等式, 条款 (1) 把 g_j 穿过 e_j 之后施加的诸前向映射带出去, 把 e_j 的见证留给它唯一成立的状态使用:

$$g_j(\delta_u) = (g_j \circ f_u \circ \dots \circ f_{j+1})(\delta_j) = (f_u \circ \dots \circ f_{j+1})(g_j(f_j(\delta_{j-1}))) = \delta'_u$$

最后一个等式依赖 $g_j(f_j(\delta_{j-1})) = \delta_{j-1}$, 即定义 3.8 对 e_j 在 δ_{j-1} 处所要求的见证。

2. 由 (1), 状态 δ_{i-1} 是 $f_j(\delta'_{i-1})$, 而 $f_j \in \mathfrak{M}(e_j)$, 故定义 3.19 对 e_i 与 e_j 的条款 (2) 给出 $\text{pr}_2(e_i(f_j(\delta'_{i-1}))) = \text{pr}_2(e_i(\delta'_{i-1}))$ 。

□

条款 (1) 定位了一个逆所到达的状态：它是同一序列在该副作用从未被施加的情况下将到达的状态，无论其后施加过什么。条款 (2) 定位了其余副作用在那里持有的逆，两者合起来，使该定理可以再次应用于更短的序列：

推论 3.21. 设 $e_1, \dots, e_n \in \mathfrak{C}_\Gamma^*$ 两两独立，并从 γ_0 出发依次施加， $g_1, \dots, g_n$ 如上。在 δ_n 处按 $\{1, \dots, n\}$ 的任意一个置换的顺序施加这 n 个逆，都到达 γ_0 。

证明. 对 n 作下降归纳。设该置换以 j 开头。由定理 3.20(1)，在 δ_n 处施加 g_j 到达 δ'_n ，即去掉 e_j 的序列所到达的状态；由定理 3.20(2)，其余副作用在那里产生的逆正是手中的那些 g_i 。该序列作为子族两两独立，故归纳假设适用于它以及置换的其余部分；空序列到达 γ_0 。□

LIFO（后进先出）序是这样一个置换，而定理 3.16 在无任何假设的情况下就按它撤销。独立性买到的是所有其他顺序，随之而来的还有交错多个组件的序列，第 4.4.2 节把它推展为整个系统的一条迹。

综合起来，这些构造构成了可逆副作用： $\mathfrak{C}_\Gamma^*$ 中的每个副作用函数显式提供自己的逆，effect 在副作用上下文 $\partial\Gamma$ 上追踪这些逆，而 $\diamond$ 运算在保持可逆性的同时复合它们。它们所交付的是局部的时间性可组合，所谓局部，是指该保证只从单个组件自身施加的副作用读出。我们把它当作如下判据：对组件施加的任何副作用函数序列，累加器都恢复其出发时的上下文（定理 3.7），且按相反顺序撤销该序列时，每个逆所得到的都是其自身施加所针对的状态（定理 3.16）。加载一个组件，就是施加这样一个序列并把它的诸逆累积于 φ ；卸载它，就是施加 φ 。

这一判据遗漏了两件事，而两者都在多个组件参与时到来：按累加器所强加之顺序以外的顺序撤销，以及交错他人副作用的序列。独立性把它们交付出来（推论 3.21），而它是关于副作用的一个条件，并非构造本身的性质——第 3.3.2 节给出满足这一条件的纪律，第 4.4.2 节则从整个系统的迹读出该保证。在独立性失效之处，顺序必须由别处承载：组件内部由累加器承载，它无论副作用如何都按 LIFO 序撤销（第 4.3.2 节）；组件之间则由一个声明的协副作用承载，它把一次激活针对另一次激活进行排序（第 4.3.1 节）。

3.2 反应式协副作用

空间性可组合是指组件能够彼此声明依赖，且系统能够在运行时解析、提供并撤销这些依赖的能力。这要求每当共享上下文发生变化时就重新评估依赖的满足情况，使得一个组件在其依赖变为可用时激活，在依赖被撤销时去激活。因此，我们把组件的依赖建模为一个规格说明，并依据该规格说明把上下文的每次变化分类为激活、去激活或中性。依据规格说明进行分类，检测的是满足情况的变化；对该分类作出响应，驱动的是激活与去激活。我们称这类协副作用为反应式协副作用（reactive coeffects）：通过分类上下文变化并据此驱动激活与去激活，正确的协副作用次序便成为一项结构性保证。

3.3 协副作用上下文

传统的控制反转（inversion-of-control, IoC）容器 [38] 通常将依赖建模为简单的键值映射。本节将 IoC 形式化为协副作用上下文（coeffect context），使之与可逆副作用（reversible effects）协

同作用，为动态组合提供数学基础。

定义 3.22. 给定类型族 (type family) $\mathcal{V} : K \rightarrow \text{Type}$ ，将副作用上下文定义为依赖偏函数类型 (dependent partial function type)：

$$\Sigma := (k : K) \multimap \mathcal{V}_k \quad (3.17)$$

其中 $\sigma : \Sigma$ 是一个有限偏函数，为每个 $k \in \text{dom}(\sigma) \subseteq K$ 赋予一个类型为 $\mathcal{V}_k$ 的值。我们记：

- $\sigma(k)$ 表示函数应用（当 $k \in \text{dom}(\sigma)$ 时有定义）；
- $\sigma[k \mapsto v]$ 表示表绑定，即在 k 处绑定 v ，其余处与 σ 一致；
- $\sigma \setminus k$ 表示限制（当 $k \in \text{dom}(\sigma)$ 时有定义）；
- $k \in \text{dom}(\sigma)$ 表示成员属于。

使用类型族 $\mathcal{V}$ 确保每个依赖键 k 都关联到一个特定的值类型 $\mathcal{V}_k$ ，为依赖访问提供静态类型安全。扩展与限制都带有前置条件，由下面的操作施加：一个依赖不能被提供两次（扩展要求 $k \notin \text{dom}(\sigma)$ ），也不能在缺席时被撤销（限制要求 $k \in \text{dom}(\sigma)$ ）。被违反的前置条件作为错误发出信号且不产生任何转移，因此描述实际发生的转移的副作用代数 (effect algebra) 对这些操作原样适用。偏好将失败内化的读者，可以把下文的每个 $\Sigma \multimap \Sigma$ 读作 $\Sigma \rightarrow \text{Maybe}(\Sigma)$ ，并在 **Maybe** 单子 (monad) 中复合 (第 2.1 节)，代价是将每个恒等映射换成该操作定义域上的偏恒等映射。基于这一上下文结构，我们定义两个核心操作：

定义 3.23. Σ 上的 *get* 与 *set* 操作定义如下：

$$\begin{aligned} \text{get} & : (k : K) \rightarrow \Sigma \multimap \mathcal{V}_k \\ \text{get} & = k \mapsto \sigma \mapsto \sigma(k) \\ \text{set} & : (k : K) \times \mathcal{V}_k \rightarrow \Sigma \multimap \Sigma \times (\Sigma \multimap \Sigma) \\ \text{set} & = (k, v) \mapsto \sigma \mapsto (\sigma[k \mapsto v], \lambda \sigma'. \sigma' \setminus k) \end{aligned} \quad (3.18)$$

其中 $\text{get}(k)$ 以 $k \in \text{dom}(\sigma)$ 为前置条件， $\text{set}(k, v)$ 以 $k \notin \text{dom}(\sigma)$ 为前置条件。

值得注意的是， $\text{set}(k, v)$ 的类型为 $\mathfrak{E}_\Sigma^*$ ，恰是副作用上下文上的一个副作用函数 (effect function)。因此我们可以直接套用第 3.1 节的副作用机制： $\mathfrak{E}_\Sigma$ 提供了对依赖注册的自动追踪与恢复。这正是反应式副作用 (reactive coeffects) 与可逆副作用之间的协同：副作用操作是副作用，而副作用是可逆的。

get 交给组件的是一个值，而组件能用这个值做的事情，取决于该键上的副作用所提供的內容。因此，一个键承载的不止是值类型：

定义 3.24. 键 k 上的副作用是一个三元组 $(\mathcal{V}_k, \simeq_k, \mathcal{A}_k)$ ，其中 $\mathcal{V}_k$ 是定义 22 的值类型， $\simeq_k$ 是 $\mathcal{V}_k$ 上的一个等价关系，键 k 上的值依之进行比较 (第 3.3.2 节)， $\mathcal{A}_k$ 是副作用操作的集合，即绑定在 k 上的值向持有它的组件提供的操作。一个操作 $a \in \mathcal{A}_k$ 带有参数类型 X_a 与结果类型 B_a ，且仅作用于值本身：

$$a : X_a \rightarrow \mathcal{V}_k \multimap \mathcal{V}_k \times (\mathcal{V}_k \multimap \mathcal{V}_k) \times B_a \quad (3.19)$$

其前两个成分构成 $\mathcal{V}_k$ 上的一个副作用函数，并按定义 8 的要求带有见证，第三个成分是一个结果 (outcome)。

要求每个操作尊重 $\simeq_k$ ：在 $\simeq_k$ 相关的值上，它要么在两处都有定义，要么在两处都无定义；在有定义之处，它给出 $\simeq_k$ 相关的后继、把 $\simeq_k$ 相关的值再次送到 $\simeq_k$ 相关的值的逆，以及相等的结果。操作经由其提升 (lift) 作用于协副作用上下文：

$$a^\Sigma(x)(\sigma) := \text{let}(v, g, b) = a(x)(\sigma(k)) \text{ in } (\sigma[k \mapsto v], \lambda\sigma'. \sigma'[k \mapsto g(\sigma'(k))], b) \quad (3.20)$$

它在 $k \in \text{dom}(\sigma)$ 时有定义，其前两个成分是 Σ 上的一个副作用函数。

把 k 的操作类型化为作用于 $\mathcal{V}_k$ ，正是将它限制在 k 处的绑定之内：提升只读写该绑定，而让其余每个键保持原样，因此无需附加条件来说明这一点。在隔离 (isolation) 生效之处，它触及的绑定是域 (realm) 所解析到的那一个 (定义 28)，共享同一个域的两个键共享同一个绑定。一个其行为系于另一个键的操作，会把该键的值读入其参数 X_a ，而下一小节的反应式纪律，正是让读它的组件运行的整个期间该值保持固定 (定理 63)。

3.4 规约与通知

前面的定义描述了单个依赖如何被注册与访问。然而，访问一个缺席的依赖是运行时失败。因此，组件应当只在它声明的所有依赖全部在场之后才激活，而不是乐观地访问它们并在缺失时失败。这引出两个问题：组件声明的依赖是否联合得到满足，以及当这一状态变化时系统应如何响应。协副作用上下文 Σ 带有一个自然的观测结构，使两个问题皆可处理：对任意协副作用规约 $d \subseteq K$ ，定义满足谓词 (satisfaction predicate)：

$$\sigma \models d := \forall k \in d. k \in \text{dom}(\sigma) \quad (3.21)$$

该谓词是可判定的 (因为 $\text{dom}(\sigma)$ 有限)。由于对 σ 的所有变更都经过副作用函数 (其逆恢复先前的定义域)，满足情况的变化在每个副作用边界处都可检测。这是反应性的代数基础：副作用系统保证每个协副作用变化都被观测到。

定义 3.25. 协副作用规约 (coeffect specification) 为：

$$\mathfrak{D}_\Sigma := \text{Set}(K) \quad (3.22)$$

表示组件从环境声明的依赖集合。

使这一规约具有反应性的，是它对状态转移的分类方式。任何把 σ 变换到 σ' 的副作用，都可依 $d \in \mathfrak{D}_\Sigma$ 是否被改变了满足状态，而由规约分类：

定义 3.26. 给定协副作用规约 $d \subseteq K$ 与状态 $\sigma, \sigma' \in \Sigma$ ，定义：

$$\text{notify}_d(\sigma, \sigma') := \begin{cases} \text{激活} & \text{若 } \sigma \not\models d \wedge \sigma' \models d \\ \text{停用} & \text{若 } \sigma \models d \wedge \sigma' \not\models d \\ \text{中立} & \text{否则} \end{cases} \quad (3.23)$$

此定义良定，因为 $\sigma \models d$ 可判定，且所有状态转移都由副作用函数介导。反应式不变量为：激活转移触发组件副作用的执行 (带完整的副作用追踪)，而停用转移通过应用累加器 (accumulator) 触发恢复。这些转移的精确操作语义取决于它们与控制流的交互，在第 4 章展开。

set 与 notify 共同交付的是局部空间性可组合 (spatial composability)，局部与此前含义相同，即该保证只就单个组件自身的协副作用来读。我们将其取为如下判据：组件仅在满足其规约的状态处激活，因此它绝不会读取一个缺席的绑定；且上下文的每次变更都对照该规约分类，因

此满足性的丧失在发生之处即被检测并驱动一次停用。两半都立即可由上述定义得出：满足性是在组件将激活之处检查的前置条件，而 notify_d 在每次转移处都有定义。

该判据覆盖协副作用排序的一个方向，而不覆盖另一个。若组件 A 提供键 k ，而组件 B 声明 $k \in d_B$ ，则 B 只能在 A 激活并提供 k 之后激活，因为 $\sigma \models d_B$ 要求 $k \in \text{dom}(\sigma)$ 。反向则不成立：卸载 A 将 k 从 $\text{dom}(\sigma)$ 移除，从而破坏 B 的满足性，但通知本身既不能在 B 自身的 teardown 需要键 k 可读的整个期间维持其可读，也不能把 A 的恢复推迟到 B 完成之后。将被引发的停用排在其后的撤离（*withdrawal*）排序，是施加于其他组件而非行动组件的条件，因此属于该保证的全局形式，第 4.3.1 节提供了它所需的机制。

3.5 隔离与拦截

基本的协副作用上下文 Σ 建模一张扁平的依赖表。然而实践中，系统可能需要为不同组件将不同的值绑定到同一逻辑依赖上。本节以两个机制扩展协副作用上下文：协副作用隔离（*coeffect isolation*，同一键在不同上下文中解析不同）与协副作用拦截（*coeffect interception*，依赖访问上的横切行为）。

实现（realization）。这两个机制与 get 和 set 的差别在于其作用对象。一次提供写入的是每个组件都读取的共享表，因此它是该表上的副作用，并带有撤销它的逆。隔离与拦截则调整一个键在某一上下文之下的组件处的解析方式，而让表本身保持原样。将一个操作类型化为副作用，固定的是它的指称——后继状态与逆的配对——而非它的实现，后者决定该逆如何被执行。

定义 3.27. 上下文上的副作用函数容许两种实现：

- 原地实现（*in-place realization*）原地修改上下文并返回一个非平凡逆；后继与输入互为别名，恢复执行该逆以撤销修改。
- 派生实现（*derived realization*）保持输入不动，返回一个由它派生的新鲜上下文，以恒等映射为其逆；恢复丢弃该派生上下文。由另一上下文派生的上下文，正是定义 3.2 的递归结构所承载的东西。

在纯函数式场景中二者重合，命令式宿主可逐操作任选其一；第 5.1.2 节实现了两者。隔离与拦截径直采用派生实现：各自产生一个自有表不同于所继承表的新鲜上下文，因此下面将它们类型化为从上下文到上下文的映射，而非副作用函数。共享表没有任何变化，因此没有逆需要追踪，也没有什么东西可供定义 12 提升，恢复将派生上下文连同它所携带的调整一并丢弃。派生表上的赋值覆盖继承表在该键上原先持有的内容，这就是两个操作都不带前置条件的原因。

协副作用隔离。通过引入隔离域（*isolation realm*），协副作用隔离允许同一依赖在不同上下文中绑定到不同的值。这在多租户系统、测试环境与组件沙箱化（*sandboxing*）中有广泛的应用。

定义 3.28. 定义带隔离的协副作用上下文为：

$$\Sigma^{\text{iso}} := (K \multimap R) \times ((r : R) \multimap \mathcal{V}_r) \quad (3.24)$$

它可表示为一对 (ρ, σ) ，其中：

- $\rho : K \multimap R$ 是隔离域表，为每个被隔离的键赋予一个域标识符； $\text{dom}(\rho)$ 之外的键解析到它自身的域，故在该处我们写 $\rho(k) = k$ （ $R \supseteq K$ ）；

- $\sigma : (r : R) \rightarrow \mathcal{V}_r$ 是依赖表，一个从域标识符到带类型值的偏依赖函数。

两层映射结构将逻辑层与存储层解耦，使依赖访问具备上下文感知。访问键 k 时，系统先解析 $\rho(k)$ 得到域标识符 r ，再访问 $\sigma(r)$ 取得实际值。

定义 3.29. Σ^{iso} 上的 *get*、*set* 与 *isolate* 操作为：

$$\begin{aligned}
\text{get} & : (k : K) \rightarrow \Sigma^{\text{iso}} \rightarrow \mathcal{V}_{\rho(k)} \\
\text{get} & = k \mapsto (\rho, \sigma) \mapsto \sigma(\rho(k)) \\
\text{set} & : (k : K) \times \mathcal{V}_{\rho(k)} \rightarrow \Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}} \times (\Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}}) \\
\text{set} & = (k, v) \mapsto (\rho, \sigma) \mapsto ((\rho, \sigma[\rho(k) \mapsto v]), \lambda(\rho', \sigma'). (\rho', \sigma' \setminus \rho'(k))) \\
\text{isolate} & : K \times R \rightarrow \Sigma^{\text{iso}} \rightarrow \Sigma^{\text{iso}} \\
\text{isolate} & = (k, r) \mapsto (\rho, \sigma) \mapsto (\rho[k \mapsto r], \sigma)
\end{aligned} \tag{3.25}$$

其中 *get* 与 *set* 承载定义 23 的前置条件沿 ρ 迁移后的形式，即 $\rho(k) \in \text{dom}(\sigma)$ 与 $\rho(k) \notin \text{dom}(\sigma)$ 。*isolate*(k, r) 派生的上下文把域 r 赋给 k ，并原样继承依赖表，因此一个已被隔离的键是被重新赋值，而非被拒绝。

协副作用隔离机制实质上实现了一个运行时特设多态 (ad-hoc polymorphism) 系统。借助隔离域标识符，同一依赖键可以在不同上下文中解析到完全不同的值，且这种多态可在运行时动态调整。与传统依赖注入相比，协副作用隔离提供更细粒度的控制，可为特定组件定制隔离；*set* 仍是一个副作用函数 ($\mathfrak{C}_{\Sigma^{\text{iso}}}^*$)，从而继承可逆性，而 *isolate* 无需可逆性——它派生一个上下文，而非写入共享表。

协副作用拦截。第二个机制即协副作用拦截，在依赖访问上附加横切元数据，在不修改依赖值的前提下添加行为。这种元数据既可以由上下文携带，也可以由组件声明，因此我们同时扩展协副作用上下文与协副作用规约：

定义 3.30. 定义带拦截的协副作用上下文与规约为：

$$\begin{aligned}
\Sigma^{\text{inter}} & := ((k : K) \rightarrow \mathcal{M}_k) \times ((k : K) \rightarrow (\mathcal{M}_k \rightarrow \mathcal{V}_k)) \\
\mathfrak{D}^{\text{inter}} & := (k : K) \rightarrow \mathcal{M}_k
\end{aligned} \tag{3.26}$$

上下文 Σ^{inter} 是一对 (ι, σ) ： ι 是安装在上下文自身上的上下文携带元数据，默认为空 (ϵ_k)； σ 将每个键 k 映到一个从元数据 $\mathcal{M}_k$ 到值 $\mathcal{V}_k$ 的提供方函数。规约 $d \in \mathfrak{D}^{\text{inter}}$ 携带组件声明的元数据，为每个键赋予其元数据 $d(k)$ ，并以 $\text{dom}(d)$ 充当依赖集合。每个键为其元数据配备一个幺半群 (*monoid*) $(\mathcal{M}_k, \oplus_k, \epsilon_k)$ ：合并 $\oplus_k$ 结合，单位元为 ϵ_k (空元数据)。

定义 3.31. Σ^{inter} 上的 *get*、*set* 与 *intercept* 操作为：

$$\begin{aligned}
\text{get} & : (k : K) \times \mathcal{M}_k \rightarrow \Sigma^{\text{inter}} \rightarrow \mathcal{V}_k \\
\text{get} & = (k, \mu) \mapsto (\iota, \sigma) \mapsto \sigma(k)(\mu \oplus_k \iota(k)) \\
\text{set} & : (k : K) \times (\mathcal{M}_k \rightarrow \mathcal{V}_k) \rightarrow \Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}} \times (\Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}}) \\
\text{set} & = (k, \psi) \mapsto (\iota, \sigma) \mapsto ((\iota, \sigma[k \mapsto \psi]), \lambda(\iota', \sigma'). (\iota', \sigma' \setminus k)) \\
\text{intercept} & : (k : K) \times \mathcal{M}_k \rightarrow \Sigma^{\text{inter}} \rightarrow \Sigma^{\text{inter}} \\
\text{intercept} & = (k, \nu) \mapsto (\iota, \sigma) \mapsto (\iota[k \mapsto \iota(k) \oplus_k \nu], \sigma)
\end{aligned} \tag{3.27}$$

其中 *get* 与 *set* 在提供方表上承载定义 23 的前置条件，即 $k \in \text{dom}(\sigma)$ 与 $k \notin \text{dom}(\sigma)$ 。*intercept*(k, ν) 派生的上下文把 ν 合并到 k 处继承的元数据上，并原样继承提供方表。

当规约为 d 的组件访问键 k 时，系统求值 $\sigma(k)(d(k) \oplus_k \iota(k))$ ：组件声明的元数据与上下文携带的元数据 ι 合并，再将提供方函数应用于其结果。这一合并遵循每个键自身的语义（例如标量字段被覆盖、集合值字段取并），且是右偏的，故 $\iota(k)$ 优先，并可以覆盖组件的声明，使一个外层上下文无需修改组件即可约束该组件对协副作用的使用（例如第 6.3 节）。

3.6 上下文范式

第 3.1 节与第 3.2 节各自作用于一个上下文，前者作为副作用的载体，后者作为协副作用的载体，而同时承载两者的单一上下文是什么样子仍未回答。本节为这一统一给出具体构造，从协副作用中组装出一个观测等价（observational equivalence），以提供第 3.1.3 节留开的副作用独立性，并论证所得的上下文类型本身就构成一种编程范式。

3.6.1 统一上下文

对上下文 Γ ，副作用上下文（effect context） $\partial\Gamma$ （第 3.1 节）提供更高层的抽象，携带上一层的上下文与该层的累加器（定义 2）。将这一结构递归化，并与协副作用上下文 Σ 结合，得到如下的类型：

定义 3.32. 上下文类型 Γ_∞ 定义为：

$$\Gamma_\infty := \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma \quad (3.28)$$

其三个投影为：

- Γ ：当前上下文状态（递归）；
- $\Gamma \rightarrow \Gamma$ ：累加器，恢复本层的副作用；
- Σ ：携带依赖信息的协副作用上下文。

在该定义之下，副作用把 $\mathfrak{C}_{\Gamma_\infty}$ 映到其自身，将 ∂ -塔统一为单个自相似的类型。协副作用上下文 Σ 被结构性地整合进来：依赖操作（set、get）作用于 Σ ，累加器追踪它们的逆转。由于 Σ 底层的类型族 $\mathcal{V}$ 不受约束，系统需要在组件间共享的任何状态，都可以带上适当的值类型编码为一个依赖—— Σ 涵盖所有共享可变状态，而不只是组件间依赖。组件与其环境之间的每次交互都经过这一单一实体。

层级式组合。 Γ_∞ 的递归结构支持层级式控制：父上下文聚合多个子层副作用，形成树状控制结构，在保持模块性的同时实现统一的跨层管理。副作用变换实现了字面意义上的“即插即用”比喻：

- 加载一个组件对应于执行其副作用（插入）；
- 卸载一个组件对应于恢复其副作用（拔出，且不影响其他运行中的组件）；
- 层级中各层的组件可独立加载与卸载；父上下文聚合并管理其所有子组件的副作用，支持任意嵌套的组合。

3.6.2 观测等价

第 3.1 节的恢复保证断言的是状态相等（定理 7），这是一种理想化，因为物理状态无法恢复如初。例如，free 将一个块归还给分配器，并不恢复 malloc 之前堆的布局；生成式名称也不会因

丢弃它的逆而恢复，因为下一次创建会取出一个新名称 [39]。因此，第 3 章中的诸等式应在一个等价关系 $\simeq$ 之下解读，而我们取 $\simeq$ 为一个观测等价：当没有观察者能区分两个状态时，它们相关。比较行为而非表示，是通往程序等价的既定途径 [40]，而这种比较所得到的关系，取决于观察者被给予了什么来工作 [41]。上下文的观察者被给予的，是它所携带的协副作用，其中每个都自带一个等价关系（定义 24），因此上下文上的关系由它们组装而成。组装它是本小节的任务，而按其作商，正是买到第 3.1.3 节所要独立性的方式。

定义 3.33. 两个协副作用上下文在将相同的键绑定到相关的值时相关，上下文两个状态在其协副作用投影相关时相关：

$$\begin{aligned}\sigma &\simeq \sigma' &:=& \text{dom}(\sigma) = \text{dom}(\sigma') \wedge \forall k \in \text{dom}(\sigma). \sigma(k) \simeq_k \sigma'(k) \\ \gamma &\simeq \gamma' &:=& \sigma_\gamma \simeq \sigma_{\gamma'}\end{aligned}\tag{3.29}$$

其中 σ_γ 记 γ 的协副作用投影（定义 32）。

状态的没有被任何键绑定的部分因此被遗忘，而遗忘它，才使定理 7 能够在 $\simeq$ 之下解读：上例中的堆布局与生成式名称落在关系之外，除非某个键绑定了它们。第 3.2.2 节对 $\simeq$ 的所需是推论而非假设。相关的状态有相同的定义域，因此它们在满足谓词 $\sigma \models d$ 与定义 26 的分类 notify 上一致，而反应性是 $\Sigma/\simeq$ 的性质。

称该关系为观测的，是关于每个 $\simeq_k$ 的一个断言，即它区分的不多于 k 的操作所能分辨的。值的观察者运行这些操作并读取其结果。

定义 3.34. 设 V 带有定义 24 意义下的操作集 $\mathcal{A}$ ，并对每个参数 $x : X_a$ ，将 $\mathfrak{M}(a)$ 记为副作用函数 $a(x)$ 的变换幺半群（*transformation monoid*，定义 17）。 $\mathcal{A}$ 上的一个测试（*test*）是诸幺半群 $\mathfrak{M}(a)$ （ $a \in \mathcal{A}$ ）的生成元上的一个有限字，每个字母应用于此前诸字母留下的值；其结果是其中作为操作前向映射的字母沿途给出的结果，而在前置条件失败之处它无定义。值 $v, v' : V$ 不可区分，记作 $v \approx_{\mathcal{A}} v'$ ，当 $\mathcal{A}$ 上的每个测试在两处皆有定义或皆无定义，且在两处给出相同的结果。

引理 3.35. 不可区分性是诸操作所尊重的最粗关系。即：

1. $\mathcal{A}$ 的每个操作按定义 24 的意义尊重 $\approx_{\mathcal{A}}$ ；
2. $\mathcal{A}$ 的每个操作都尊重的每个等价关系，都包含于 $\approx_{\mathcal{A}}$ 。

因此， $\simeq_k$ 的每个可接受的选择都包含于 $\approx_{\mathcal{A}_k}$ ，且 $\approx_{\mathcal{A}_k}$ 本身可接受。

证明. 1. 设 $v \approx_{\mathcal{A}} v'$ ，并将 $a \in \mathcal{A}$ 应用于某个参数。在一个测试前添加一个字母仍是测试，因此前向映射所到达的值不可区分，任一给出的逆从不可区分的参数出发所到达的值亦然；单字母测试给出两处皆有定义或皆无定义，以及结果相等。

2. 设 R 是这样一个等价关系，且 $v R v'$ 。测试的每个字母是某个操作的前向映射或给出的逆，而尊重性使 R 沿两者保持，使到达的值保持相关，且每个字母处结果相等。因此，每个测试在 v 与 v' 处一致。

□

全程把 $=$ 换成 $\simeq$ 本身还不够，因为副作用函数返回的既有状态也有逆，而 $\simeq$ 所等同的两个状态，还必须给出 $\simeq$ 所等同的逆。

定义 3.36. 映射 $f: \Gamma \rightarrow \Gamma$ 尊重 $\simeq$, 当:

$$\forall \gamma, \gamma' \in \Gamma. \quad \gamma \simeq \gamma' \Rightarrow f(\gamma) \simeq f(\gamma') \quad (3.30)$$

两个映射在每个状态处一致时相关, $\partial\Gamma$ 中的两个对在两个成分都相关时相关:

$$\begin{aligned} f \simeq g &:= \forall \gamma \in \Gamma. f(\gamma) \simeq g(\gamma) \\ (\delta, g) \simeq (\delta', g') &:= \delta \simeq \delta' \wedge g \simeq g' \end{aligned} \quad (3.31)$$

尊重 $\simeq$ 的映射, 是能下降到 $\Gamma/\simeq$ 的映射, 而由 $\simeq$ 相关的两个映射, 是下降到其上同一映射的两个映射。副作用函数两者皆需: 前者使其计算的状态在商上良定, 后者使其返回的逆在商上良定。

定义 3.37. 在 $\simeq$ 之下解读定义 8: $e \in \mathfrak{E}_\Gamma$ 属于 $\mathfrak{E}_\Gamma^*$, 当 e 作为映射 $\Gamma \rightarrow \partial\Gamma$ 尊重 $\simeq$, 且写 $(\delta, g) = e(\gamma)$ 时, 对每个 $\gamma \in \Gamma$:

1. $g(\delta) \simeq \gamma$;
2. g 尊重 $\simeq$ 。

取 $\simeq$ 为 Γ 上的相等, 即回到定义 8。

引理 3.38. 在将 $\mathfrak{E}_\Gamma^*$ 按定义 37 解读之下, 第 3.1 节断言的每个状态等式在把 $=$ 换为 $\simeq$ 后成立, 且从 $(\gamma_0, \text{id}_\Gamma)$ 可达的每个状态的累加器尊重 $\simeq$ 。

证明. 累加器是逆的复合, 每个逆由定义 37(2) 尊重 $\simeq$, 而尊重 $\simeq$ 的映射的复合尊重 $\simeq$, 基例为 id_Γ 。第 3.1 节的证明随后原样通过, 尊重性正是把一个关系带过逆的东西: 由 $g_2(\delta_2) \simeq \delta_1$ 与 $g_1(\delta_1) \simeq \gamma$, 尊重性给出 $(g_1 \circ g_2)(\delta_2) \simeq \gamma$, 这正是逆的复合所走的每一步, 而定理 7 的可靠性不变量按该步读作 $\varphi(\gamma) \simeq \gamma_0$ 。□

定义 19 所要求的交换, 由同一引理在 $\simeq$ 之下解读, 而这样解读才使它可达: 两个操作可以留下被 $\simeq_k$ 等同的值, 而仍算作交换。对两个操作, 它比其提升所诱导的副作用函数多要求一件事, 因为操作还给出一个结果。

定义 3.39. 操作 a 与 a' 独立, 当它们的提升在每个参数对处都作为副作用函数独立 (定义 19), 且任一个的变换都不扰乱另一个给出的结果:

$$\forall x: X_a, g \in \mathfrak{M}(a'^\Sigma), \sigma \in \Sigma. \quad \text{pr}_3(a^\Sigma(x)(g(\sigma))) = \text{pr}_3(a'^\Sigma(x)(\sigma)) \quad (3.32)$$

并将 a 与 a' 互换后同样成立, 其中 $\mathfrak{M}(a^\Sigma)$ 记提升 $a^\Sigma(x)$ 在每个参数上的变换幺半群, 正如定义 34 对操作本身记 $\mathfrak{M}(a)$ 。键 k 交换 (commutative), 当 $\mathcal{A}_k$ 的任意两个操作独立, 一个操作也被视为与自身独立。

跨不同的键, 该条件直接成立。

定理 3.40. 不同键上的操作独立。

证明. 设 a 属于 $\mathcal{A}_k$, a' 属于 $\mathcal{A}_{k'}$, 且 $k \neq k'$ 。由定义 24, $\mathfrak{M}(a^\Sigma)$ 的每个生成元形如 $\sigma \mapsto \sigma[k \mapsto u(\sigma(k))]$, 其中 u 是 $\mathcal{V}_k$ 上的映射, 它或是某个前向映射的提升, 或是某个给出的逆的提升, a' 在 k' 处同理。这样的两个映射交换——各自只读写单独一个键, 而两个键不同——且引理 18(1) 把交换性从生成元推广到两个幺半群。对第二个条件, a^Σ 在 σ 处给出的东西——逆与结果皆然——由 $\sigma(k)$ 决定, 而 $\mathfrak{M}(a'^\Sigma)$ 的每个生成元都让它保持原样。□

其值为一张可独立增删条目之表的键是交换的，路由或事件监听器 (listener) 的注册是代表性情形：任意顺序的两次注册留下的表，对每个测试的回答都相同，且任一注册都可在另一注册保留之时撤销。其值为有序链的键则不然，因为先于另一个插入的中间件会看到不同的请求，且任一顺序都无法在不扰乱另一个的情况下撤销。开篇例子中的分配器，依其接口发布的内容而分。在它给出的句柄不被该键的任何操作比较之处， $\simeq_k$ 可以在句柄重命名之下关联两个堆，这正是 CompCert 关联程序与其翻译之内存状态的方式 [42]，分配是交换的；在地址是被相等性比较的结果之处，没有任何可接受的 $\simeq_k$ 能使两种分配次序一致，该键不交换。

组件所执行的，是一个其中每个操作都可能依赖之前的操作所给出之物的操作序列，而具有这种形状的副作用函数，正是下面定理所谈论的。

定义 3.41. 协副作用介导的副作用函数 (coeffect-mediated effect function) 构成含单位元 η_Σ 且对下述规则封闭的最小集合 $\mathfrak{E}_\Sigma^A \subseteq \mathfrak{E}_\Sigma$ ：对键 k 、操作 $a \in \mathcal{A}_k$ 、参数 $x : X_a$ ，以及成员族 $(e_b)_{b \in B_a}$ ，

$$\sigma \mapsto \mathbf{let}(\delta, s, b) = a^\Sigma(x)(\sigma) \mathbf{in let}(\varepsilon, t) = e_b(\delta) \mathbf{in}(\varepsilon, s \circ t) \quad (3.33)$$

仍是成员。每个阶段执行一个操作，并按其结果选择后继，因此一个参数可能依赖已经获得的结果。一个成员中出现的操作，是它的各阶段在每次结果选择之下所执行的那些操作。

定理 3.42. 设 $e_1, e_2 \in \mathfrak{E}_\Sigma^A$ ，且两者的操作都出现的每个键都交换（定义 39）。则 e_1 与 e_2 独立（定义 19）。

证明. 对定义 41 的构造作归纳， $\mathfrak{M}(e_i)$ 落在 e_i 中出现的操作的生成元所生成的子幺半群中：单位元生成平凡幺半群，而一个阶段是 $a^\Sigma(x)$ 与一个成员的 $\diamond$ -复合，引理 18(2) 对之适用。

对定义 19 的条款 (1)，由引理 18(1)，只需 e_1 中出现的某个操作的生成元，与 e_2 中出现的某个操作的生成元交换。当两个操作在不同键上时，这是定理 40；当它们在同一键上时，该键承载两者的操作，且由假设交换。

对条款 (2)，取 $g \in \mathfrak{M}(e_2)$ ，它是 e_2 中出现的操作的生成元的复合，并对 e_1 的构造作归纳。单位元在每个状态处给出 id_Σ 。在一个阶段处，设 $(\delta, s, b) = a^\Sigma(x)(\sigma)$ 且 $(\varepsilon, t) = e_b(\delta)$ ，从而该阶段在 σ 处给出 $s \circ t$ 。操作的独立性逐个应用于 g 的生成元，给出在 $g(\sigma)$ 处仍是 s 与 b ，因此选择同一后继 e_b ，而条款 (1) 把它运行所自的状态置于 $g(\delta)$ ，归纳假设在该处再次给出 t 。于是，该阶段在 $g(\sigma)$ 处给出 $s \circ t$ 。□

组件与其环境之间的每次交互都经过上下文，而类型族 $\mathcal{V}$ 不受约束，因此系统可以把它在组件间共享的每个位置，绑定到它自己的一个键上（第 3.3.1 节）。组件的副作用函数于是是一个协副作用介导的副作用函数沿协副作用投影的提升，而独立性转移到该提升之上，其变换只移动投影。第 3.1.3 节留开的假设就这样得到满足，随之而来的，是整个组件系统的时间性可组合 (temporal composability)。

该分解所划分的，是计算的交换部分与其次序敏感部分。交换部分由副作用承载：组件以其任务所需的任意次序执行它们，而推论 21 以系统觉得方便的任意次序复原它们，没有两个组件相互约束。次序敏感部分由协副作用承载，因为操作不交换的键，正是其余必须从副作用之外施加次序的键，而有两个可供施加的位置。在单个组件内部，由累加器施加，按 LIFO 次序复原副作用的全部内容（定理 16）。跨组件，则由声明的协副作用施加，一个组件提供另一个所声明的东西，且提供先行于声明的满足（第 3.2.2 节）。可组合性因此在组件的粒度而非单个副作用的粒度上获得，这正是第 4 章工作的尺度。

定理的两个限度值得一提。把每个共享位置绑定到键上，是范式的纪律而非构造的属性，因此系统无法具体化为副作用的位置，落在第6.1节的边界之外，也随之落在定理之外。而键的交换性，是该键所发布接口的属性，因此满足它是对提供该键之组件的义务，而非对消费它的诸组件的义务。

3.6.3 上下文范式的定位

编程范式在如何处理副作用上有着根本差异。两个既立的极点定义了这一谱系：

显式状态穿线（函数式）。为保持引用透明性（referential transparency），纯函数式语言把副作用建模为状态上的显式变换。State 单子 $S \rightarrow (A, S)$ [23] 将一个环境穿线过每个计算。这一途径给出强的组合保证：副作用在类型中可见，且适于等式推理。然而，它带来显著的人体工学代价：调用链中的每个函数都必须接受并返回状态参数，即便它只是原样传递状态。随着副作用维度数目的增长（日志、配置、I/O），单子堆叠或 effect-handler 样板代码激增。

隐式变更（命令式/OOP）。主流命令式语言允许组件修改共享状态，并在调用点无需显式声明地访问依赖。在副作用一侧，代表性的例子是 React 的 `useEffect` 钩子：它在组件内部的纤程（fiber）上注册一个持久的副作用，然而副作用目标与注册机制都不作为显式参数出现——识别依赖隐藏运行时状态中的调用次序位置。在副作用一侧，Java 的服务定位器模式（例如 Spring 的 `ApplicationContext.getBean(...)`）在运行时从进程级注册表检索依赖，每个调用点都需要空值检查与类型转换；依赖关系是隐式的，且散布于整个代码库。更一般地，理解 `f()` 如何修改或依赖系统，需要传递地阅读其实现。重构变得脆弱，因为移动或删除一个调用，可能悄然破坏远处的不变式。

上下文范式将函数式途径的可追踪性与命令式途径的人体工学结合起来。副作用与副作用都经由一个显式的上下文参数介导。因此，每个操作都可归因于它在其上被调用的那个特定上下文，进而归因于该上下文所属的组件。

除了结合两极之长，上下文范式还让开发者逐个处理每个副作用与依赖，并自动将它们组合成系统的行为。对可逆副作用，开发者给出每个原子操作的逆，而任意复合之逆由复合得出（第3.1节），因此组件的 `teardown` 由其加载导出，而非与其并列书写。对反应式副作用，组件只声明它所需的依赖，运行时自动解析并重新接线（第3.2节），在提供方被添加、移除或替换之时保持接线一致。在两个方向上，原本依赖开发者纪律的正确性，都成为范式的结构属性。

第 4 章 动态组合演算

第 3 节仅以局部形式确立了空间性可组合 (spatial composability) 与时间性可组合 (temporal composability)。要将它们推广到整个系统, 需要把系统分解为若干组件 (component), 每个组件将一份协副作用规约与一个见证副作用函数 (witnessed effect function) 配对, 使得与共享环境的每一次交互都可归因于其中之一。下面各节为这一分解给出操作语义 (operational semantics), 并以全局形式确立空间与时间可组合性。

第 4.1 节与第 4.2 节给出可以为生命周期赋予规则的最小演算, 它把每次转换视为原子的、即时的且不会失败的; 第 4.3 节逐一放弃这三条假设——原子性按转换可运行的两个方向各放弃一次——接纳运行时在转换开始与结束之间插入的各种控制流形式, 从而到达真实运行时所实现的演算; 第 4.4 节确立该演算的元理论 (metatheory), 即保持 (preservation)、全局时间与空间可组合性、进展 (progress) 与合流性 (confluence)。

4.1 组件与纤程

本节固定规则所作用的对象: 组件; 纤程 (fiber), 即携带自身生命周期状态的组件的一次实例化; 以及注册表 (registry), 它保存一个状态所携带的纤程, 协副作用上下文即从中读取。

组件。组件以三元组给出, 其协副作用一侧划分为从环境读取的部分与向环境提供的部分。

定义 4.1 (43). 在同时携带副作用与协副作用的上下文 Γ (定义 32) 上的组件定义为:

$$\mathfrak{C}_\Gamma := \mathfrak{D}_\Gamma \times \mathfrak{P}_\Gamma \times \mathfrak{E}_\Gamma^* \quad (4.1)$$

表示三元组 (d, p, e) , 其中:

- $d : \mathfrak{D}_\Gamma$ 是定义 25 的协副作用规约, 声明从环境所需的依赖;
- $p : \mathfrak{P}_\Gamma := \text{Set}(K)$ 是供给 (provision), 声明组件可提供的协副作用键, 且其副作用函数写入的键都在 p 之内;
- $e : \mathfrak{E}_\Gamma^*$ 是定义 8 的见证副作用函数, 定义组件活动时贡献的副作用连同撤离它们的逆。

这两份声明是同一接口的两个方向: d 是组件从环境读取的内容, p 是组件向环境写入的内容, 且第 4.2 节不容许同一注册表中两个纤程的供给相交。全文下标一律取在 Γ 上, 协副作用上下文是它的一个投射 (定义 32), 所以定义 25 的 $\mathfrak{D}_\Sigma$ 在此处写作 $\mathfrak{D}_\Gamma$ 。

供给的不相交性正是本章与 3.2.3 节分道扬镳之处。定义 28 的隔离性 (isolation) 允许一个键通过域表 (realm table) 解析, 于是两个纤程可以在不同域中提供同一个键; 携带域的演算会把不相交性放松为域内不相交, 并按声明该键的纤程所在的域来解析它。我们在此不引入域, 而是在唯一共享的域上读取每个键, 这正是使上面的不相交性成为正确条件、并使每个键的提供方 (provider) 唯一 (定义 45) 的原因。它所限制的是组件可被实例化的次数: 供给非空的组件同一时刻只有一个纤程, 因此下文的多次实例化都是针对不提供任何东西的组件, 这是只消费、或只注册其他组件的组件的常见情形。

在运行系统中实例化的组件随时间被激活与停用, 因此它携带生命周期状态 (lifecycle state), 而转换 (transition) 将其从一个生命周期状态移到另一个: 激活执行 e , 在上下文上积

累副作用；停用施加累加器（accumulator）以恢复上下文。最简形式的生命周期即图 1 的双状态模型，4.2 节为其给出规则；4.3 节随着每个控制流特性被接纳而对其加以细化。

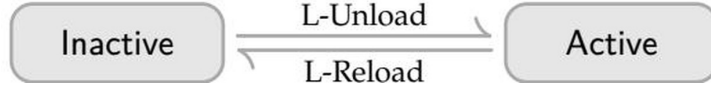


Figure 4.1: 基础组件生命周期

线程。同一个组件可以被实例化多次，每次实例化携带自身的生命周期状态。我们将这样的一次实例化称为线程。线程记录产生它的组件、实例化它时所处的那条线程、它提供的协副作用，以及它处于生命周期中的位置。

定义 4.2 (44). 固定一个线程名的集合 $\mathfrak{N}$ 。实例化组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 的线程是元组 $\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$ ，其中：

- $d : \mathfrak{D}_\Gamma$ 、 $p : \mathfrak{P}_\Gamma$ 与 $e : \mathfrak{E}_\Gamma^*$ 是定义 43 的协副作用规约、供给与副作用函数；
- $\pi : \mathfrak{N} \cup \{\text{root}\}$ 是父引用（*parent*），即本线程实例化时所处的那条线程，或根标记 root ；
- $\sigma : \Sigma$ 是线程自身的协副作用表（定义 22），在激活之前为空，并由其副作用在运行时写入；
- $\tau : \{\perp, \top\}$ 是退役标志（*retirement flag*），新建线程为 $\perp$ ，编排器（*orchestrator*）令线程退役后为 $\top$ ；
- $\theta : \Theta_\Gamma$ 是生命周期状态，在 4.2 节的双状态模型中为

$$\Theta_\Gamma := \text{Inactive} \mid \text{Active}(g, \omega) \quad (4.2)$$

其中 $g : \Gamma \rightarrow \Gamma$ 是累加器， $\omega : d \rightarrow \mathfrak{N}$ 是已提交视图（*committed view*）。

已提交视图 ω 将线程声明的每个键送到转换提交时提供该键的线程名。4.3 节将把 Θ_Γ 替换为进行中的转换所需的扩展；定义 44 的其余部分对两者一次性给出，只是 e 在 4.3 节的每一层引入的更丰富的副作用类型上读取。

注册表。状态以线程名保存其线程，线程的同一性与 3.2 节的协副作用上下文都从这一安排中读取。

定义 4.3 (45). 记 $\mathfrak{F}_\Gamma$ 为 Γ 上线程的集合。状态 $\gamma \in \Gamma$ 携带注册表

$$F_\gamma : \mathfrak{N} \rightarrow \mathfrak{F}_\Gamma \quad (4.3)$$

这是一个有限偏函数，其父指针构成一棵以 root 为根的树，再加上 Γ 中未被任何线程的 σ 命名的其余内容。我们将 $F_\gamma(n)$ 写作 $\gamma(n)$ ，并在状态明确时用下标 n 缩写 $\gamma(n)$ 的字段： $d_n, p_n, e_n, \pi_n, \sigma_n, \tau_n, \theta_n$ 是定义 44 的字段， g_n, ω_n 是 θ_n 携带的累加器与已提交视图； $\gamma[\theta_n \mapsto \theta']$ 、 $\gamma[n \mapsto \langle \dots \rangle]$ 与 $\gamma \setminus n$ 分别是在一个字段、一个线程的内容与一个线程的存在上与 γ 不同的状态。

线程的名字赋予它一种能穿越自身变化的同一性：下面每条规则都改写一个线程的生命周期状态而放任其余不动，所以规则必须说明作用于哪一个，而有两个字段是引用线程而非描述线程——父引用 π 与已提交视图 ω 。名字是原子：没有规则计算名字、检查其结构，或用相等

以外的任何关系联系两个名字，引入一个线程只是抽取一个尚未使用的名字。这就是动态创建的局部名字的纪律 [39]，此处用于线程同一性。

每个线程各拥有一张表意味着协副作用上下文是导出的而非存储的：它就是活动线程共同提供的内容。

$$\sigma_\gamma := \bigcup \{ \sigma_m \mid m \in \text{dom}(F_\gamma), \theta_m = \text{Active}(-, -) \} \quad (4.4)$$

并集是良定义的，因为线程只写它声明的键， $\text{dom}(\sigma_n) \subseteq p_n$ ，且不同线程的供给不相交（定义 43），所以每个 $k \in \text{dom}(\sigma_\gamma)$ 恰好落在一个 Active 线程的表中，该线程的名字记作 $\text{provider}_k(\gamma) \in \mathfrak{N}$ ，称为 k 的提供方。因此每个键只有唯一可能的提供方，由供给而非状态确定。没有规则直接写 σ_n ：线程的供给是其自身副作用函数执行的集合操作，它们落入 σ_n ，从而已经是 e_n 返回的状态的一部分，并随累加器一起再次离开。只有副作用的协副作用部分以这种方式记录，因为只有协副作用部分是其他线程据以声明的内容；改变 γ 中其他位置的状态的副作用与其他副作用一样由 g 跟踪，但没有线程能在规约中命名它们，所以它们不贡献任何顺序约束。

3.2.2 节的满足关系于是原样适用， $\gamma \models d$ 缩写 $\sigma_\gamma \models d$ 。一个键落在 $\text{dom}(\sigma_\gamma)$ 中，当且仅当某个 Active 线程已安装它，其供给是它可以安装的键而非它已有的键，所以 $\gamma \models d$ 已经要求每个声明的键都有一个 Active 提供方。只对 Active 线程取并集，正是让线程能够在撤离任何东西之前先停止提供的关键，4.3.1 节将此转化为顺序纪律。

4.2 基础演算

本节给出的演算只有图 1 的双状态生命周期而别无其他：每条线程与之比较的目标，以及移动它的五条规则。

目标视图。规则将每条线程与一个目标比较，即它是否应当运行，以及针对其依赖的哪一种解析。目标不是线程单独的属性，因为线程声明的键是针对整个状态解析的，所以它是该状态上的一个谓词。

定义 4.4 (46). n 在 γ 处的目标视图 (*target view*) 将每个声明的键送到其提供方，故它是全映射 $d_n \rightarrow \mathfrak{N}$ ，且当 n 根本不当运行时为 $\perp$ ：

$$\text{target}_n(\gamma) := \begin{cases} \perp & \text{若 } \tau_n \vee \neg(\gamma \models d_n) \\ (k \in d_n) \mapsto \text{provider}_k(\gamma) & \text{否则} \end{cases} \quad (4.5)$$

一个状态是静息的 (*quiescent*)，当每条线程都达到了其目标视图：

$$\text{quiet}(\gamma) := \forall n \in \text{dom}(F_\gamma). \begin{cases} \text{target}_n(\gamma) = \perp & \text{若 } \theta_n = \text{Inactive} \\ \text{target}_n(\gamma) = \omega_n & \text{若 } \theta_n = \text{Active}(-, \omega_n) \end{cases} \quad (4.6)$$

目标只回应两件事而不回应其他：退役，经由 τ_n ；协副作用解析，经由 $\gamma \models d_n$ 与 provider_k ，每个声明的键都在定义 43 的唯一共享域上从 σ_γ 读取。

定义 44 的已提交视图与目标视图类型相同，生命周期由比较二者驱动： ω_n 是 n 激活时所针对的解析， $\text{target}_n(\gamma)$ 是它应当针对的解析，下面每条规则都在二者相等或相异时触发。记录提供方而非值，正是让比较可用的关键，否则提供相等值的另一条线程会比较相等。组件读取的值经由视图到达，因为提供方的表中持有该值；实现中将该映射保存在 `fiber.committed`，将其散列保存在 `fiber.target` (5.1.3 节)。

规则。基础演算把每次转换当作原子的、即时的且不会失败的：激活一步施加其副作用函数，停用一步施加累加器，且两者都必然成功。4.3 节将放弃全部三条。

五条规则生成两个关系。编排规则 (orchestration rule) 以 O - 为前缀, 写作 $\gamma \Rightarrow \delta$, 是编排器可以执行的动作; 其前提说明该动作何时合法, 而非何时发生。生命周期规则 (lifecycle rule) 以 L - 为前缀, 写作 $\gamma \longrightarrow \delta$, 是只要前提成立系统就自动采取的步骤。步骤序列交错二者, 下文的 $\xrightarrow{*}$ 单指生命周期步骤。

$$\begin{array}{c}
 \frac{n \notin \text{dom}(F_\gamma) \quad \pi \in \text{dom}(F_\gamma) \cup \{\text{root}\} \quad (d, p, e) \in \mathfrak{C}_\Gamma \quad \forall m \in \text{dom}(F_\gamma). p \cap p_m = \emptyset}{\gamma \Rightarrow \gamma[n \mapsto \langle d, p, e, \pi, \emptyset, \perp, \text{Inactive} \rangle]} \quad \text{O-Insert} \\
 \\
 \frac{n \in \text{dom}(F_\gamma)}{\gamma \Rightarrow \gamma[\tau_n \mapsto \top]} \quad \text{O-Retire} \\
 \\
 \frac{\tau_n = \top \quad \theta_n = \text{Inactive} \quad \forall m. \pi_m \neq n}{\gamma \Rightarrow \gamma \setminus n} \quad \text{O-Remove}
 \end{array} \quad (4.7)$$

插入与退役是仅有的外部输入: 编排器请求一条线程存在或停止存在, 从不直接设定其生命周期状态。O-Retire 对线程的状态不加条件, 因为退役是一个请求, 而由生命周期规则将其付诸实施。退役与移除相分离也是出于同一理由: 仍在 Active 的已退役线程必须先被停用, 过早移除会丢弃累加器并造成泄漏。前提 $\forall m. \pi_m \neq n$ 通过先移除子代再移除父代保持树良构。O-Insert 的最后一个前提正是单来源纪律的施加之处: 一个键只有唯一可能的提供方, 因为编排器不得接纳声明它的第二个组件。

$$\begin{array}{c}
 \frac{\theta_n = \text{Inactive} \quad \omega = \text{target}_n(\gamma) \neq \perp \quad e_n(\gamma) = (\delta, g)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g, \omega)]} \quad \text{L-Reload} \\
 \\
 \frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}]} \quad \text{L-Unload}
 \end{array} \quad (4.8)$$

L-Reload 在安装逆的同时安装已提交视图; L-Unload 施加逆并丢弃已提交视图。二者由同一个比较驱动: 当线程不持有已提交视图而其目标视图非 $\perp$ 时 L-Reload 触发, 当它持有的已提交视图不是其目标视图时 L-Unload 触发。这就是 3.2 节的反应式纪律, 从既回应退役又回应副作用的目标上读取: 只要目标视图变化, 转换就被发起, 而无论是二者中的哪一个改变了它。

实例化。组件可以在安装其副作用时实例化另一个组件, 这正是插件宿主在插件加载自己的插件时所做的工作。到目前为止的规则把注册表只留给编排规则, 这样的实例化便无处发生。一个原语为其提供了场所。

定义 4.5 (47). e_n 的一次应用——或其一次迭代, 凡 4.3.2 节适用处——可以注册一个组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 。它以该组件取 $\pi = n$ 的 O-Insert 代替状态映射, 并以如此注册的线程的 O-Retire 作为其产出的逆。规则抽取名字, 须满足 O-Insert 的新鲜性前提, 并将其交给副作用函数。

其逆是退役而非移除, 原因在于逆必须在到达它的任何地方都能施加。O-Remove 带有前提, 故由它构造的逆可能无法施加: 子代仍 Active 的父代无法运行其累加器, 且没有规则会移动该子代, 因为定义 46 不读取线程树。O-Retire 只有 $n \in \text{dom}(F_\gamma)$ 这一个前提。它在注册时的状态上留下的表项是已退役、Inactive($\perp$) 且持空表的, 即引理 57 的退化表项: 它与线程的不存在仅在控制字段上有差别, 且没有规则区分二者。

令子代退役即设定 τ ，从而使其目标视图变为 $\perp$ ，此后普通规则将它带回 *Inactive*。父代不必等待，*O-Retire* 是无条件的，所以无论子代是否已离开，*L-Unload* 都对父代适用。孙代一次下降一层到达，子代自己的累加器退役子代所注册的内容。定理 66 一并涵盖这一级联与 4.3.1 节沿协副作用施加的级联。

约束。手头有了这唯一的例外，就可以给出副作用函数所遵守的纪律了。它约束一次应用写入什么，使得施加它的规则可以解释其余一切变化；也约束一次应用读取什么，使得线程只看到它声明的协副作用而看不到注册表的更多内容。约束写入正是让 4.4 节得以把表 1 当作写入的完整清单的原因。

定义 4.6 (48). 映射 $f : \Gamma \rightarrow \Gamma$ 约束于 (*confined to*) n ，当对每个满足 $n \in \text{dom}(F_\gamma)$ 的 $\gamma \in \Gamma$ ，写 $\delta = f(\gamma)$ ：

1. (写入。) $\text{dom}(F_\delta) = \text{dom}(F_\gamma)$ ，对每个 $m \in \text{dom}(F_\gamma)$ 且 $m \neq n$ 有 $\delta(m) = \gamma(m)$ ，且 $\delta(n)$ 与 $\gamma(n)$ 仅在 σ 上不同；
2. (读取。) 在 σ_n 上、在每个 $m \in \text{dom}(F_\gamma)$ 的限制 $\sigma_m|_{d_n}$ 上、以及在状态的未被任何线程表命名的部分上相等的两个状态，被 f 送到在同样三项上相等的状态。

副作用函数 e 约束于 n ，当它的每次应用——及其每次迭代，凡 4.3.2 节适用处——或者注册一个组件（定义 47），或者其状态映射 $\text{pr}_1 \circ e$ 与其产出的逆都约束于 n 。每条线程的副作用函数都被要求约束于该线程。

一次注册在它抽取的唯一名字上写入 *O-Insert* 所写的表项，别无其他；它作为逆产出的 *O-Retire* 写该名字的 τ ，别无其他。因此任一种应用都不写已存在线程的任何控制字段——除了那一个 τ ——并且完全不读取控制字段。

第 (2) 款正是组件可以读取其声明的值的原因：这些值位于其提供方的表中，所以一个除 σ_n 外不读任何表的副作用函数将无法使用自己的协副作用。它不可读取的是 d_n 之外的表或任何控制字段，这正使组件无法在其未声明的线程的生命周期状态上进行分支。

这些规则是非确定性的：若干线程可能同时持有与其目标视图相异的已提交视图，而该关系不承诺它们之间的次序。它们也纯是反应式的：没有规则提及调度器；步骤是规则应用的任意序列，所以在所有此类序列上证明的定理对运行时可能采纳的任何调度策略都成立。

4.3 进行中的转换

本节在四种设定下扩展基础演算。第一种补上 3.2 节要求而 4.2 节无法表达的东西：一段铺展开的停用，其依赖者可以占据其间的区间；其余三种放弃转换是原子、即时且不会失败的理想化——真实运行时中的转换三者皆非。被放弃的是整个转换为一步，而不是一步为一条规则的一次应用，且四者共享一个结构性的后果，此处一次说清：不是一步的转换需要在进行期间占有一个状态，每个可运行方向各一个。

定义 4.7 (49). 本节的生命周期状态将 Θ_Γ 替换为

$$\Theta_\Gamma := \text{Inactive}(\zeta) \mid \text{Reloading}(i, g, \omega) \mid \text{Active}(g, \omega) \mid \text{Unloading}(g, \omega, \zeta) \quad (4.9)$$

其中 $i : \mathfrak{C}_\Gamma^{\text{iter}*}$ 是剩余副作用迭代器（定义 51）， $g : \Gamma \rightarrow \Gamma$ 是迄今构建的累加器， $\omega : d \rightarrow \mathfrak{N}$ 是已提交视图， $\zeta : \{\perp\} \cup \Xi$ 是结局 (*outcome*)：*Unloading* 携带它为其停用所趋赴的结局，*Inactive* 携带它所到达的结局，或为 $\perp$ ，或为取自错误集 Ξ 的错误，后者由 4.3.4 节给出。

线程已安装 (installed), 当它处于携带累加器与已提交视图的三个状态之一; 已失败 (failed), 当它携带错误结局:

$$\text{installed}_n(\gamma) := \theta_n \neq \text{Inactive}(-), \quad \text{failed}_n(\gamma) := \exists \xi \in \Xi. \theta_n = \text{Inactive}(\xi) \quad (4.10)$$

已安装的线程 n 将 k 解析到 m , 当 $\omega_n(k) = m$ 。定义 46 的静息性在更宽的状态空间上读作

$$\text{quiet}(\gamma) := \forall n \in \text{dom}(F_\gamma). \begin{cases} \zeta \neq \perp \vee \text{target}_n(\gamma) = \perp & \text{若 } \theta_n = \text{Inactive}(\zeta) \\ \text{target}_n(\gamma) = \omega_n & \text{若 } \theta_n = \text{Active}(-, \omega_n) \\ \perp & \text{否则} \end{cases} \quad (4.11)$$

4.1 节的定义搬到此状态空间上, 有两处读法须固定。其一, 4.2 节的 Inactive 在 O-Insert 的结论中读作 Inactive($\perp$), 在 O-Remove 的前提中读作 Inactive($-$)。其二, σ_γ 仍只对 Active 线程的表取并, 所以转换任一方向都在进行中的线程通过它持有的 ω 读取其协副作用, 且不提供自己的任何协副作用; 其转换已经写入的键因此还不是依赖者可以据以激活的键。在双状态演算中这一区分是空的, 那里每个已安装的线程都是 Active。

图 2 画出这些状态构成的生命周期, 下面四个小节给出其各条边上的规则。

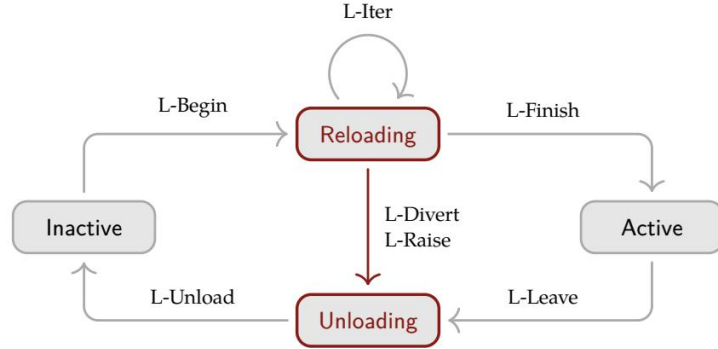


Figure 4.2: 带进行中转换的生命周期; 两个转换状态以轮廓标出

4.3.1 撤离

3.2 节要求依赖者在其依赖之后激活, 且依赖只在依赖者全部停用之后撤离其供给。前半半在基础演算中已然成立: 激活要求 $\gamma \models d_n$, 所以声明 k 的线程不能在某个线程正提供 k 之前激活。后半半是实质性的, 而且必须交付的不只是状态变化的次序。因提供方即将离去而被拆除的组件正在运行自己的 teardown 代码, 而它可能恰恰需要正在被撤离的那个协副作用; 关闭连接池通常意味着把连接还给提供它们的一方。后半半必须交付的是: 消费者在它自己的整个停用期间仍能读取 k , 而提供方对 k 的撤离仅在之后生效。基础演算根本无法交付它: 其 L-Unload 把移除供给与运行逆合在一起, 没有为消费者们的 teardown 留下可占据的区间。

本层把那一步一分为二, 并用下面的条件守护后半步。

定义 4.8 (50). 线程 n 在 γ 处被依赖 (relied upon), 当某个其他已安装线程将一个键解析到它:

$$\text{relied}_n(\gamma) := \exists m \in \text{dom}(F_\gamma), k \in d_m. m \neq n \wedge \text{installed}_m(\gamma) \wedge \omega_m(k) = n \quad (4.12)$$

$$\frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Unloading}(g, \omega, \perp)]} \quad \text{L-Leave} \quad (4.13)$$

$$\frac{\theta_n = \text{Unloading}(g, \omega, \zeta) \quad \neg \text{relied}_n(\gamma) \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}(\zeta)]} \quad \text{L-Unload}$$

L-Leave 记录停用的决定而不付诸行动，这使线程停止提供其副作用，同时保留自己的已提交视图与所有其他线程的已提交视图。L-Unload 施加累加器、丢弃已提交视图，并按其携带的结局使线程停留在 Inactive；在 4.3.4 节给出另一种情形之前，结局是 $\perp$ 。它是演算中唯一施加累加器的规则。

顺序的两半于是由形式的不同部分承担：可见性的一半由已提交视图承担，L-Unload 以其为最后的动作将其丢弃；次序的一半由前提 $\neg \text{relied}_n(\gamma)$ 承担——我们称之为守卫 (guard)——它把 k 的撤离拦住，直到每个把 k 解析到 n 的消费者都已离去。定理 63 将确立二者。

守卫是按绑定而非按线程施加的： $\text{relied}_n(\gamma)$ 测试的是某个已提交视图是否命名 n ，所以不声明 n 的任何键的线程不构成障碍，在另一个域中解析了 n 的某个键的线程同样不构成障碍 (3.2.3 节)。在 4.2 节的单一来源纪律下，按绑定的读法与较粗的测试 $\exists m \neq n, k \in d_m. \text{installed}_m(\gamma) \wedge k \in p_n$ 相一致，因为那里一个键只有唯一可能的提供方。

这种守卫通常会死锁。使其不死锁的是 Unloading 连同 σ_γ 只对 Active 线程取并集：一旦 L-Leave 标记了 n ，其表便离开 σ_γ ，于是不再有任何目标视图能命名 n ，而每个已提交到 n 的消费者自身也在离去途中。定理 66 由此得出守卫总会释放的断言。

守卫沿副作用而非沿线程树为停用排序：父代可以在其某个子代仍 Unloading 时运行其逆，因为 relied 只谈已提交视图。父代与子代因此排序得比定理 63 排序提供方及其消费者更弱，而效果在环境状态中相遇的父代与子代由定义 60 的独立性假设管辖。

4.3.2 迭代

一次激活可以顺序执行多个副作用，停用必须将它们全部恢复。我们用副作用迭代器 (effect iterator) 为这样的激活建模，其每次迭代产出修改后的上下文、一个逆与一个续延：

定义 4.9 (51). 定义副作用迭代器 $\mathfrak{E}_\Gamma^{\text{iter}}$ 与见证副作用迭代器 $\mathfrak{E}_\Gamma^{\text{iter}*}$ 为如下递归类型：

$$\begin{aligned} \mathfrak{E}_\Gamma^{\text{iter}} &:= \mu \mathfrak{J}. \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J}) & \mathfrak{E}_\Gamma^{\text{iter}*} &:= \mu \mathfrak{J}. (e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J})) \\ & & & \times ((\gamma : \Gamma) \rightarrow (\text{let } (\delta, g, o) = e(\gamma) \text{ in } g(\delta) \simeq \gamma)) \end{aligned} \quad (4.14)$$

其中 $e(\gamma)$ 产出三元组 (δ, g, o) ，表示：

- δ 是新上下文；
- g 是当前副作用的逆函数；
- o 指示续延 (continuation) :
 - *Nothing* 表示迭代终止；
 - *Just(i)* 给出下一次迭代。

见证在定义 33 的 $\simeq$ 上读取，正如定义 37 读取 $\mathfrak{E}_\Gamma^*$ 的见证： $i \in \mathfrak{E}_\Gamma^{\text{iter}}$ 属于 $\mathfrak{E}_\Gamma^{\text{iter}*}$ ，当 i 尊重 $\simeq$ 且它产出的每个 g 尊重 $\simeq$ 并满足上面的子句。三元组逐分量比较：Nothing 只与 Nothing 相比，Just(i) 与 Just(i') 相比当 $i \simeq i'$ ，而迭代器上的 $\simeq$ 是满足那些子句的最大关系。取 $\simeq$ 为 Γ 上的相等即恰好还原原读法。

副作用迭代器变换 $\text{effect}_\Gamma^{\text{iter}}$ 通过递归调用把 effect_Γ 扩展到迭代器结构：

定义 4.10 (52). 定义副作用迭代器变换 $\text{effect}_\Gamma^{\text{iter}}$ 为:

$$\begin{aligned} \text{effect}_\Gamma^{\text{iter}} &: \mathfrak{C}_\Gamma^{\text{iter}} \rightarrow \partial\Gamma \rightarrow \partial^2\Gamma \\ &\quad \text{let } (\delta, g, o) = i(\gamma) \text{ in} \\ &\quad \text{let } t = \text{track}_\Gamma(g, \text{pr}_1 \circ i) \text{ in} \\ \text{effect}_\Gamma^{\text{iter}} &= i \mapsto (\gamma, \varphi) \mapsto \text{match } o \mid \text{Nothing} \Rightarrow ((\delta, \varphi \circ g), t) \mid \text{Just}(i') \Rightarrow \text{let } (s, r) = \text{effect}_\Gamma^{\text{iter}}(i')(\delta, \varphi \circ g) \text{ in} \end{aligned} \quad (4.15)$$

每次迭代中, 逆 g 按应用次序复合到 φ 上, 于是累加器 $\varphi \circ g_1 \circ \dots \circ g_k$ 施加时自然以后进先出 (LIFO) 次序恢复副作用。由于 $\text{effect}_\Gamma^{\text{iter}}$ 与 effect_Γ 落在同一个 $\partial\Gamma \rightarrow \partial^2\Gamma$ 上, 迭代器本身就是一种副作用, 凡可用副作用处皆可用它。组件的整次激活就是这样一种用法, 这正是本节余下部分形式化的内容, 而实现在每个变更点都接纳迭代器 (5.1.1 节)。Maybe($\mathfrak{C}_\Gamma^{\text{iter}}$) 续延使任意两次相继迭代之间都有一个边界可用, 在该边界处, 上下文是迄今各次迭代所造成的, 而累加器恰好恢复这些而不多恢复。在这个意义上, 副作用迭代器是一种具体化的定界续延 (reified delimited continuation), 即主流语言通过 `yield` 算子暴露的结构 [43], 所以该模型直接映射到它们已有的生成器上。

在演算中, 定义 44 的 e_n 从此处起读作 $\mathfrak{C}_\Gamma^{\text{iter}*}$, 而把原子副作用函数替换为迭代器将把基础的 L-Reload 拆为轨迹经过的一个已开始状态, 并给该线程第二条离开该状态的路。

$$\begin{aligned} &\frac{\theta_n = \text{Inactive}(\perp) \quad \omega = \text{target}_n(\gamma) \neq \perp}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Reloading}(e_n, \text{id}_\Gamma, \omega)]} \quad \text{L-Begin} \\ &\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad (\delta, h) = (\gamma, \text{id}_\Gamma) \vee i(\gamma) = (\delta, h, -)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Unloading}(g \circ h, \omega, \perp)]} \quad \text{L-Divert} \\ &\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \text{Just}(i'))}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Reloading}(i', g \circ h, \omega)]} \quad \text{L-Iter} \\ &\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \text{Nothing})}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g \circ h, \omega)]} \quad \text{L-Finish} \end{aligned} \quad (4.16)$$

每次迭代按定义 52 把新产出的逆以 $g \circ h$ 复合到累加器上, 使累加器以后进先出次序施加各逆。在任意两次相继迭代之间, 若目标视图已变化, 系统可以转移该转换, 施加迄今积累的逆以恢复上下文。L-Divert 与其他所有停用一样经由 Unloading 路由, 而不是就地施加累加器, 且它在彼处遇到的守卫是空真的: 从未 Active 过的线程不提供任何东西, 也不出现在任何已提交视图中。它两个备选中的第一个中止线程正持有的迭代, 这只有迭代边界才使之可能, 所以转移可以发生的粒度就是迭代器的粒度; 第二个让该次迭代着陆, 4.3.3 节正是需要它的地方。

普通副作用函数 ($\mathfrak{C}_\Gamma$) 是退化情形: 第一次迭代已产出 Nothing。这样的转换仍经过 Reloading, L-Divert 在彼处仍适用, 但累加器是 id_Γ 且没有迭代运行过, 所以没有任何东西被恢复, 转换要么安装其全部副作用, 要么一个也不安装。

4.3.3 异步性

到此为止的各层允许环境在一次次迭代之间移动, 并假设每次迭代自身瞬时完成, 其发起与着陆是一步。我们抽象地为非即时性建模: 一次迭代产出类型 $\text{Future}(A)$ 的值, 其中 Future 是一

个不透明的类型构造子，其定义性质是在提交与完成之间外部状态可以变化。

在此模型下，一次迭代在一个状态上发起，在另一个状态上着陆，而线程在飞行期间处于 Reloading。本层加入的是惯性：迭代一经发起就会着陆，且其着陆不可拒绝。飞行期间翻转的目标视图因此不能靠中止迭代来回应，只有让迭代着陆的那一备选 L-Divert 仍然可用：迭代着陆，线程随后停用。本层因此不加入任何规则，也不加入任何规则匹配的类型；在 Γ 的粒度上惯性就是其全部内容，其形式是对宿主可取 L-Divert 哪个备选的一条限制。

那个备选正是基础演算无法表达的。在那里，目标视图已翻转的转换在被发现的同一步中被撤销；这里飞行中的迭代必须先着陆，所以线程需要在其逆运行期间有个容身之处，而唯一可靠的位置是持有该迭代产出的逆的 Unloading。若改经 Active 路由，会让线程在一步之内提供其副作用，迫使依赖者对一个已在离去的组件激活。这就是实现中 reload 与 unload 的相互链式衔接。

停用也可以直接链回激活，经由复合而非规则。L-Unload 对目标视图没有前提，所以无论线程停用期间目标视图变成了什么，累加器都会运行，线程变为 Inactive，而 L-Begin 可以随即从那里开始新转换。

4.3.4 失败

到此为止的每条规则都假设它运行的副作用会成功，而运行时做不到这一点。组件安装的副作用触及跟踪它们的上下文之外，被触及者可能拒绝：端口已被绑定、文件不存在、对端无应答。失败的转换仍须让线程的副作用得到恢复而非搁浅。

设 Ξ 为错误集，并细化定义 51 的副作用迭代器，使一次迭代可以抛出而非产出三元组：

$$\begin{aligned}\mathfrak{C}_{\Gamma}^{\text{fail}} &:= \mu\mathcal{J}. \Gamma \rightarrow \text{Either}(\Xi, \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathcal{J})) \\ \mathfrak{C}_{\Gamma}^{\text{fail}*} &:= \mu\mathcal{J}. (e : \Gamma \rightarrow \text{Either}(\Xi, \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathcal{J}))) \\ &\quad \times ((\gamma : \Gamma) \rightarrow (\text{let Right}(\delta, g, o) = e(\gamma) \text{ in } g(\delta) \simeq \gamma))\end{aligned}\quad (4.17)$$

见证只约束 Right 的情形，在模式不匹配处空真——抛出没有任何需要撤销的东西——且 Reloading 携带的 i 从此处起读作 $\mathfrak{C}_{\Gamma}^{\text{fail}*}$ 。定义 52 的提升在抛出代替三元组传播的意义下原样搬用，所以会抛出的迭代器与普通迭代器一样，凡可用副作用处皆可用。本层加入一条规则，并启用定义 49 的第二种结局，O-Remove 无需加宽即可接纳它。L-Iter、L-Finish 与 L-Divert 的前提在它们匹配的三元组外再读一层 Right。抛出是迭代做的事情，所以该规则是离开 Reloading 的一个出口。

$$\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad i(\gamma) = \text{Left}(\xi)}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Unloading}(g, \omega, \xi)]} \quad \text{L-Raise} \quad (4.18)$$

L-Raise 先恢复后记录。线程携带错误作为结局路由进入 Unloading，构建至失败迭代的累加器在那里被施加，线程什么也没安装便到达 Inactive(ξ)，所处的状态与中止式 L-Divert 所产生的状态仅在线程携带的结局上不同。将失败像其他所有停用一样路由，正是使每种结局都只能经由 L-Unload 到达的原因，这就是定理 59 所依赖的唯一事实。L-Begin 以 Inactive($\perp$) 为前提，所以生命周期不会从错误结局重入；这是结局的实质所在：它扣住一条副作用函数已表明在其运行的状态中不可靠的线程，而不是让它在未曾改变的环境中重试。失败的线程也不阻碍任何东西：它是 Inactive，不携带已提交视图，无法使 relied 成立。

失败记录在线程上而非传播到其父代，所以转换失败的组件让其同胞继续运行，这正是插件宿主想要的行为，也是结局按线程给出而非作为整个状态的属性的原因。

4.4 元理论

4.3 节给出十条规则：4.2 节的三条编排规则；激活的 L-Begin、L-Iter 与 L-Finish；激活可能提前结束的两种方式的 L-Divert 与 L-Raise；以及停用的 L-Leave 与 L-Unload。本节从这些规则中读取其两个维度可组合性的全局形式——一条纤程的保证无论其余纤程其间做什么都成立——并加上只有整个系统才能被要求的東西：它总是到达其目标所要求的配置，且该配置是静态装配本会产生那个。下面的每条性质都是步骤序列的性质，所以我们为步骤编号，并按该编号读取状态的字段。

两条约定把 3.3.2 节搬入本节。下文状态之间的每个等式都在定义 33 的观测等价 (observational equivalence) $\simeq$ 上读取，正如引理 38 读取 3.1 节的等式；副作用函数所遵守的见证条件是定义 37 给出的那个，按定义 51 在迭代器上读取，并在下文的 $\approx$ 上在注册式迭代上读取。

定义 4.11 (53). 为步骤编号为 t ，使 γ^t 为前 t 步到达的状态，并写

$$\text{step}^t := r(n) \quad (4.19)$$

表示 γ^t 处采取的步骤：它应用的规则 r (十条之一) 与它应用该规则的名字 $n \in \mathfrak{N}$ 。序列从满足 $\text{dom}(F^0) = \emptyset$ 的 γ^0 出发，所以每条纤程都经由 *O-Insert* 而诞生——无论是编排器的还是某次迭代采取的 (定义 47)。 γ^t 的字段以 t 为上标： $\theta_n^t, \omega_n^t, \sigma_n^t, g_n^t$ 与 i_n^t 是 n 在 γ^t 处的生命周期状态、已提交视图、表、累加器与剩余迭代器，而 F^t 与 σ^t 是 γ^t 自身的注册表与协副作用上下文，即定义 45 的 F_γ 与 σ_γ 在彼处的读取。谓词以状态为参数、其余皆为下标，所以 installed_n^t 、 target_n^t 、 relied_n^t 与 quiet_n^t 是定义 46、定义 49 与定义 50 的谓词在 γ^t 处的取值。 n 的一段活跃段 (*episode*) 是使 installed_n^t 成立的指标的最大区间 $[b, u]$ 。它在 b 处开启，其中 $b > 0$ 且 $\neg \text{installed}_n^{b-1}$ ，空的 F^0 使开始时没有已安装的纤程；它在 u 处关闭，当 installed_n^u 且 $\neg \text{installed}_n^{u+1}$ ，而最后一段活跃段不必关闭。

4.3 节的每条规则都以 $\gamma \rightarrow \delta[\dots]$ 的形状结束，其中前提从 γ 计算 δ ，在不计算时让 δ 为 γ ，而方括号编辑注册表的具名字段。这两半分别命名，且都是 Γ 上的映射。在 γ^t 处采取的、作用于 n 的规则的那一步，其状态映射为

$$\Psi^t := \begin{cases} \text{pr}_1 \circ i & \text{在 L-Iter、L-Finish 与着陆式 L-Divert 处} \\ g & \text{在 L-Unload 处} \\ \text{id}_\Gamma & \text{在其他每条规则处} \end{cases} \quad (4.20)$$

其中 i 与 g 是 θ_n^t 携带的迭代器与累加器，而编辑 $\text{edit}^t : \Gamma \rightarrow \Gamma$ 是把方括号当作函数读取，给它命名的字段赋予前提在 γ^t 处计算的值。二者因此由 step^t 连同 γ^t 固定，且在每个状态处有定义，这正是让定理 61 与引理 71 能在远离 γ^t 之处求值它们的原因。每一步分解为

$$\gamma^{t+1} = \text{edit}^t(\Psi^t(\gamma^t)) \quad (4.21)$$

例如在 L-Unload 处， edit^t 是 $[\theta_n \mapsto \text{Inactive}(\zeta)]$ ；在 O-Remove 处它是移除 $\backslash n$ ——这就是为什么后半是编辑而非赋值。字段沿同一接缝划分：表 σ_m ，在创建 m 的 O-Insert 将其置空之后没有 edit^t 再写它；控制字段 $\theta_m, \tau_m, \pi_m, d_m, p_m, e_m$ 连同 $\text{dom}(F_\gamma)$ ，除经定义 47 的原语外没有 Ψ^t 写它们。写 $\gamma \approx \delta$ 当两个状态在控制字段之外的一切上都一致。

关系 $\approx$ 不是定义 33 的 $\simeq$ ，二者也互不细化，因为各自遗忘另一方必须保留的东西。恢复精确性是关于副作用的断言，所以 $\approx$ 精确比较表与环境状态，只遗忘注册表中关于哪条纤程安

装了它们的记录。规则读取控制字段来决定是否适用，所以 $\simeq$ 必须保留它们，而本节把它读作定义 33 与注册表定义域及每条线程每个控制字段上的一致的合取：

$$\gamma \simeq \delta := \sigma_\gamma \simeq \sigma_\delta \wedge \text{dom}(F_\gamma) = \text{dom}(F_\delta) \wedge \forall n, c \in \{\theta, \tau, \pi, d, p, e\}. c(\gamma(n)) \simeq c(\delta(n)) \quad (4.22)$$

函数类型的字段，如 e_n 与 θ_n 内部的 g 和 i ，按定义 36 比较映射的方式比较，迭代器按定义 51 比较两个迭代器的方式比较，其他类型的字段按相等比较。下面的结果在两个关系上成立，每个对应状态的一半，引理 55 为全部十条规则一次性确立 $\simeq$ 那一半。

表 1 是把 4.3 节的十条规则当作这样的写入来读取。累加器、已提交视图与剩余迭代器是 θ_n 的成分，所以第三列也记录对它们的写入，其中的 h 命名第四列的迭代所产出的逆，在 L-Divert 中止该迭代时为 id_Γ 。凡由迭代器构造的 Ψ^t 注册了线程（定义 47），该注册在其抽取的名字上携带 O-Insert 行的写入；凡 L-Unload 的累加器退役了一条线程，它携带 O-Retire 行的写入。下面的每个案例分析都是查表，而有五次查表出现得足够频繁，值得命名。

规则	θ_n^t	θ_n^{t+1}	Ψ^t	编辑的控制字段
O-Insert	未定义	Inactive($\perp$)	id_Γ	$\text{dom}(F_\gamma)$
O-Retire	无约束	不变	id_Γ	τ_n
O-Remove	Inactive($-$)	未定义	id_Γ	$\text{dom}(F_\gamma)$
L-Begin	Inactive($\perp$)	Reloading($e_n, \text{id}_\Gamma, \omega$)	id_Γ	θ_n
L-Iter	Reloading(i, g, ω)	Reloading($i', g \circ h, \omega$)	$\text{pr}_1 \circ i$	θ_n
L-Finish	Reloading(i, g, ω)	Active($g \circ h, \omega$)	$\text{pr}_1 \circ i$	θ_n
L-Divert	Reloading(i, g, ω)	Unloading($g \circ h, \omega, \perp$)	id_Γ 或 $\text{pr}_1 \circ i$	θ_n
L-Raise	Reloading(i, g, ω)	Unloading(g, ω, ξ)	id_Γ	θ_n
L-Leave	Active(g, ω)	Unloading($g, \omega, \perp$)	id_Γ	θ_n
L-Unload	Unloading(g, ω, ζ)	Inactive(ζ)	g	θ_n

Table 4.1: 十条规则作为对其所作用线程 n 的写入，其中 step^t 是该规则在 n 处的应用。

引理 4.12 (54). 结合定义 48 读表 1，对每一步 t 及 γ^t 处存在的所有线程 m, n ：

1. $\sigma_m^{t+1} \neq \sigma_m^t$ 仅当步骤 t 作用于 m ，且该写入落在 Ψ^t 之内；
2. ω_n 仅在 $\text{step}^t = L\text{-Begin}(n)$ 处产生，仅在 $\text{step}^t = L\text{-Unload}(n)$ 处消亡，所以 ω_n^t 在 n 的一段活跃段内为常量；
3. $\Psi^t = g_n^t$ 仅当 $\text{step}^t = L\text{-Unload}(n)$ ，且没有其他步骤把 g_n 施加到状态上；
4. $\neg \text{installed}_n^t \wedge \text{installed}_n^{t+1} \Rightarrow \text{step}^t = L\text{-Begin}(n)$ ，且 $\text{installed}_n^t \wedge \neg \text{installed}_n^{t+1} \Rightarrow \text{step}^t = L\text{-Unload}(n)$ ；
5. π_n, d_n, p_n 与 e_n 随 n 的表项产生而不再被写， τ_n 单调，只在 $\top$ 处写且只由 O-Retire 写。

证明. 设步骤 t 在 n 处应用 r 。由定义 53，它分解为 $\text{edit}^t \circ \Psi^t$ ，其中 edit^t 写表 1 第五列举出的字段而不写别的，而 Ψ^t 是 id_Γ 、 n 的某次迭代的一次应用、或累加器 g_n^t ——后者是那些迭代产出的逆的复合。三者都由定义 48 约束于 n ，所以 Ψ^t 不写 γ^t 处任何线程的字段， σ_n 除外，连同注册添加的表项及其逆写的 τ 。两半于是划分了写入，而每一款都是该划分在一个字段上的

读取。第二、三列的一种读法被用了两次：Inactive 是唯一不携带已提交视图的生命周期状态，L-Begin 是离开它的唯一规则，L-Unload 是进入它的唯一规则，而其余每一行都把其前提的 ω 原样带进结论。

(1) edit^t 不写任何表，第五列未列出任何表，而 Ψ^t 对存在的 $m \neq n$ 不写 σ_m 。所以 σ_m 只能在 $m = n$ 处移动，且只在 Ψ^t 之内。

(2) ω_n 是 θ_n 的成分，只有 edit^t 写它且只在步骤作用的纤程处，故由上面的读法， ω_n 在 n 的 L-Begin 处产生，在 n 的 L-Unload 处消亡。 n 的活跃段是 installed_n 成立的区间，因而是 ω_n 有定义的区间，所以两条规则都不落在其内部。

(3) 第四列中累加器只在 L-Unload 处出现：其余规则取前向映射 $\text{pr}_1 \circ i$ 或 id_Γ ，而 edit^t 根本不把映射施加到状态上。

(4) installed 即 $\theta_n \neq \text{Inactive}(-)$ ，且由上面的读法，L-Begin 与 L-Unload 是前提与结论在 θ_n 是否 Inactive 上不同的仅有两条规则。作用于某个 $m \neq n$ 的步骤不写 θ_n ，注册添加的表项在 γ^t 处不存在的名字上。

(5) 第五列没有一行列出 π, d, p 或 e ；它们随 O-Insert 添加的表项产生，其结论写之，注册采取的 O-Insert 亦然。只有 O-Retire 写 τ ，写在 $\top$ 处，无论由编排器采取还是作为注册的逆（定义 47）；O-Insert 在本不存在的名字上设 $\tau = \perp$ ，所以没有步骤把 τ 归回 $\perp$ 。□

另有三次查表说明规则看不到什么。第一次是：规则只经由上面的观测读取状态，于是整个演算下降到 $\Gamma / \simeq$ 。

引理 4.13 (55). ($\simeq$ -不变性) 设 $\gamma \simeq \gamma'$ 如上读取。则 4.3 节的一条规则在 γ 处作用于 n ，当且仅当它在 γ' 处作用于 n ，且两次应用到达的状态仍由 $\simeq$ 关联。

证明. 4.3 节的每个前提属于四类之一，且各自读取该关系保留的成分。把 θ_n 或 τ_n 与模式匹配的前提，以及 O-Remove 的前提 $\forall m. \pi_m \neq n$ ，读取控制字段。O-Insert 的前提 $(d, p, e) \in \mathfrak{C}_\Gamma$ 与 $\forall m. p \cap p_m = \emptyset$ 读取 d, p, e 。提及 target_n 或 relied_n 的前提读取 τ_n 、 θ_m 内部的各已提交视图以及 $\text{dom}(\sigma_\gamma)$ ——后者由定义 45 从诸 θ_m 与诸 $\text{dom}(\sigma_m)$ 计算，而定义 33 只在两个协副作用上下文的定义域一致时关联它们。其余前提读取 $\text{dom}(F_\gamma)$ 。没有任何前提在 $\simeq$ 之外读取值 $\sigma_\gamma(k)$ ，所以没有前提分开两个 $\simeq$ 关联的状态。

至于结论，由定义 53 有 $\gamma^{t+1} = \text{edit}^t(\Psi^t(\gamma^t))$ 。 edit^t 赋的值是它匹配的前提的成分，由上一段及定义 51 在两个状态处关联——定义 51 关联迭代器在 $\simeq$ 关联状态处产出的三元组。而 Ψ^t 尊重 $\simeq$ ：它是 id_Γ ，或 e_n 的一次迭代——定义 51 要求其尊重 $\simeq$ ——或 θ_n 内部的累加器，后者是每个逆都由同一定义尊重 $\simeq$ 的逆的复合。□

状态携带的名字由上述观测中的两个读取—— $\text{dom}(F_\gamma)$ 与控制字段的索引——而抽取名字的规则抽取任意尚未使用的名字（定义 47）。因此要在 $\simeq$ 上读取下面的结果，还须在重命名上读取它们，这正是 4.1 节的纪律兑现之处。

引理 4.14 (56). (等变性) 设 $\chi: \mathfrak{N} \rightarrow \mathfrak{N}$ 为双射， $\chi \cdot \gamma$ 为携带注册表 $F_\gamma \circ \chi^{-1}$ 、且出现在任何 π_m 或 ω_m 中的名字皆换成其像的状态。则 $\chi \cdot \gamma$ 是状态，在 γ 良构处良构，且 $\text{step}^t = r(n)$ 把 γ^t 带到 γ^{t+1} ，当且仅当 $r(\chi(n))$ 把 $\chi \cdot \gamma^t$ 带到 $\chi \cdot \gamma^{t+1}$ 。

证明. 前提读取名字的方式只是与另一个名字比较：或直接比较，如 O-Insert 的新鲜性 $n \notin \text{dom}(F_\gamma)$ 与 O-Remove 的 $\forall m. \pi_m \neq n$ ；或经由名字的表，如 target_n 与 relied_n 读取诸 π_m 与诸

ω_m 。双射保持每一个这样的比较。规则写的名字只有 O-Insert 设定的 π 与 L-Begin 设定的 ω ，二者都取自其前提读取的内容，所以写入与 χ 交换；副作用函数根本不写名字，只经定义 47 的原语抽取，而定义 48 把它约束到该原语添加的表项。良构性（定义 58）是四个比较名字与名字的条件。 $\square$

一个序列与其重命名因此以相同次序采取相同规则，到达仅在 χ 上不同的状态。除注册抽取的名字外都一致的两个序列据此被视为同一，下面的结果在将它们视为同一的重命名上读取。

第二次查表是：剥得只剩名字的表项对规则不可见，这正是让定义 47 能在其恢复的状态中没有该纤程的地方令纤程退役、让引理 72 移除被删除活跃段所作注册的原因。

引理 4.15 (57). (退化表项) 称 n 在 γ 处退化 (*vestigial*)，当 $\tau_n = \top$ 、 $\theta_n = \text{Inactive}(\perp)$ 、 $\sigma_n = \emptyset$ 且没有 m 满足 $\pi_m = n$ ；退化表项满足 $\gamma \approx \gamma \setminus n$ 。若 n 在 γ 处退化，则对每条规则与每个 $m \neq n$ ：

1. 在 γ 处作用于 m 的规则也在 $\gamma \setminus n$ 处作用于 m ，且二者到达的状态仅在 n 处的表项上不同，该表项保持退化；
2. 反之，在 $\gamma \setminus n$ 处作用于 m 的规则也在 γ 处适用，除非它是抽取名字 n 的 *O-Insert*，或声明 p_n 中某键的 *O-Insert*。

证明. 退化的 n 对作用于 $m \neq n$ 的规则的任何前提所读取的观测都没有贡献。它不是 Active，所以 σ_n 不进入任何 σ_γ ， n 不是任何键的提供方， $\gamma \models d_m$ 与 target_m 不受影响； installed_n 失败，所以 n 不向 relied_m 贡献任何析取支；没有 $\pi_{m'}$ 命名 n ，所以 m 的 O-Remove 的前提 $\forall m'. \pi_{m'} \neq m$ 不受影响；而 θ_n, τ_n, π_n 只被作用于 n 的规则读取。第 (2) 款排除的两个前提正是移除所放松的两个：不存在的名字是新鲜的，不存在的供给与任何其他供给不相交。由引理 54，作用于 $m \neq n$ 的规则不写 n 的字段，所以该表项存续，且该步的状态映射由定义 48 约束于 m ，故它让 σ_n 保持为空。 $\square$

化简生命周期状态连同匹配它们的规则得到一个子演算，而并非每条结果都能在化简后幸存。丢弃 4.3.1 节是紧要的情形，即 4.3 节开头的那一划分从元理论一侧的读法：其守卫正是确立定义 58 第 (3)、(4) 款的东西，而定理 63 依赖守卫创造的区间，所以这三者没有它便失效。其余三个小节所添加的内容都可以被化简掉而不扰动下面的结果，它们各自只是向定义 49 固定的同一个状态空间添加规则。

4.5 保持

定义 45 固定了注册表 (registry) 的形状，在下面的结果能够对其有所补充之前，必须先让诸规则对照它接受检验。本小节识别出诸规则所保持的不变式 (invariant)：其第一条条款就是该形状，其余条款则是那些结果所假定的东西。

定义 4.16. 一个注册表 F_γ 是良构的 (*well formed*)，当对所有 $m, n \in \text{dom}(F_\gamma)$ 及所有 $k \in K$ ：

1. $\pi_n \in \text{dom}(F_\gamma) \cup \{\text{root}\}$;
2. $m \neq n \Rightarrow p_m \cap p_n = \emptyset$;

3. $\text{installed}_n(\gamma) \Rightarrow \omega_n$ 在 d_n 上全, 且取值于 $\text{dom}(F_\gamma)$;
4. $\text{installed}_n(\gamma) \wedge k \in d_n \wedge \omega_n(k) = m \Rightarrow \text{installed}_m(\gamma)$ 。

条款 (1) 是把定义 45 的树每次读一条边, 并让父指针落回注册表之内。该定义同样要求的无环性不需要条款, 因为指针所指称的线程先于指称它的线程被注册。

定理 4.17 (保持). 若 F^t 良构, 则无论步 t 应用哪条规则, F^{t+1} 亦良构。每一条条款都在 γ^{t+1} 处由 γ^t 处的全部四条建立。

证明. 设步 t 作用于 n 。

(1) 由表 1, 只有 O-Insert 与 O-Remove 写某个 π 或 $\text{dom}(F_\gamma)$ 。O-Insert 以 $\pi_n \in \text{dom}(F^t) \cup \{\text{root}\}$ 为前提, 这正是它所添加线程的条款; 它让其余每个 π 保持原样, 同时扩大 $\text{dom}(F_\gamma)$ 。O-Remove 以 $\forall m. \pi_m \neq n$ 为前提, 故没有任何幸存的 π_m 指称它移走的线程。

(2) O-Insert 的最后一条前提是 $\forall m. p_n \cap p_m = \emptyset$, 这正是它所添加线程的条款; 而由表 1, 没有别的规则写任何 p 或扩大 $\text{dom}(F_\gamma)$ 。下面用到两个推论: 由定义 43 有 $\text{dom}(\sigma_m) \subseteq p_m$, 故不同的表两两不交, σ_γ 是一个函数; 且 $k \in p_m \cap p_{m'}$ 强制 $m = m'$, 故 k 至多有一个可能的提供方。

(3) 由引理 54(2), 写某个 ω_n 的唯一规则是 L-Begin, 其前提 $\omega = \text{target}_n^t \neq \perp$ 使它在 d_n 上全并取值于 $\text{dom}(F^t)$, 因为 target 指称的是提供方。由表 1, 收缩 $\text{dom}(F_\gamma)$ 的唯一规则是 O-Remove, 其前提 $\theta_n^t = \text{Inactive}(-)$ 给出 $\neg \text{installed}_n^t$, 从而由 γ^t 处的条款 (4), 不存在 m 与 $k \in d_m$ 在 installed_m^t 之下使 $\omega_m^t(k) = n$; 而 n 自身不携带任何 ω 。

(4) 由引理 54(2) 与 (4), 该条款只可能在如下情形于 γ^{t+1} 处失效: 某个已安装者跌落、某个 ω 被写出、或某个 ω 所指称的线程离开了 $\text{dom}(F_\gamma)$ 。最后一种是 O-Remove, 其移除的线程并非已安装, 从而由 γ^t 处的条款 (4), 没有任何已安装的 m 的 ω_m^t 指称它。第一种是 n 的一次 L-Unload, 其前提 $\neg \text{relied}_n^t$ 读作

$$\forall m \neq n, k \in d_m. \text{installed}_m^t \Rightarrow \omega_m^t(k) \neq n$$

且它不为 $m \neq n$ 写任何 ω_m , 并留下 $\neg \text{installed}_n^{t+1}$, 故条款对 n 也成立。第二种是 n 的一次 L-Begin, 写出 target_n^t , 其值是 d_n 诸键的提供方, 从而在 γ^t 处皆为 Active; 该步不改任何其他线程的 θ , 故它们在 γ^{t+1} 处也照旧已安装。□

L-Unload 上的守卫 (guard) 正是承载条款 (3) 与 (4) 的东西。O-Remove 的前提 $\forall m. \pi_m \neq n$ 只谈及父指针; 使一个已提交视图 (committed view) 不指称被移除线程的, 是那个在早若干步、且出于另一种理由施加的守卫。由于失败同样经由 Unloading 路由, 该论证无须对错误结局再重复一遍。由此得到基础演算所没有的两点: 被 O-Remove 释放的名字可由 O-Insert 重新发放, 因为没有任何过时的已提交视图能指称它; 而且线程一旦 Inactive 即可被移除, 无须另行检查是否有人依赖它。

4.6 时间性可组合

局部时间性可组合用一个累加器恢复一个副作用序列 (3.1.3 节)。注册表为每个线程各持有一个累加器, 而线程交错运行: 在 n 把一个逆复合到 g_n 上的时刻与 g_n 运行的时刻之间, 其他线程已经移动了状态。 g_n 是否仍能撤销它被构造来撤销的东西, 正是该保证的全局形式所断言的, 而它所依赖的条件是中间那些步与 g_n 交换。

定义 4.18. 对 $i \in \mathcal{C}_\Gamma^{\text{iter}*}$, 令 $\text{reach}(i)$ 为包含 i 且在续体下封闭的最小迭代器集合, 并在一个迭代器处解读定义 17 的变换幺半群 $\mathfrak{M}$, 取其生成元为 $\text{reach}(i)$ 中每个迭代器的前向映射与产出的逆:

$$\begin{aligned} \text{reach}(i) &:= \bigcap \{S \mid i \in S \wedge \forall i' \in S, \gamma \in \Gamma. i'(\gamma) = (-, -, \text{Just}(i'')) \Rightarrow i'' \in S\} \\ \mathfrak{M}(i) &:= \langle \{\text{pr}_1 \circ i' \mid i' \in \text{reach}(i)\} \cup \{\text{pr}_2(i'(\gamma)) \mid i' \in \text{reach}(i), \gamma \in \Gamma\} \rangle \end{aligned} \quad (4.23)$$

其中在 4.3.4 节适用的地方, 围绕该三元组解读 **Right**; 并对续体所排序的链 $C \subseteq \text{reach}(i)$, 写 $\text{len}(i)$ 为 $|C|$ 的上确界。两个迭代器 i, j 独立, 当它们在定义 19 的意义上独立——按这些变换幺半群来解读, 并把一次迭代的产出解读为它的逆连同它的续体:

$$\begin{aligned} \forall f \in \mathfrak{M}(i), g \in \mathfrak{M}(j). \quad f \circ g \simeq g \circ f \\ \forall i' \in \text{reach}(i), g \in \mathfrak{M}(j), \gamma \in \Gamma. \quad \text{pr}_{2,3}(i'(g(\gamma))) \simeq \text{pr}_{2,3}(i'(\gamma)) \end{aligned} \quad (4.24)$$

且对 j 对称, 其中映射上的 $\simeq$ 按定义 36 解读, 续体上的 $\simeq$ 按定义 51 解读, 注册迭代 (定义 47) 上的 $\simeq$ 解读为它所指名的组件之间的一致。迭代器族 $(i_l)_{l \in L}$ 两两独立, 当对每个 $l \neq l', i_l$ 与 $i_{l'}$ 独立; 步序列两两独立, 当 $(e_n)_{n \in N}$ 如此, 其中 N 是该序列曾经持有过的名字集——编排器 (orchestrator) 插入的每个线程与每次迭代注册的每个线程各占一个。

这种意义上的独立性正是迹理论 (trace theory) 当作本原的东西: 交换的动作在序列上生成一个等价关系, 在其下重排两个相邻的独立动作保持终点不变 [44], 而引理 71 就是这些规则之下的那种重排。用族而非集合, 为的是让同一组件的两个名字仍在范围之内: 此时条件要求该组件的副作用函数与自身独立, 也就是要求 $\mathfrak{M}(i)$ 交换。定理 61 用到的是第一个条件, 定理 73 额外需要第二个: 重排两个线程的步, 会在被另一线程移动过的状态处求值一个迭代器, 而映射的交换本身并不说明该迭代器在那里产出同一个逆与同一个续体。检验第一个条件只需要迭代本身, 因为引理 18(1) 把交换性从生成元带到它们所生成的幺半群。

在这些条件下, 定理 7 的单累加器不变式在交错中幸存, 且正是以赋予时间性可组合其内容的形式: 运行一个逆, 撤销该线程的贡献而无其他。

定理 4.19 (恢复精确性). 设步序列两两独立, n 的一个活动期 (episode) 在 b 处开启, $u \geq b$ 落于其内, 且 $t_1 < \dots < t_l$ 是 $[b, u)$ 中行动线程不是 n 的那些指标。则

$$g_n^u(\gamma^u) \approx (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b) \quad (4.25)$$

即, 在 γ^u 处施加 n 的累加器, 在控制字段之外得到正是同样这些步从 γ^b 出发所产生的状态。把右边读作“ n 从未开始”时所达的状态, 则额外假定了 n 注册的线程没有一个在 $[b, u)$ 中步进, 因为 n 注册的线程本不会在场去步进。

证明. 对 u 归纳, 遍历使 $u+1$ 落在活动期内的指标 u 。在 $u=b$ 处, $b-1$ 处的步是 L-Begin (依定义 53 活动期开启), 故由表 1 有 $g_n^b = \text{id}_\Gamma$, 指标集为空, 断言即 $\gamma^b \approx \gamma^b$ 。每一步用到两个事实。由于 edit^t 只写控制字段,

$$\gamma^{t+1} \approx \Psi^t(\gamma^t)$$

又由于 $\mathfrak{M}(e_n)$ 中的每个映射除注册所添加者外不写任何控制字段 (依定义 48 连同定义 47), 每个这样的映射把 $\approx$ -相等的状态送到 $\approx$ -相等的状态。

设步 u 作用于 n 。由于活动期在 u 与 $u+1$ 处皆开启, 引理 54(4) 排除了 n 的 L-Begin 与 L-Unload, 而 O-Insert 与 O-Remove 读到的 θ_n 被 installed_n^u 否定, 故余下两种情形。当规则为

L-Iter、L-Finish 或着陆式 L-Divert 时，表 1 给出 $\Psi^u = \text{pr}_1 \circ i_n^u$ ，且 $g_n^{u+1} = g_n^u \circ h$ ，其中 h 是该迭代产出的逆。定义 51 的见证条件读作 $h(\Psi^u(\gamma^u)) = \gamma^u$ ，在迭代注册纤程之处则仅到 $\approx$ 为止 (引理 57)，而 g_n^u 依上面的等式保持 $\approx$ ，故

$$g_n^{u+1}(\gamma^{u+1}) \approx (g_n^u \circ h)(\Psi^u(\gamma^u)) = g_n^u(\gamma^u)$$

当规则为 L-Leave、L-Raise、中止式 L-Divert 或 n 的 O-Retire 时，表 1 给出 $\Psi^u = \text{id}_\Gamma$ 且 $g_n^{u+1} = g_n^u$ ，故同一等式以 $h = \text{id}_\Gamma$ 成立。无论哪种情形，归纳假设都在指标集不变之下延续，这正是定理 7 的计算逐步推进的样子。

设步 u 作用于 $m \neq n$ 。则由表 1 有 $g_n^{u+1} = g_n^u$ ，且 $\Psi^u \in \mathfrak{M}(e_m)$ ，或者在规则为编排规则时 $\Psi^u = \text{id}_\Gamma$ ，故独立性给出

$$g_n^u(\gamma^{u+1}) \approx g_n^u(\Psi^u(\gamma^u)) = \Psi^u(g_n^u(\gamma^u))$$

这就是追加了 Ψ^u 之后的归纳假设。 $\square$

推论 4.20 (终态恢复). 设步序列两两独立， n 的一个活动期在 b 处开启并在 u 处闭合，无论 n 到达何种结局。则依定理 61 取 $t_1 < \dots < t_l$ ，

$$\gamma^{u+1} \approx (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b) \quad (4.26)$$

被 O-Remove 移除的纤程同样不留痕迹，其前提只容许 $\theta_n = \text{Inactive}(-)$ 。

证明. 由引理 54(4)，步 u 是 n 的一次 L-Unload，其 Ψ^u 依引理 54(3) 即 g_n^u ，故 $\gamma^{u+1} \approx g_n^u(\gamma^u)$ ，定理 61 适用。陈述与 $\approx$ 都不提及 ζ ，而由表 1， ζ 正是 L-Divert 与 L-Raise 所导致状态唯一不同的字段。 $\square$

上述结果把两两独立当作对组件的假定，而 3.6.2 节正是解除这一假定的地方：当组件执行的每个副作用都是某个键的操作、且每个键都交换时，由这些操作构造的任何两个副作用函数都独立 (定理 42)。把该结果从副作用函数带到迭代器不需要任何新东西：经协副作用中介的副作用函数 (定义 41) 已经按每一阶段产出的结果选择其后继，而这正是迭代器在其续体中携带的东西。3.2 节的协副作用操作是完全无须假设的情形：组件在那里贡献的映射是集合操作与相应限制的复合，两个这样的映射只要触及不相交的键就交换，而定义 58 的条款 (2) 使不同纤程的提供集两两不交。

4.7 空间性可组合

局部空间性可组合让组件遵守自己的规约，只在依赖得到提供之处激活它，并针对依赖对每个上下文变更分类 (3.4 节)。全局形式添加的是对其他纤程做量化的内容：提供方只在解析了它的每个依赖方都已失活之后才撤离一个绑定，且转移据以安装其副作用的解析不在其下移动。协副作用一侧的两个性质分别交付这两点，而且它们是一起证明的，是同一个不变式的两半，即引理 54(2) 所建立的 ω_n 在一个活动期上的固定性。次序定理是这一固定性在活动期中 n 先为 Active 尔后为 Unloading 的部分所换得的东西，一致性定理则是它在 n 正在安装其副作用的部分所换得的东西。

定理 4.21 (次序). 纤程只在依赖得到提供之处开始转移：

$$\text{step}^t = \text{L-Begin}(m) \Rightarrow \gamma^t \models d_m \quad (4.27)$$

进一步设 $[b', u']$ 是 m 的一个活动期, 其中 $\omega_m^{b'}(k) = n$, $m \neq n$ 且 $k \in d_m$; 设 $[b, u]$ 是包含 b' 的 n 的活动期, 且 t 遍历 $[b', u']$ 。则

1. $\omega_m^t(k) = n$;
2. $b < b'$, 且当 $[b, u]$ 闭合时 $u' < u$;
3. $k \in \text{dom}(\sigma_n^t)$ 且 $\sigma_n^t(k) = \sigma_n^{b'}(k)$ 。

证明. 第一个断言即 L-Begin 的前提 $\text{target}_m^t \neq \perp$, 它依定义 46 给出 $\gamma^t \models d_m$ 。

(1) 即引理 54(2)。

对 (2), $b' - 1$ 处的 L-Begin 写出 $\omega_m^{b'} = \text{target}_m^{b'-1}$, 其值是提供方, 故 $\theta_n^{b'} = \text{Active}(-, -)$; $b - 1$ 处的 L-Begin 留下 $\theta_n^b = \text{Reloading}(-, -, -)$, 故 $b \neq b'$, 从而 $b < b'$ ——两个活动期都依定义 53 开启。设 $[b, u]$ 闭合且假设 $u \leq u'$ 。则 $u \in [b', u']$, 故 installed_m^u , 且由 (1) 有 $\omega_m^u(k) = n$; 这就是 relied_n^u , 而 u 处的 L-Unload 否定它。故 $u' < u$ 。

对 (3), n 是 $\gamma^{b'}$ 处 k 的提供方, 故 $k \in \text{dom}(\sigma_n^{b'})$ 。没有 n 的 L-Unload 落在 $[b', u']$ 内: 当 $[b, u]$ 闭合时, 它由 (2) 落在 $u > u'$ 处; 当 $[b, u]$ 不闭合时, 引理 54(4) 使 n 根本没有 L-Unload。由于 $\theta_n^{b'} = \text{Active}(-, -)$, 表 1 因此只留下 L-Leave 作为 $[b', u']$ 内可以作用于 n 的规则, 而其 Ψ^t 为 id_Γ ; 由引理 54(1), σ_n 在该区间上为常量。□

跨步展开的转移, 否则可能安装依据一个已在其下改变的解析所计算出的副作用, 而有两条前提阻止这一点。L-Iter 与 L-Finish 携带 $\text{target}_n(\gamma) = \omega$, 故转移只在其已提交视图仍是其目标视图 (target view) 时推进; L-Divert 携带其否定, 故目标视图的任何变更都把纤程带出转移。L-Raise 完全不以目标视图为条件: raise 是迭代自身所做的事情, 而非环境所请求的事情, 而且无论如何它都退出转移。变化的两个方向不被区分: 依赖已消失的组件与依赖已被替换的组件走同一条路离开, 因为变成 $\perp$ 的目标视图与变成另一纤程的目标视图, 同样地不等于 ω 。

惯性使这不成为关于每一步的保证。目标视图翻转时尚在飞行中的迭代仍会着陆 (经 L-Divert), 而那次着陆安装的副作用是依据一个不再成立的解析计算的。因此规则所交付的是一个析取, 而第二个分支正是使第一个分支安全的东西。

定理 4.22 (解析一致性). 设 n 的活动期 $[b, u]$ 在 b 处开启, 且 $\omega_n^b = \omega$ 。则 θ_n 在活动期的一个初始区间 $[b, r]$ 上为 $\text{Reloading}(-, -, -)$, 且转移的每次迭代都对着同一个解析 ω 运行:

$$\forall t \in [b, r]. \text{step}^t \in \{\text{L-Iter}(n), \text{L-Finish}(n)\} \Rightarrow \text{target}_n^t = \omega \quad (4.28)$$

当纤程离开该区间 (即 $r < u$) 时, 恰有以下二者之一成立:

1. $\text{step}^r = \text{L-Finish}(n)$ 且 $\theta_n^{r+1} = \text{Active}(-, \omega)$;
2. $\text{step}^r \in \{\text{L-Divert}(n), \text{L-Raise}(n)\}$, 且活动期在某个 $u > r$ 处闭合, 并依推论 62 有 $\gamma^{u+1} \approx (\Psi^{t_1} \circ \dots \circ \Psi^{t_1})(\gamma^b)$ 。

证明. $b - 1$ 处的 L-Begin 写出 Reloading , 且由表 1, 它是通向该生命周期状态的唯一规则; 其前提 $\theta_n = \text{Inactive}(\perp)$ 与引理 54(4) 把它的任何第二次应用排除在活动期之外。所以 Reloading 占据 $[b, u]$ 的一个初始区间 $[b, r]$, 且不会被重新进入。

第一个断词于是就是表 1 给 L-Iter 与 L-Finish 的前提 $\text{target}_n(\gamma) = \omega'$, 连同依引理 54(2) 的 $\omega' = \omega$ 。

对二分法, 步 r 是一条前提含 $\theta_n = \text{Reloading}(-, -, -)$ 而结论不含的规则, 表 1 给出 L-Finish、L-Divert 与 L-Raise 三者; 第一个落入 $\text{Active}(-, \omega)$, 后两个落入 $\text{Unloading}(-, \omega, -)$, 而由引理 54(4), L-Unload 是那里唯一的出口, 推论 62 供给该等式。着陆式 L-Divert 所贡献的迭代是 n 自己的迭代之一, 故属于该累加器撤销的那些映射。而当 $r = u$ 时, 序列以仍在飞行中的转移结束, 第一个断言即全部所断言者。□

4.8 进展

把提供方的撤离推迟到其依赖方消失的守卫, 只有最终会释放, 才能交付定理 63。注册表线程上的一个关系承载该论证。

定义 4.23. 注册表诸名字上的先行关系 (*precedence relation*) 为

$$n \prec m := p_n \cap d_m \neq \emptyset \quad (4.29)$$

因而 n 可以提供 m 所声明的某个键。它只读 d 与 p , 而由引理 54(5), 二者随线程的进入而存在, 此后不再被写。

定理 66 与定理 73 都在 $\prec$ 无环的假设之下建立, 这是一个假定, 而非该定义所交付的东西: 声明一个由自己提供的键的组件会使 $n \prec n$ 成立。 $\prec$ 所排序的是两个线程的激活而非它们的寿命: $n \prec m$ 说的是 n 必须先成为 **Active**, m 才能如此; 而提供方比其消费者活得久, 则是定理 63(2), 一条关于受守卫演算的定理。

线程的目标视图除了应答其提供方之外, 还应答创建它的线程。创建者所写的是 τ_n , 经由定义 47 的原语, 而 τ 依引理 54(5) 是单调的。因此在一个子线程的整个存在期间, 创建者至多使其目标视图翻转一次。

进展是“某条规则适用”的断言, 故它针对宿主 (host) 必须提供的规则来表述: L-Begin、L-Leave、L-Unload, 着陆规则 L-Iter、L-Finish 与 L-Raise, 以及 L-Divert。它何处也不诉诸 L-Divert 的中止选择, 故受 4.3.3 节惯性约束的宿主也被覆盖。

定理 4.24 (进展). 假设 $\prec$ 无环、对每个 n 有 $\text{len}(e_n) \leq K$ 、且定义 60 的名字集 N 有限; 并设每一步都应用一条生命周期规则。写 $S(n)$ 为作用于 n 的步数, 并写

$$V(n) := |\{t : \text{target}_n^t \neq \text{target}_n^{t+1}\}| \quad (4.30)$$

为其目标视图翻转的次数。则

1. (无死锁。) $\neg \text{quiet}^t$ 蕴含某条生命周期规则在 γ^t 处适用;
2. (终止。) $S(n) \leq (K + 4)(V(n) + 1)$, 且 $V(n)$ 与 $\sum_n S(n)$ 都有限。因此每个极大的生命周期步序列终结于一个静寂状态。

证明. 无死锁。设 $\neg \text{quiet}^t$, 则某线程 n 不满足定义 49 的静寂的任何一条。对照表 1, 它只能是以下四种情形之一:

- $\theta_n^t = \text{Inactive}(\perp)$ 且 $\text{target}_n^t \neq \perp$: L-Begin 适用;

- $\theta_n^t = \text{Reloading}(-, -, \omega_n)$ 且 $\text{target}_n^t = \omega_n$: $i_n^t(\gamma^t)$ 的值所选中的 L-Iter、L-Finish 与 L-Raise 三者之一适用;
- $\theta_n^t = \text{Reloading}(-, -, \omega_n)$ 且 $\text{target}_n^t \neq \omega_n$: 若 $i_n^t(\gamma^t)$ raise 则 L-Raise 适用, 否则 L-Divert 适用, 让该迭代着陆而非中止;
- $\theta_n^t = \text{Active}(-, \omega_n)$ 且 $\text{target}_n^t \neq \omega_n$: L-Leave 适用。

设没有线程属于上述任何一种情形, 则存在某个 m_0 使 $\theta_{m_0}^t = \text{Unloading}(-, -, -)$ 。如下构造 $m_0, m_1, \dots$: 给定处于 Unloading 的 m_j , 或者 $\neg \text{relied}_{m_j}^t$, 此时 L-Unload 对 m_j 适用而构造停止; 或者存在 $m_{j+1} \neq m_j$ 与 k_j 使 $\text{installed}_{m_{j+1}}^t$ 且 $\omega_{m_{j+1}}^t(k_j) = m_j$ 。后一情形中

$$k_j \in d_{m_{j+1}} \cap \text{dom}(\sigma_{m_j}^t) \subseteq d_{m_{j+1}} \cap p_{m_j}$$

第二个属于关系是定理 63(3) 在包含 t 的 m_{j+1} 的活动期处的结论, 故 $m_j \prec m_{j+1}$ 。此外 $\text{target}_{m_{j+1}}^t \neq \omega_{m_{j+1}}^t$: Unloading 的线程不在定义 σ_γ 的并之内, 故 k_j 在 γ^t 处未被提供、或由 m_j 以外的线程提供。倘若 m_{j+1} 处于 Active 或 Reloading, 它便是被排除的四情形之一, 故它处于 Unloading, 构造继续。诸 m_j 按 $\prec$ 递增, 由无环性故两两不同, 而 $\text{dom}(F^t)$ 有限, 故构造停止。

终止。两个断言界定 $S(n)$ 。

(A) 在 target_n^t 恒为 ω^* 的一个极大区间上, 至多 $K+4$ 步作用于 n 。读表 1 的 θ_n 各列: 从 $\text{Active}(-, \omega)$ ($\omega \neq \omega^*$) 出发, 线程经历一次 L-Leave 与一次 L-Unload, 随后若 $\omega^* \neq \perp$ 则是一次 L-Begin 与至多 $\text{len}(e_n) \leq K$ 次着陆, 外加最后一次着陆为 L-Raise 时的第二次 L-Unload; 从 Reloading 对着一个 $\omega \neq \omega^*$ 出发, 则以一次 L-Divert 代替 L-Leave; 从任何其他状态出发则是该序列的后缀。区间内没有更多的 L-Divert 或 L-Leave 落入, 因为 L-Begin 写下的 ω 正是 $\text{target}_n^t = \omega^*$ 自身; 而在 $\text{Active}(-, \omega^*)$ 处、在 $\omega^* = \perp$ 的 $\text{Inactive}(\perp)$ 处、以及在 $\text{Inactive}(\xi)$ 处, 根本没有规则适用。

(B) 若 $\text{target}_n^t \neq \text{target}_n^{t+1}$ 且步 t 作用于 m , 则或者 $m \prec n$, 或者步 t 写 τ_n 。依定义 46, target 的值是 τ_n 与 d_n 诸键的提供方之表的函数; 提供方满足 $k \in \text{dom}(\sigma_m) \cap d_n$, 故 $m \prec n$; 而由引理 54(1), 一张表只在作用于其自身线程的步处改变。无环性在第一种情形给出 $m \neq n$, 引理 54(5) 的单调性在第二种情形对每个线程至多容许一个 t 。

由 (A), 区间计数界定 $S(n) \leq (K+4)(V(n)+1)$; 由 (B), target 的每次翻转或者消耗一个严格 $\prec$ -低于 n 的线程的一步, 或者是 τ_n 所容许的那一次翻转, 故 $V(n) \leq 1 + \sum_{m \prec n} S(m)$ 。由于 $\prec$ 无环且 N 有限, 递归

$$B(n) := (K+4) \left(2 + \sum_{m \prec n} B(m) \right)$$

良基, 并定义了满足 $S(n) \leq B(n)$ 的 B ; 故 $V(n)$ 有限, 且 $\sum_n S(n) \leq \sum_n B(n)$ 。由 (1), 无法延伸的序列是静寂的。□

N 的有限性是假定而非推导, 而对组件的一个条件即可交付它。宿主持有的组件是运行之前给定的有限多个程序, 因此只要没有组件能够——无论多么间接——注册一个“会注册其自身组件之线程”的组件的线程, 注册就形成一棵深度有界的树, 而 $\text{len}(e_n) \leq K$ 界定其分支。该假定所排除的, 是无界地注册自身实例的组件。

目标记录的是提供方线程而非布尔值, 而在 4.2 节的单源纪律下, 二者驱动相同的转移, 因为在那里一个键只有一个可能的提供方。视图买到的是上述诸结果的词汇表: 定理 63 与定

理 64 都谈及线程据以激活的那个解析；它也是使那些结果在 3.5 节的作用域化解析下幸存的东西——在那种解析下，同一个键在不同域中解析到不同的提供方，提供集不再强制该视图。实现携带这一作用域化，并把该视图保存于 `fiber.committed` (5.1.3 节)。

4.9 合流性

到此为止的结果都是关于单个线程的。把系统作为整体加以刻画性质是：其动态历史不留痕迹。无论一个运行中的系统经历过怎样的激活与失活序列，它静寂于其上的状态，就是同样的插入与退役在”每个最终处于 **Active** 的组件各按依赖次序被加载一次、且从不卸载”的情况下所产生的状态。生命周期关系是合流的，而它收敛到的正规形是静态装配的那一个。对于动态组合而言，这是变更传播为增量计算所建立的”与从头求值一致”性质的对应物 [45]。

该断言仅关于 $\rightarrow$ 。编排步是输入，被给予不同输入的两个序列出于无趣的原因落在不同的地方；问题在于生命周期规则——它们在”下一个步进的是哪个线程”与”一个 Reloading 线程取哪个出口”上是非确定性的——能否被弄得互相矛盾。

先需要三条引理。第一条把最终处于 **Active** 的线程集合固定下来，而不提及任何步序列，这正是使它成为输入的函数、而非调度之函数的东西。

定义 4.25. 一个线程在 γ 处受支撑 (*supported*)，当它未被退役、注册它的线程受支撑、且它声明的每个键都由某个受支撑的线程提供。 $\text{dom}(F_\gamma)$ 上的支撑关系是这些条款所读的两个关系的并，

$$m \triangleleft n := m \prec n \vee \pi_n = m \quad (4.31)$$

在它良基之处 (引理 68)，我们把 A 写作支撑集，即 γ 处受支撑的线程：

$$n \in A := \neg \tau_n \wedge (\pi_n = \text{root} \vee \pi_n \in A) \wedge \forall k \in d_n. \exists m \in A. k \in p_m \quad (4.32)$$

其中 $\pi_n = \text{root}$ 标记编排器插入的线程，否则 π_n 是其激活注册 n 的那个线程。诸条款只读 τ, π, d, p 四个字段。两半都把一个线程与紧邻其下的一个相关联——是父者而非祖先、是直接提供方而非传递提供方——因为条款读到的就是这些；当下面的结果需要一个次序时，它们取传递闭包，其极小元、极大元与线性化都是 $\triangleleft$ 的。

诸条款提及 A 自身，故该定义是沿 $\triangleleft$ 的递归，而使它成为有解递归的正是下面这条。

引理 4.26 (支撑是良基的). 设 $\prec$ 无环，且 γ 由一个步序列到达。则 $\triangleleft$ 良基，且 A 是定义 67 的唯一解，是 τ, π, d 与 p 的函数。

证明. 按注册各名字的步指标排列 $\text{dom}(F_\gamma)$ 的名字，定义 53 通过让序列从空注册表出发供给了这一指标。 $\triangleleft$ 的父指针半在该指标中下降：O-Insert 以 $\pi \in \text{dom}(F_\gamma)$ 为前提，故父指针指称一个较早注册的线程，而迭代它便在有限步内到达一个名字的整条祖先链。于是环必须用到 $\prec$ ，而 $\prec$ 无环，故必须混合二者，这需要某个 m 声明一个由 m 自身子树中的线程可能提供的键。这样的线程由 m 或 m 的某个后代的激活注册，故注册于 m 的 L-Begin 之后的一个步；那个 L-Begin 以 $\gamma \models d_m$ 为前提，故提供该键的线程在它之前便已 **Active**，而定义 58 的条款 (2) 不给该键第二个可能的提供方。于是本会闭合环的线程从未被注册，该边在 $\text{dom}(F_\gamma)$ 中缺席。良基递归有唯一解，而诸条款只读那四个字段。□

最后一条条款读的是 p ，即组件可以提供的键，而目标读的是 $\text{dom}(\sigma_\gamma)$ ，即其线程已安装的键，定义 43 仅以 $\text{dom}(\sigma_n) \subseteq p_n$ 联系二者。因此支撑集一般是对 **Active** 线程的过近似，而弥合这一差距的条件如下。

定义 4.27. 一个组件 (d, p, e) 在其提供集上是全的 (*total on its provision*), 当它的一个得以完成的激活已安装了 p 的每个键, 从而在每个实例化它的 **Active** 线程处 $\text{dom}(\sigma_n) = p_n$ 。

与独立性 (定义 60) 一样, 这是仅关于组件的条件, 不提及任何生命周期状态与任何步, 而独立性已经限定了它可能失效的程度: 倘若一个组件只在另一组件的副作用所能到达的上下文状态处才安装某个键, 其前向映射便不与该组件的交换, 故线程安装哪些键由其组件固定, 而非由调度固定。全性添加的是: 被固定的集合是整个 p , 而非其真子集。

引理 4.28 (静寂处的支撑). 设 $\prec$ 无环、 $\text{quiet}(\gamma)$ 、 γ 的线程无一失败、且 γ 的每个组件都在其提供集上是全的 (定义 69)。则支撑集就是 **Active** 线程之集:

$$A = \{n : \theta_n = \text{Active}(-, -)\} \quad (4.33)$$

证明. 把右边记作 A' 。既然没有线程失败, 定义 49 的静寂只留下 **Inactive**($\perp$) 与 **Active** 两种状态, 并读作

$$n \in A' \iff \text{target}_n(\gamma) \neq \perp$$

依定义 46, 右边成立恰当 $\neg \tau_n$ 且每个 $k \in d_n$ 落在 $\text{dom}(\sigma_\gamma)$ 中, 而由定义 69 有 $\text{dom}(\sigma_\gamma) = \bigcup_{m \in A'} p_m$ 。中间那条条款是 target 不再携带的, 而注册供给它: $\pi_n \neq \text{root}$ 的线程只由 π_n 的激活注册, 而若 $\pi_n \notin A'$, 则 π_n 不是 **Active**, 故其累加器已经运行, 并依定义 47 退役了 n , 给出 τ_n 。于是 A' 满足定义 67 的诸条款, 而引理 68 给出唯一解, 故 $A = A'$ 。□

引理 4.29 (换位). 设诸步两两独立、 F^t 良构, 且步 t 与步 $t+1$ 作用于不同的线程 m 与 n 。

1. 若两步都应用激活规则, 即 *L-Begin*、*L-Iter* 或 *L-Finish*, 且步 $t+1$ 在 γ^t 处适用, 则步 t 在步 $t+1$ 从 γ^t 产生的状态处适用, 且两种次序到达同一个 γ^{t+2} 。
2. 若步 t 在 m 处应用激活规则、步 $t+1$ 在 n 处应用编排规则、且步 t 不注册 n , 则两步同样如此。

证明. 对 (1), 由表 1, m 的步写 θ_m , 并在 $\Psi^t \in \mathfrak{M}(e_m)$ 之内写表 σ_m 与副作用部分。因此它不触动 θ_n 与 i_n , 且由定义 60 的第二个条件, 也不触动 i_n 产生的逆与续体, 故只剩步 $t+1$ 中提及 target_n 的前提有待检验。其退役半不可能跌落, 因为没有激活规则写任何 τ 。其解析半也不可能移动: 步 $t+1$ 在 γ^t 处适用, 把每个 $k \in d_n$ 放进了 $\text{dom}(\sigma^t)$, 而定义 58 的条款 (2) 使提供这样一个 k 的线程是唯一可能的提供方, 故 $k \notin p_m$, σ_m 的任何写都够不到 d_n 的键。同一论证换个方向便知步 t 保持适用。最后, $\Psi^t \in \mathfrak{M}(e_m)$ 与 $\Psi^{t+1} \in \mathfrak{M}(e_n)$ 依定义 60 的第一个条件交换, 且两次编辑写的是不同线程的控制字段, 故复合在两种次序下相同。

对 (2), 编排步由表 1 有 $\Psi^{t+1} = \text{id}_\Gamma$, 故两个状态映射彻底交换; 其 edit^{t+1} 只在 n 处写 τ_n 或 $\text{dom}(F_\gamma)$, 而激活步对二者既不读也不写: 后者的前提读 θ_m, i_m, τ_m 与 target_m , 且新鲜 n 的 *O-Insert* 不移动任何目标——新鲜的线程不提供任何东西——而 n 的 *O-Retire* 或 *O-Remove* 让 σ_γ 原地不动, n 在一种情形是 **Inactive**, 在另一种情形其表未受影响。故步 t 保持适用。反过来, 编排步的每条前提或者读在 n 处——步 t 不写它——或者是 *O-Insert* 的那两条被更小的注册表所放松的前提之一, 于是它在 γ^{t+1} 处的适用给出它在 γ^t 处的适用; 这里, 步 t 不注册 n 正是使 n 在 γ^t 处保持在场的东西, 而 *O-Retire* 与 *O-Remove* 都要求它在场。□

引理 4.30 (删除). 设步序列两两独立、每个组件都在其提供集上是全的 (定义 69)、序列到达一个无线程失败的静寂 γ^T 、 $[b, u]$ 是 n 的一个闭合活动期、序列中没有任何满足 $n \prec m$ 的 m 的

活动期闭合、且 n 在 $[b, u]$ 期间注册的线程都没有活动期。把那些注册抽到的名字记作 R 。则删除 $[b, u]$ 中作用于 n 的诸步，连同作用于 R 中名字的每个步，留下一个到达与 γ^T 为 $\approx$ -相等、且在 R 之外与之 $\simeq$ -相等的状态的步序列。

证明. 被删除的诸步把状态留在它们找到它的地方。设 $t_1 < \dots < t_l$ 是 $[b, u]$ 中作用于 n 以外线程的诸步。推论 62 读作

$$\gamma^{u+1} \approx (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b)$$

其右边正是 $[b, u]$ 的幸存诸步独自产生的东西： $\gamma^{b-1} \approx \gamma^b$ ，且它们的编辑写的是 n 以外线程的控制字段，删除并不触及。由表 1， n 的被删除诸步除 θ_n 外不写任何字段，而引理 54(4) 在 u 处把 θ_n 恢复为 $\text{Inactive}(\perp)$ (没有线程失败)，也正是它在 γ^{b-1} 处所有的。

一个不变式携带后缀。把幸存诸步在与 t 对应之点到达的状态记作 γ'^t 。我们断言：对每个 $t > u$ ，都有 $\gamma^t \approx \gamma'^t$ ， R 的每个名字在 γ^t 处都是残留 (vestigial) 的且不在 γ'^t 中，且两个状态在 R 以外每个名字的每个字段上一致。在 $t = u + 1$ 处，这是上面一段连同定义 47：后者让 R 的每个名字被在 u 处运行的累加器退役，处于 $\text{Inactive}(\perp)$ 且持有空表，而 R 的诸线程依假设没有活动期。归纳步是引理 57(1) 依次在 R 的每个名字处应用：作用于 R 之外的步在两个状态处有相同的前提，到达再度 $\approx$ -相等的状态，且让 R 的条目保持残留。作用于 R 中名字的步是被删除的步之一，而引理 57(2) 正是它必须被删除而非保留的原因：缺席名字上的 O-Retire 或 O-Remove 没有线程可作用；由 (1)，这样的步同样不移动 R 之外的任何字段，故丢弃它保持不变式。因此最终状态 $\approx$ -相等，且在 R 之外相等。

没有幸存的步丢失前提。作用于 $m \notin R \cup \{n\}$ 的步只经 $\text{target}_m(\gamma)$ 或 $\text{relied}_m(\gamma)$ 读 n 。前者在 m 声明一个 n 提供的键时依赖 n ——故 $n < m$ ——且在 n 注册 m 时依赖 n ，后者把 m 放入 R 。第一种情形中， m 的活动期依假设不闭合，故它在 γ^T 处开启，静寂给出 $\omega_m = \text{target}_m^T$ ，引理 70 把它的值置于 Active 线程之中，而 n 不在其列；由于一个键至多有一个可能的提供方， n 在 m 的 L-Begin 处也没有提供过 d_m 的任何键。后者只经 ω_n 的值读 n ，而删除活动期只会让 relied 为假，这放松 L-Unload 上的守卫而非阻塞它。这样的步对 R 中名字所读的东西由不变式覆盖。两两独立是副作用函数的性质，故删除诸步保持它。 $\square$

定理 4.31 (合流性). 设一个步序列到达一个无线程失败的静寂 γ^T 、诸步两两独立、每个组件都在其提供集上是全的 (定义 69)，且 A 依定义 67。则

1. (规范形。) γ^T ——除归约所撤除条目的那些名字外——由 γ^0 经如下序列到达：该序列以原有次序取同样的编排步，作用于编排器所插入线程的那些先于每个生命周期步，其余每个都紧随注册它所作用线程的那一步之后；并且对 A 的任一线性化 $\triangleleft$ 的枚举 $n_1, \dots, n_k$ ，按该次序取每个 n_i 的恰好一个活动期。
2. (合流性。) 从 γ^0 出发取同样编排步的任何两个这样的序列，到达经引理 56 式的重命名后由 $\simeq$ 与 $\approx$ 相关联的状态。

证明. 对 (1)，序列的活动期分两类：闭合的那些，与在 γ^T 处仍开启的那些——后者依 quiet^T 与引理 70 是 A 的每个线程各一个活动期。

闭合活动期先行，对其数量归纳。每一阶段选取一个线程 n 的闭合活动期，使 n 在活动期仍闭合的诸线程中 $\triangleleft$ -极大；由引理 68 与 N 的有限性，这样的线程存在。引理 72 的三个假设随之满足。由极大性，没有满足 $n < m$ 的 m 有闭合活动期。而 n 在 $[b, u]$ 期间注册的线程没有活动期：这样的线程被在 u 处运行的累加器退役 (定义 47)，且由引理 54(5) 保持退役，故其目

标视图为 $\perp$ ，引理 70 把它置于 A 之外，于是它没有在 γ^T 处开启的活动期；而 $\triangleleft$ 经其父指针把它与 n 相连，故由极大性它也没有闭合的活动期。该引理移除这个活动期，连同它注册的名字的诸步，把 γ^T 留在原处——最多相差那些名字。测度每次降一，故不再有闭合活动期。

A 之外的纤程不取生命周期步。由引理 70 与 quiet^T ，它在 γ^T 处没有开启的活动期，而此刻闭合的也不复存在，故它根本没有活动期，且自始至终为 $\text{Inactive}(\perp)$ ；L-Begin 是唯一在那里适用的规则，而应用它会开启一个活动期。

编排步其次。作用于编排器所插入纤程的编排步，经引理 71(2) 越过不同纤程的生命周期步向前移一位；该引理适用，因为 A 之纤程的步不注册这样的名字：注册抽取新鲜名字，而此处的名字是原序列的一次 O-Insert 引入的。与同一纤程的生命周期步之间则无可交换： n 的 O-Insert 已先于 n 的每个步，而 n 的 O-Retire 或 O-Remove 只在 A 之外适用， A 不取生命周期步。逐个把各编排步移到最前，保持其相对次序。作用于某激活所注册纤程的编排步不能到最前，其前提要求该纤程在场，故它留在注册放下它的地方；它作用于 A 之外（见上段），故由引理 71 的同一款，它与它和注册之间的一切交换。

活动期被排序并变得连续，对 $|A|$ 归纳。设 n_1 是 A 中的 $\triangleleft$ -极小元。则 $d_{n_1} = \emptyset$ 且 $\pi_{n_1} = \text{root}$ ，因为定义 67 把 d_{n_1} 之键的提供方与注册 n_1 的纤程放进 A ，而 $\triangleleft$ 把二者置于 n_1 之下。于是 target_{n_1} 不读任何其他纤程的字段，且既没有编排步留下来写 τ_{n_1} 、也没有 n_1 之下的纤程留下来退役它，故它是常量。作用于 n_1 的每一步都是激活步（没有活动期闭合），其余前提读 θ_{n_1} 与 i_{n_1} ，而由表 1 只有 n_1 写它们；故每一步在每个更早的状态处都适用，且引理 71 把它向前移一位而不动终点。先于 n_1 之步的其他纤程步数每次应用降一，故 n_1 的活动期成为一个初始的连续块。论证在块后之缀上对 $A \setminus \{n_1\}$ 重复，那里 n_1 自始至终为 **Active** 且不再步进，故它也贡献一个常量目标。如此产生的枚举由构造线性化 $\triangleleft$ 。

对 (2)，两个序列都由 (1) 归约为规范形，而两次归约在相差一个重命名之下跑过同一个 A 。定义 67 读 τ, π, d 与 p ，其中后三者随纤程的进入一次写出（引理 54(5)），故须看的是：同样的名字带着同样的 d, p 与 π 进入存在，且同样的名字被退役。插入由假设为两序列共享。注册亦然： A 之纤程的激活在它的每次迭代处注册该迭代器在那里指名的组件，而定义 60 的第二个条件使它在交错中保持固定，故 A -纤程之下的注册树是该纤程组件的函数；那些注册抽到的名字不共享，引理 56 正是在此应用，以一个双射匹配两棵树。而退役或者是一次编排步——共享——或者是累加器所取的 O-Retire，它恰好退役同一次激活所注册的名字。线性化 $\triangleleft$ 的两个枚举相差不可比活动期的换位，而引理 71 再次使换位不改变终点，故两个规范序列一致。连同定理 66 的终止性，生命周期关系因此有唯一的正规形。□

失败被排除在陈述之外，因为它是分歧的一个真实来源，不应把该演算读作否认它：一步是否 **raise** 依赖于它运行时所对的状态，故一个调度可能使一个纤程失败而另一个调度完成它，两个静寂状态遂在该纤程的生命周期状态上不同。除此之外它们无处不同，由推论 62，失败纤程对状态的贡献被置为零。

在 4.2 节的基础演算中同一定理成立，且证明无须替换，只须删去一条条款。L-Unload 在那里不携带守卫，故引理 72 的最后一段是空的；该引理其余部分只诉诸 quiet^T ，基础演算原样供给它。

这条定理正是允许把 Cordis 应用当作静态装配来推理的依据。一个添加组件、移除它、替换提供方、再回退该替换的编排器，被保证到达“一开始就写下最终组合”所能得到的状态；而推理哪些协副作用在范围之内的组件作者，可以只对静寂状态推理。它同时也划定了保证的边界：它谈的是状态，而非系统沿途产生的发射（emission）——这正是 6.1 节在边界之内跟踪的

获取 (acquisition) 与跨越边界的发射之间的区分。

第 5 章 实现与案例研究

本章介绍 Cordis，它将第 3 章的形式模型实现为一种实用的编程抽象。Cordis 是一个时空可组合性 (spatiotemporal composability) 的元框架 (meta-framework)：与面向特定领域的应用框架 (如 Web 路由、ORM、UI 渲染) 不同，它不规定任何具体场景；其唯一职责是提供通用的动态组合语义。实现分为三层：(1) 核心库 (5.1 节) 直接实现副作用 (effect) 系统与协副作用 (coeffect) 系统；(2) 组件加载器 (component loader) (5.2 节) 在核心库之上扩展配置调和 (configuration reconciliation) 与热模块替换 (hot module replacement, HMR)；(3) Koishi 等应用框架 (5.3 节) 在前两层之上构建领域专属功能。

5.1 核心库

表 5.1 概括了理论构造与其运行时对应物之间的对应关系。特别地，本节通篇使用下文引入的运行时名称，而将理论符号留给形式对应之用。我们还用 $@@name$ 表示框架内部的符号键 (symbol key)，因此 $ctx[@@store]$ 中的方括号表示按符号键访问上下文上的一个不透明槽位，而非在字符串键映射中做索引。

本节其余部分自底向上构建核心库。5.1.1 节实现可逆副作用 (reversible effects)，它是上下文发生变更的唯一原语；5.1.2 节在其上实现反应式协副作用 (reactive coeffects)；5.1.3 节将二者组合为组件生命周期；5.1.4 节给出建立在其上的上下文级操作。

5.1.1 副作用追踪

本节实现可逆副作用 (3.1 节)。Cordis 中每一次上下文变更都流经唯一原语 $ctx.effect$ ：协副作用的提供、组件实例化，以及所有其他改变上下文的操作，都归约为一次 $ctx.effect$ 调用，因此凡经上下文执行的操作都会被自动追踪，并在组件卸载时被自动恢复。在操作层面， $ctx.effect$ 是 $effect^{iter}$ (定义 52) 的实现：它接受一个类型为 $\mathfrak{C}_T^{iter}$ 的回调，将其提升为 $\mathfrak{C}_{\partial T}^{iter}$ ，并产生一个 $dispose$ 闭包，调用它即可恢复该副作用。Cordis 通过这单一操作同时接受 $\mathfrak{C}_T$ 与 $\mathfrak{C}_T^{iter}$ (即特定多态，ad-hoc polymorphism)；我们以迭代器形式为代表，因为普通副作用函数恰是只产生一个逆变换的退化迭代器。该操作不检查的，是 $\mathfrak{C}_T^*$ 所携带的证据 (witness)：回调提供逆变换，而逆变换确能恢复其伴随的副作用，这是组件作者的义务，而非运行时验证的性质。定理 61 是演算诉诸它的地方，6.1 节则划定了该义务的边界。

Algorithm 1 给出 $ctx.effect$ 的构造。我们记 $f \circ g$ 为先运行 g 后运行 f 的 disposer (资源释放函数)，记 id 为空操作；因此每个新逆变换都被前插，从而得到 LIFO 式恢复。

理论 (第 3、4 章)	实现
Γ_∞	ctx, 一等上下文
$\gamma \in \Gamma$	上下文树, 连同运行中系统触及过的一切
$\mathfrak{C}_\Gamma, \mathfrak{C}_\Gamma^{\text{iter}}$	返回/产出逆变换的副作用回调
$\text{effect}_\Gamma(e)$	ctx.effect(callback)
$\Sigma, \Sigma^{\text{iso}}, \Sigma^{\text{inter}}$	ctx[@@store]、ctx[@@isolate]、 ctx[@@intercept]
$\text{get}(k), \text{set}(k, v)$	ctx.get(key)、ctx.set(key, value)
$\text{isolate}(k, r)$	ctx.isolate(key, realm)
$\text{intercept}(k, \nu)$	ctx.intercept(key, metadata)
$\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$	纤程 (fiber), 组件在 $\mathfrak{C}_\Gamma$ 中的实例化
$\text{dom}(F_\gamma)$	经由 ctx.registry 枚举
$n : \mathfrak{N}$	fiber.uid
$d : \mathfrak{D}_\Gamma$	fiber.inject
$p : \mathfrak{P}_\Gamma$	组件的 provide
$e : \mathfrak{E}_\Gamma^*$	fiber.apply
$\pi : \mathfrak{N}$	fiber.parent.fiber.uid, 即拥有其实例化所用上下文的纤程
派生实现 (定义 27)	fiber.ctx, 纤程运行于其中的子上下文
θ (定义 44)	fiber.state, 生命周期状态, 其 LOADING 即 Reloading, 其 FAILED 即 Inactive(ξ)
recover、累加器 (accumulator) g	fiber.dispose, 累加器
ω (定义 44)	fiber.committed, 已提交的视图
$\text{provider}_k(\gamma)$	其提供方纤程处于 ACTIVE 的 Impl
$\text{target}(\gamma, n)$	fiber.target, 由 refresh (Algorithm 5) 重算, 其中 $\perp$ 即 INACTIVE
Future、惯性 (4.3.3 节)	fiber.inertia, 进行中转移的句柄
O-Insert、O-Retire (定义 47)	ctx.use 及其回调的逆变换 (Algorithm 4)
O-Remove	纤程从其运行时中除名, uid 清除
L-Begin、L-Iter、L-Finish	execute 的迭代循环 (Algorithm 1)
L-Divert	守卫在迭代边界处失守 (Algorithm 1), 或 reload 链入 unload
L-Leave	refresh 将纤程标记为 UNLOADING (第 10 行)
L-Unload	unload 及其惯性链 (Algorithm 5)
L-Unload 上的守卫	unload 等待已通知的依赖方 (第 25 行)
L-Raise	错误记录于纤程上, 其目标置为 $\perp$

Table 5.1: 理论—实现对应表

Algorithm 1: Effect tracking

```

1 async function function execute(callback, guard) iter ← callback();
2 inverse ← id;
3 while guard() do
4   (value, done) ← await iter.next();
5   if value then
6     | inverse ← value ◦ inverse
7   if done then
8     | break
9 return inverse;
10 function function effect(ctx, callback) armed ← true;
11 task ← execute(callback, () ↦ armed);
12 async function function dispose() if not armed then
13   | return
14 armed ← false;
15 recover ← await task;
16 recover();
17 ctx.dispose ← dispose ◦ ctx.dispose;
18 return dispose;

```

引擎 `execute` 将回调作为副作用迭代器 ($\mathfrak{E}_T^{\text{iter}}$, 定义 51) 驱动, 并把每步产出的逆变换折叠为单个复合逆变换。每步之前它查询一个由调用方提供的守卫; 一旦守卫失守, 迭代即停止, 只剩迄今累积的逆变换。这就是 4.3.2 节的步边界中断: $\text{Maybe}(\mathfrak{E}^{\text{iter}})$ 续体由迭代器的 `done` 标志与 `guard` 共同实现。

`ctx.effect` 是 `execute` 之上的薄封装, 增添了两样东西。其一, 自我处置: 守卫报告 `armed` 标志, 而返回的 `dispose` 将 `armed` 翻转为 `false`, 这同时中止任何进行中的迭代, 并使恢复至多触发一次。触发两次会在一个未曾由该副作用的任何一次应用所产生的状态上应用逆变换, 此时没有任何东西约束它去恢复什么。其二, 父级组合: `dispose` 被前插到外围上下文已累积的逆变换 `ctx.dispose` 之上, 于是子副作用的逆变换本身是父级上的一个副作用, 这正是 $\partial^2\Gamma$ 的递归结构。组件层 (5.1.3 节) 复用同一个 `execute`, 只是守卫改为检测 `fiber.target` 的稳定性而非 `armed`。

5.1.2 协副作用操作

本节实现反应式协副作用 (3.2 节)。所有协副作用操作都作用于每个上下文携带的三个符号键槽位:

- `@@store`: 值存储 $\sigma : (r : R) \rightarrow \mathcal{V}_r$, 从 realm 符号到带类型的值;
- `@@isolate`: realm 表 $\rho : \text{Map}(K, R)$, 从协副作用键到 realm 符号;
- `@@intercept`: 拦截 (interception) 表 $\iota : (k : K) \rightarrow \mathcal{M}_k$, 为每个键指派其元数据。

前两个槽位复合成两层解析 $k \rightarrow \rho(k) \rightarrow \sigma(\rho(k))$: `ctx.get(key)` (Algorithm 2) 先从 `@@isolate` 读出 realm 符号 $\rho(k)$, 再从 `@@store` 读出绑定的值 $\sigma(\rho(k))$ 。 ρ 这层间接使隔离 (isolation) 能

把一个键重定向到一个独立绑定，而 `@@intercept` 仅在绑定被访问时查询，调整的是绑定如何被使用而非它解析到什么。我们分两部分实现这些操作：(1) 提供与通知，安装或撤销绑定并把变更传播给依赖方；(2) 隔离与拦截，重塑键的解析方式。

提供与通知。由于 $set(k, v)$ 的类型是 $\mathfrak{E}_\Sigma$ (3.1 节)，副作用的提供是一次 `ctx.effect` 调用，并继承其自动追踪与恢复。Algorithm 2 实现 `ctx.set(key, value)`，即具体的 $set(k, v)$ ：回调在存储中按 realm 符号 $\rho(k)$ 绑定一个值，返回的 `dispose` 函数则将其移除。安装与移除都调用 `notify` 以将变更传播给依赖组件。

Algorithm 2: Coeffect operations

```

1 function function get(ctx, key) realm  $\leftarrow$  ctx[@@isolate][key]  $\boxtimes$   $\rho(k)$ 
2 ;
3 return ctx[@@store][realm]  $\boxtimes$   $\sigma(\rho(k))$ 
4 ;
5 function function set(ctx, key, value) function function callback() realm  $\leftarrow$ 
   ctx[@@isolate][key]  $\boxtimes$   $\rho(k)$ 
6 ;
7 ctx[@@store][realm]  $\leftarrow$  value  $\boxtimes$   $\sigma[\rho(k) \mapsto v]$ 
8 ;
9 notify(ctx, [key]);
10 return function function () delete ctx[@@store][realm]  $\boxtimes$   $\sigma \setminus \rho(k)$ 
11 ;
12 notify(ctx, [key]);
13 return ctx.effect(callback);

```

Algorithm 3 将每个绑定变更传播给依赖方：对每个存活纤程，测试是否有变更过的键出现在其 `fiber.inject` 中且解析到同一 realm；若有，则调用 `refresh` (5.1.3 节) 针对新状态重新求值该纤程，并返回它所重新求值的纤程集合，使调用方能等待它们。这正是定义 26 的反应式分类：翻转满足性的变更激活或停用纤程，而 `refresh` 的幂等性使中性变更无害。这种重新求值与多样控制流的交互在 5.1.3 节展开。

Algorithm 3: Reactive notification

```

1 function function notify(ctx, keys) affected  $\leftarrow$   $\square$ ;
2 for fiber in all_fibers do
3   for key in keys do
4     if key  $\in$  fiber.inject and fiber.ctx[@@isolate][key] = ctx[@@isolate][key] then
5       refresh(fiber);
6       affected  $\leftarrow$  affected  $\cup$  {fiber};
7       break;

```

只有当安装绑定的纤程处于 ACTIVE 时，该绑定才算对依赖方可用，因此 `refresh` 是针对活跃提供方解析每个声明键，而非仅针对存储。这就是定义 46 的 `provided by` 关系，也是使撤离 (`withdrawal`) 提前一步对依赖方可见的关键：进入 UNLOADING 的提供方已停止提供，因此其依赖方会重新计算出一个不满足的目标视图，并在其绑定尽数在位之时就开始自己的 `teardown`。

隔离与拦截。两个操作在结构上做同一件事：各自派生一个子上下文，只为 `key` 调整一张继

承来的表，父上下文不受影响，因此恢复是隐式的：丢弃子上下文即可，无须运行任何显式逆变换。ctx.isolate(key, realm) 用 realm 覆写 realm 映射 ρ ，缺省则用一个新生成的符号（实现 isolate，定义 29），于是为同一键指派不同符号的两个上下文解析到相互独立的绑定。ctx.intercept(key, metadata) 将 metadata 合并进拦截表 ι （实现 intercept，定义 31）：依照该定义，新元数据与上下文已为 key 携带的元数据合并，并优先于后者。

5.1.3 组件生命周期

组件由 ctx.use 实例化为一个纤程。本节赋予纤程（5.1 节引入）以 4.3.3 节惯性状态机的操作含义。两个字段驱动下述算法：fiber.parent，即 fiber.ctx 的父上下文，构成组件层级（ Γ_∞ 的递归结构，3.6.1 节）；fiber.inertia，进行中异步转移的句柄（空闲时为 null）。

Algorithm 4 给出组件实例化。组件将协副作用规约 component.inject (d) 与副作用函数 component.apply 配对；实例化把组件的 config 绑入 fiber.apply（第 9 行），即生命周期随后运行的、经配置应用的副作用函数 (e)。回调函数（第 2 行）是父纤程中追踪的副作用：执行时，它调用 refresh（Algorithm 5）启动子生命周期；恢复时，它强制子纤程的目标为 $\perp$ 并触发 unload。这就是定义 47 的注册原语，callback 充当其 O-Insert，callback 返回的闭包充当其 O-Retire：一次实例化是父级的一个普通被追踪副作用，因此卸载父级会级联到其子级。

Algorithm 4: Component instantiation

```

1 function function use(ctx, component, config) function function callback() refresh(fiber);
2 return function function () fiber.target  $\leftarrow \perp$ ;
3 unload(fiber);
4 fiber  $\leftarrow$  Fiber(parent: ctx, inject: component.inject);
5 fiber.ctx  $\leftarrow$  ctx[fiber  $\mapsto$  fiber];
6 fiber.apply  $\leftarrow$  ()  $\mapsto$  component.apply(fiber.ctx, config);
7 ctx.effect(callback);
```

Algorithm 5 实现 4.3.3 节的惯性状态机，其中 reload 与 unload 都是惯性的：一旦进入，转移便运行完毕，系统才响应目标状态的变更。它使用两个针对协副作用存储的辅助查询：resolve(inject) 返回声明键当前解析到的绑定，provided(fiber) 返回该纤程自己安装了绑定的键。refresh 函数从协副作用存储重算 fiber.target，若纤程尚不在转移中，则发起一个 reload 或 unload 任务¹。reload 函数记录当前目标并执行组件的副作用函数 apply。完成后它检查目标是否仍然匹配：若匹配，纤程进入 ACTIVE；若不匹配（无论新目标是 $\perp$ 还是另一组提供方），则链入 unload。对称地，unload 以 LIFO 顺序恢复所有被追踪副作用，随后要么进入 INACTIVE，要么链入 reload。这一互递归实现了惯性性质：转移一旦开始，必先完成，新的转移才能开始。

¹create_task 调度一个异步函数并发运行并返回其句柄（存于 fiber.inertia）。我们显式写出它以求语言无关：在急切调度下（如 TypeScript 的 promise），该调用是隐式的，返回的 promise 即句柄；在惰性调度下（如 Python 协程、Rust future），宿主必须 spawn 该任务它才会推进。

Algorithm 5: Component lifecycle

```

1 function function refresh(fiber) target  $\leftarrow$  target( $\gamma$ , n);
2 if target = fiber.target then
3   return
4 fiber.target  $\leftarrow$  target;
5 if fiber.inertia then
6   return
7 if target  $\neq \perp$  then
8   fiber.state  $\leftarrow$  LOADING;
9   fiber.inertia  $\leftarrow$  create_task(reload(fiber));
10 else
11   fiber.state  $\leftarrow$  UNLOADING  $\boxtimes$  out of service before any inverse is scheduled
12   ;
13   fiber.inertia  $\leftarrow$  create_task(unload(fiber));
14 async function function reload(fiber) target0  $\leftarrow$  fiber.target;
15 fiber.committed  $\leftarrow$  resolve(fiber.inject)  $\boxtimes$  commit the view
16 ;
17 recover  $\leftarrow$  await execute(fiber.apply, ()  $\mapsto$  fiber.target = target0);
18 fiber.dispose  $\leftarrow$  recover  $\circ$  fiber.dispose;
19 if fiber.target = target0 then
20   fiber.state  $\leftarrow$  ACTIVE;
21   notify(fiber.ctx, provided(fiber));
22   fiber.inertia  $\leftarrow$  null;
23 else
24   fiber.state  $\leftarrow$  UNLOADING;
25   fiber.inertia  $\leftarrow$  create_task(unload(fiber));
26 async function function unload(fiber) await all(notify(fiber.ctx,
    provided(fiber)).map(f  $\mapsto$  f.await()))  $\boxtimes$  drain dependents
27 ;
28 await fiber.dispose();
29 fiber.dispose  $\leftarrow$  id;
30 fiber.committed  $\leftarrow$   $\perp$ ;
31 if fiber.target =  $\perp$  then
32   fiber.state  $\leftarrow$  INACTIVE;
33   fiber.inertia  $\leftarrow$  null;
34 else
35   fiber.state  $\leftarrow$  LOADING;
36   fiber.inertia  $\leftarrow$  create_task(reload(fiber));

```

fiber.target 的计算方式是：针对当前协副作用存储解析每个声明键，并将提供该键的纤程的 uid 组成元组，因此它是 *target*(γ , *n*)（定义 46）的一个摘要。以提供方而非绑定值来识别绑定，使得与已记录目标的一次比较即告充分：uid 是新抽取且永不复用的，因此被替换的提供

方不会与其前任混淆，即便二者提供相等的值。由于 `notify` (5.1.2 节) 在每次副作用变更时都重算目标，纤维恰在其某个声明键改由另一纤维提供时 `reload`。就地覆写自身绑定的提供方因此不会被观测到；希望其替换得以传播的组件，应先撤销该绑定再重新安装。

该算法在两个互补层面运作。在转移层面，`reload` 与 `unload` 在完成时检查目标，实现跨转移的惯性链接。在每个转移内部的迭代层面，副作用执行 (Algorithm 1) 在每个迭代边界检查目标，实现单次转移内的部分回滚。这两个机制分别对应 4.3.3 节的转移间链接与定理 64 所依赖的转移内过期检查。

有三行代码承载着定理 63 的副作用次序，而它们各自的位置正是次序成立的原因。`reload` 在第 14 行提交解析出的视图，`unload` 则在所有逆变换运行完毕后才丢弃它，因此纤维在其加载期间——包括自身的 `teardown`——读到的始终是同一组绑定。`refresh` 在第 10 行、转移任务创建之前将纤维标记为 `UNLOADING`，这是 `L-Leave` 步骤：纤维停止提供，依赖方在任何逆变换排程之前就据此重新计算。`unload` 随后在第 25 行等待每个已通知的依赖方到达 `INACTIVE`，这是 `L-Unload` 上的守卫；`notify` 只在依赖方的声明键解析到与提供方相同的 `realm` 符号时才接纳它，这正是该守卫要求依赖方从该纤维看见此键、而非仅仅声明此键的运行时形式。这一等待位于整个恢复之前，而非位于某个被等待的逆变换之内，因为 `fiber.dispose` 并发地发起一个纤维的各项副作用，若把等待放在其中之一里，其余副作用便失去次序。终止性遵循定理 66：纤维只等待已不再可满足的依赖方，而本身是提供方的依赖方又以同样方式等待它自己的依赖方，因此提供方图是按需遍历的，而非预先分析。

5.1.4 上下文访问

5.1.2 节的副作用操作构成一套反射式 API：副作用以 `ctx.set(key, value)` 写入、以 `ctx.get(key)` 读取，二者均以名称为键。Cordis 在此反射式 API 之上叠加了第二种更原生的扩展与消费上下文的方式：属性访问。组件可以像访问上下文的原生结构那样以 `ctx[key]` 访问一个副作用，而无须经过方法调用。在 TypeScript 中，Cordis 用一个 Proxy 实现这一点，其 `get` 陷阱中介每次属性访问。Algorithm 6 展示上下文如何在 5.1.2 节原语 `get` 之上将此类访问解析到副作用。

Algorithm 6: Proxy-mediated context access

```

1 function function resolve(ctx, key) fiber ← ctx.fiber;
2 repeat if key ∈ fiber.committed then
3   | return fiber.committed[key]
4 if key ∈ fiber.inject then
5   | throw INACTIVE_ACCESS
6 if fiber~root then
7   | throw UNDECLARED_ACCESS
8 fiber ← fiber.parent.fiber;
9 until false;

```

Algorithm 6 从发起访问的上下文沿纤维链上行：到达第一个已提交视图绑定了 `key` 的纤维时，访问获得授权并返回该绑定；若走到一个声明了 `key` 却尚未提交它的纤维，说明该纤维未加载，访问失败；若到达根仍未见到任何声明，访问以未声明为由被拒绝。这正是 proxy 与裸 `ctx.get` 的分歧所在：`ctx.get(key)` 是针对存储的查找，返回绑定的值或空、从不失败；而 proxy 针对发起访问的纤维自身的视图解析，并在使用点上强制执行副作用规约 d 。读视图而非读存储也是定理 63 的立足点，因为它使一个依赖项对“因其消失而触发 `teardown`”的组件仍然可读。

这种拒绝是在访问点执行的运行时检查。由于组件的副作用规约 d 是静态声明的，同一违规原则上可在编译期检出——在执行前针对声明的 d 解析每个 $\text{ctx}[\text{key}]$ ；6.4 节讨论宿主语言的类型级依赖声明与编译期元编程如何恰好完成这种中介。

5.2 组件加载器

核心库为组件开发者提供了命令式的动态组合原语，如 ctx.effect 、 ctx.use 与 ctx.set 。应用编排者面临另一个问题：他们把既有组件装配成运行中的系统，并在系统生命周期内调整这一组合。组件加载器通过引入一个声明式配置（declarative configuration）层来解决该问题：编排者将期望的组合描述为一个持久数据结构，加载器把对该规约的变更翻译为相应的命令式线程操作。

5.2.1 声明式配置

第 4 章把运行中的系统分解为线程，每个线程是一个组件的一次实例化。实例化所需的一切皆可声明，因此编排者可以把整个系统描述为一份声明式配置：一条持久记录，加载器将其实现为线程，并使之与线程保持同步。

条目。配置由条目（entry）组成。每个条目指定一个线程并管理它，且绑定是双向的：加载器响应条目字段的变化以调整线程，而组件修改自身配置或停用自身时，变更也会写回其条目。

定义 5.1 (条目). 一个条目声明单个线程，记录：

- *id* ——稳定标识符，在其分组的子列表变更时用作调和键；
- *url* ——待实例化的组件模块的 URL；
- *isolate* ——施加于该条目上下文的隔离注记；
- *intercept* ——施加于该条目上下文的拦截注记；
- *config* ——绑入组件以构成其副作用函数 *apply* 的配置；
- *disabled* ——该条目是否被管理性地关闭。

条目之所以能充当忠实的规约，是因为支撑线程的恰是条目所记录的。定义 67 的支撑集读取 τ 、 π 、 d 与 p ，别无其他，而条目给出了全部四项：*disabled* 给出 τ ，条目在树中的父节点给出 π ，*url* 选出声明 d 与 p 的组件。支撑集未读取的字段是线程的运行时状态，实例化同样不需要它们；引理 70 在每个组件安装其声明的每个键（定义 69）的前提下，把支撑集与静息状态（定义 49）的 Active 线程等同起来。

这些条目构成一棵配置树，它是系统加载内容的权威记录。条目可以是映射到单个线程的叶子，其组件也可以进而加载更多组件，使该条目成为分支节点。Cordis 为此类分组与嵌套加载提供组件：`@cordisjs/group` 将子条目列表作为其配置并作为子组加载，`@cordisjs/include` 加载外部配置文件（YAML 或 JSON）并将其条目嫁接为一棵嵌套子树。二者都是建立在定义 47 注册原语（Algorithm 4）之上的普通组件，因此嵌套树仍处于演算之内，下文的结果对它同样成立。

调和。当条目的记录变化时，加载器增量调和，而非将线程推倒重建。如此调和之所以可靠，理由由元理论（metatheory）提供。

- 定理 73 使静息状态仅是最终配置的函数：无论加载器沿途执行了哪些实例化与退役、以何种顺序，系统静息之处恰是从零加载最终配置所会到达的状态。哪些组件最终被加载仅由声明读出，前提是每个组件安装其声明的每个键（定义 69）；声明了某键却只在某些配置下安装它的组件，加载器仍可调和，但已加载组件的集合便也取决于这些配置。
- 定理 66 证明系统确实会静息，因此一旦调和的实例化与退役均已发出，调和即告完成。
- 推论 62 将离场线程对状态的贡献计为零，因此重建一个条目会撤销其线程安装的内容，而让周围线程保持原状。
- 定理 63 允许条目被一并实例化，编排者无须安排加载顺序：声明键尚未被提供的线程在其 L-Begin 处等待，提供方离场的线程则提前被停用。因此依赖约束的是线程何时激活，而非其模块何时被获取与求值，于是加载器并发加载模块——这正是大型配置启动耗时之所在。

在条目声明的线程之上，加载器按条目的哪些字段发生变化进行分派，并对每种情况施加破坏性最小的操作。

- id、url ——重建该条目，因为其身份或其组件已变化；
- isolate ——重新指派该条目的 realm（Algorithm 7）；
- intercept ——就地更新，因为拦截元数据在读取时查询，无须 reload；
- config ——交给组件，由组件决定如何应用新载荷，典型做法是与前一份配置做差分，仅在实质性变更时 reload。特别地，@cordisjs/group 条目的 config 就是其子条目列表，因此它把该更新实现为按子条目 id 的键控差分，对每个子条目创建、删除或更新；由于更新一个存活的子条目会重新进入这同一套逐字段分派，分组调和与条目更新便沿树递归下行；
- disabled ——置位时卸载线程，清除时重新加载。

受管 realm。核心中的隔离派生一个在单个键上覆写 realm 表 ρ 的子上下文（5.1.2 节），这在上下文树静止不动时已足够。条目可能在运行时于分组间移动，因此加载器自行管理 realm，isolate 字段按键在两种作用域规则之间选择。值为 true 时请求一个局部 realm，为该条目私有、以其 id 标记，随条目移动而随身携带；值为字符串时请求一个全局 realm，由所有以该字符串命名的条目共享，因此移动此类条目改变的是它与哪些条目共享绑定，而非它属于哪个 realm。一旦没有条目再命名某个 realm，它即被丢弃。

重新指派条目的 realm 取决于哪些键更换了 realm、该条目自身是否是变更键上的提供方，以及要通知哪些依赖方。中间这个问题是难点，因为一个 realm 符号可能被多个线程共享，其中只有一个是提供方。加载器用分隔符（delimiter）回答它：每键一个符号 δ_k ，每个上下文在其下存储自己的一个标签。分隔符写在上下文上并被其后代继承，因此条目的标签与提供方的标签恰好在该二者于同一个针对 k 的 isolate 作用域内派生时一致——这正是 k 处的绑定属于条目自身、须随之移动的情形。

该测试取决于分隔符的一条性质。 δ_k 之下的标签写在条目的上下文上，并由从它派生的每个上下文继承，且在每次重新指派时新抽取，因此对上下文 γ' 而言

$$\gamma'[\delta_k] = d_1 \iff \gamma' \text{ 从条目的上下文派生} \quad (5.1)$$

记该条件为 $\text{own}(\gamma')$, $d_2 = d_1$ 是它在提供方处的实例。重新指派把满足 own 的上下文从 s_1 移到 s_2 , 其余保持原位; 而依上面的循环, 恰当提供方满足 own 时绑定被移到 s_2 。依赖方在其 k 处的 realm 是绑定所在 realm 期间看见该绑定。在依赖方与提供方上 own 一致处, 二者同移或同留, 因此依赖方事后看见绑定恰如其事前看见; 在 own 分离二者处, 一方移动另一方留下, 依赖方因此获得或失去该绑定。那个不等式正是此分离, 而成员测试剔除了在两个 realm 中均不解析 k 的依赖方——移动的任何部分都触不到它们。

Algorithm 7: Isolation realm reassignment

```

1 function function patch_isolation(entry,  $\rho'$ )  $\rho \leftarrow \text{entry.ctx}[\text{@@isolate}]$ ;
2 store  $\leftarrow \text{entry.ctx}[\text{@@store}]$ ;
3  $\Delta \leftarrow \{k \mid \rho(k) \neq \rho'(k)\}$   $\triangleright$  keys whose realm changes
4 ;
5 for  $k$  in  $\Delta$  do
6   entry.ctx[ $\delta_k$ ]  $\leftarrow$  fresh tag;
7   diff[ $k$ ]  $\leftarrow (\rho(k), \rho'(k), \text{entry.ctx}[\delta_k], \text{store}[\rho(k)].\text{fiber.ctx}[\delta_k])$ ;
8 entry.ctx[ $\text{@@isolate}$ ]  $\leftarrow \rho'$ ;
9 reload(entry.fiber);
10 for  $k$  in  $\Delta$  do
11    $(s_1, s_2, d_1, d_2) \leftarrow \text{diff}[k]$ ;
12   if  $d_1 = d_2$  and  $\text{store}[s_1]$  and not  $\text{store}[s_2]$  then
13      $\triangleright$  the binding is the entry's own
14     store[ $s_2$ ]  $\leftarrow \text{store}[s_1]$ ;
15     delete store[ $s_1$ ];
15 function function affected(fiber,  $k$ )  $(s_1, s_2, d_1, d_2) \leftarrow \text{diff}[k]$ ;
16 return fiber.ctx[ $\text{@@isolate}$ ][ $k$ ]  $\in \{s_1, s_2\}$  and (fiber.ctx[ $\delta_k$ ] =  $d_1$ )  $\neq$  ( $d_2 = d_1$ );
17 notify(entry.ctx,  $\Delta$ , affected)  $\triangleright$  in place of the realm test of Algorithm 3
18 ;
```

5.2.2 热模块替换

热模块替换 (HMR) 把可逆副作用模式应用到模块层: 当源文件变化时 (典型地发生在开发期间), 系统就地替换受影响的模块而无须重启进程。由于线程已经界定了组件的全部副作用与协副作用, 本身就是组件的模块可以仅凭线程操作完成替换: 处置旧线程即恢复该组件安装的一切, 而从重载后的模块实例化的新线程再将其重新安装。因此 HMR 不需要开发者标注的接受边界, 这与 Webpack [46] 或 Vite [47] 的 HMR 形成对照。

@cordisjs/hmr 组件提供 HMR 引擎, 它分三个阶段运作。

阶段 1: 模块分类。引擎接受两个输入: stashed 集 (自上次 reload 以来内容已变化的文件 URL) 与 externals 集 (无法热替换、转而触发完全重启的模块)。记 $\text{get_imports}(\text{url})$ 为 url 直接导入的模块, 它对变更的依赖子图做分类, 将每个模块标记为接受或拒绝:

以 stashed 文件的导入为种子, 该不动点在模块的某个导入被接受时接受它, 在其全部导入被拒绝时拒绝它; 任何悬而未决、被困于导入环中的模块默认被拒绝。

Algorithm 8: Module classification

```

1 function function classify(stashed, externals) accepted  $\leftarrow$  stashed;
2 declined  $\leftarrow$  externals;
3 pending  $\leftarrow$   $\square$ ;
4 for url in stashed do
5   pending  $\leftarrow$  pending  $\cup$  (get_imports(url)  $\setminus$  (accepted  $\cup$  declined));
6 repeat
7   progress  $\leftarrow$  false;
8   for url in pending do
9     if get_imports(url)  $\cap$  accepted  $\neq \square$  then
10       accepted  $\leftarrow$  accepted  $\cup$  {url};
11       pending  $\leftarrow$  pending  $\setminus$  {url};
12       progress  $\leftarrow$  true;
13     else
14       if get_imports(url)  $\subseteq$  declined then
15         declined  $\leftarrow$  declined  $\cup$  {url};
16         pending  $\leftarrow$  pending  $\setminus$  {url};
17         progress  $\leftarrow$  true;
18       else
19         pending  $\leftarrow$  pending  $\cup$  (get_imports(url)  $\setminus$  (accepted  $\cup$  declined));
20 until not progress;
21 declined  $\leftarrow$  declined  $\cup$  pending;
22 return (accepted, declined);

```

阶段 2：过期条目检测。继而，引擎利用 *accepted* 与 *declined* 把组件条目过滤到过期的那些——其依赖树触及变更模块的条目。它用 *get_dependencies* 走每个条目的树，该函数在把 *declined* 当作边界的同时收集一个模块的传递导入：

条目过期恰当其树与 *accepted* 相交；该树随即并入 *accepted*，因此沿它的每个过期模块都在下一阶段被失效。

Algorithm 9: Stale-entry detection

```

1 function function get_dependencies(root, declined) deps ←  $\square$ ;
2 function function traverse(url) if url ∈ deps or url ∈ declined then
3   | return
4 deps ← deps ∪ {url};
5 for child in get_imports(url) do
6   | traverse(child)
7 traverse(root);
8 return deps;
9 function function detect(entries, accepted, declined) stale_entries ←  $\square$ ;
10 for entry in entries do
11   | tree ← get_dependencies(entry.url, declined);
12   | if tree ∩ accepted ≠  $\square$  then
13     | accepted ← accepted ∪ tree;
14     | stale_entries ← stale_entries ∪ {entry};

```

阶段 3：事务性 reload。最后，引擎重载过期条目。它使 *accepted* 模块的缓存失效²，备份每个被移除的模块以支持回滚，然后按 *url* 重新导入每个过期条目的组件模块，并换入一个新纤程：

Algorithm 10: Transactional module reload

```

1 function function reload(ctx, accepted, stale_entries) backup ← invalidate_caches(accepted);
2 try { for entry in stale_entries do
3   | entry.fiber.dispose();
4   | entry.fiber ← ctx.use(import(entry.url), entry.config);
5 } catch error { restore_caches(backup);
6 for entry in stale_entries do
7   | entry.fiber.dispose();
8   | entry.fiber ← ctx.use(backup[entry.url], entry.config);
9 throw error;
10 }

```

事务性保证确保系统绝不会进入半重载状态：若任何模块导入失败（例如因语法错误），缓存被恢复，每个过期条目都从 *backup*[*entry.url*]——缓存刚被恢复的先前组件——重建，从而撤销已进行的换入。

²在 Node.js 上，这意味着同时清除 ES 模块与 CommonJS 模块两套系统的缓存，因为经 ES loader 导入的模块可能同时出现在两者之中。

5.3 案例研究：Koishi

Koishi 是构建于 Cordis 之上的开源聊天机器人应用框架³。历经四年开发，它累积了 4000 余个社区贡献的插件（plugin）⁴，从即时通讯（IM）适配器（adapter）、数据库驱动到管理控制台与终端用户功能。其规模与多样性使它成为 Cordis 动态可组合性在生产环境中的一次有代表性验证。

元框架的表达力与通用性。Koishi 作为一个服务器端机器人运行，其每项功能都是 5.1 节上下文原语之上的插件；Koishi 自身只贡献聊天机器人领域的词汇表。同一模型再现于迥异的运行时：Koishi 的 Web 控制台是第二个独立的 Cordis 应用，其插件组合的是浏览器及其用户界面的原语，而非服务器的。上述迥异场景确立了第 3 章模型的两条性质。(1) 它有表达力：其原语足以承载一个完整的生产系统，宿主框架只提供领域词汇。(2) 它有通用性：它固定副作用与协副作用如何组合，而把其含义留给各应用，因此既不预设特定领域，也不预设特定运行时。

无认知负担的时间性可组合。1.2.1 节考察的插件系统无法在不重启扩展宿主的情况下卸载单个扩展的副作用。Koishi 日常执行这一操作：编排者从控制台禁用一个插件，其副作用即就地撤离；开发期间，HMR 引擎在保存时重新应用已编辑的插件，同时保留缓存状态与系统其余部分的活跃连接。Cordis 使这种移除不仅可能，而且对插件作者毫不费力。由于经上下文执行的副作用被追踪、其逆变换被自动组合（3.1 节），即使缺乏经验的作者也为插件的经上下文中介的副作用获得有序清理，而无需编写任何卸载路径。这实现了 1.2.1 节指出其缺失的关注点局部性（locality of concern）：本会依赖于每位作者勤勉的正确性，如今由抽象一次性解除。

开放生态中的空间性可组合。与 1.2.1 节插件系统中插件间依赖大体缺席的情形相反，Koishi 的生态展现出真正的依赖拓扑：IM 适配器提供对各消息平台的访问，数据库驱动提供持久存储，功能型插件将这些声明为协副作用并访问它们。在运行时重新配置提供方（例如切换存储后端或重连适配器）只重新激活解析依赖发生变化的依赖方（3.2 节）；依赖不可用的插件保持非活跃直至其出现，而不报错。本案例研究证实的，是这种组合在独立编写的代码之间依然成立：插件及其依赖通常由不同作者编写，他们除连接彼此的协副作用之外别无协调，因此反应式协副作用使用由独立贡献者构成的开放生态中的装配保持一致。

效度威胁。此处的证据取自单一宿主语言中的单一生态，因此无法把该范式的优点与其 TypeScript 实现或 Koishi 特定领域的优点分离开来；且它是观察性的，而非与另一架构的受控比较。因此本案例研究确立的是存在性与采用性结果，而非定量结果；相对基线测量该抽象的开销及其对开发者生产力的影响，仍是未来工作。

³Koishi 目前使用 Cordis v3。本文呈现的是 Cordis v4，它精化了副作用与协副作用语义并重新设计了加载器；两个版本共享同一核心组合模型。

⁴Koishi 所称的 plugin，即本文形式化为组件（component）的概念。

第 6 章 讨论

前述各节给出的形式模型与实现引入了一种面向动态可组合性的编程范式。本节考察该范式如何延伸到更广泛的工程议题，并讨论其中的设计张力与悬而未决的问题。

6.1 系统边界

第 3.1 节中的每个副作用 (effect) 都携带一个逆变换，而这个逆变换究竟意味着什么，是由系统边界 (system boundary) 裁定的。边界把系统运行于其中的环境划分为两部分。(1) 若系统能够独占地修改某个位置、并能恢复该修改之前的状态，则该位置位于边界之内，于是对它的操作被记录在 Γ 中，并且此后可以被恢复。(2) 若这两种能力有任一不成立，则该位置位于边界之外，于是对它的操作表现为 id_T ，既不被记录也不被恢复。本节展开这一边界的性质及其对恢复的影响。

源于副作用的边界。副作用 (coeffect) 通过将一个外部位置具体化来移动边界：它把对该位置的每一次访问都限制在它提供的一组操作之内，其中每个操作它都能给出逆变换，于是原本表现为 id_T 的操作得以被记录在 Γ 中并被恢复。因此，边界是按位置而非按介质划定的，因为前述两种能力都是位置的属性，而具体化改变的是一个位置被访问的方式，其介质则保持原样。例如，一块内存区域在只有系统写入它时位于边界之内，在其他进程也写入它时位于边界之外；一个文件在只有系统能够触及它时（如私有路径下的临时文件）位于边界之内，在它是其他程序读写的路径时位于边界之外。移动边界本身是一种权衡：一边是环境是否为某个位置提供可逆语义，另一边是在每次访问上供给这些语义的代价。我们在第 6.7 节讨论由此引出的协同设计。

获取与发射。一个触及边界之外的操作通常分两个阶段进行。(1) 在获取阶段，操作取得访问权，并在边界之内安装一条记录：open 安装一个由 close 移除的描述符，malloc 保留一个由 free 释放的内存块，fork 启动一个由 kill 终止的子进程。这条记录本身是将该位置具体化的副作用的一部分，例如它所维护的一个映射中的表项，而安装该表项是一个可逆副作用 (reversible effect)。这条记录同时也是数据得以外流的通道。(2) 在发射阶段，操作把数据推入该通道，如 write 交给文件的字节或 send 放上线路的数据报，而这一推送表现为 id_T ，把数据留在了其他各方皆可读写之处。因此，两个阶段落在边界的两侧：获取留在边界之内，而发射越出到边界之外。

扣留与补偿。一个仍须从发射中恢复的系统有两条途径可用。其一是扣留发射，直至产生它的状态确定会被持久化，这就是回滚-恢复 (rollback-recovery) 的输出提交问题 [48]。其二是补偿 (compensation) [49]：一个把状态恢复到应用所提供的某个等价关系之下的动作，它比定义 33 的 $\simeq$ 更粗，例如删除一个已创建的文件，或为一笔已发生的扣款退款。这类动作与逆变换一样按相同的 LIFO 顺序复合，因此第 3.1 节的复合可以直接迁移到它们之上。元理论 (metatheory) 则不能：定义 60 的交换是针对 $\simeq$ 证明的，针对更粗的等价关系必须重新建立。

6.2 服务多路复用

诸如 OSGi [50] 这样的动态组件平台围绕服务 (service) 组织组合：提供方 (provider) 在一个接口之下发布功能单元，消费者 (consumer) 绑定到它。Cordis 的副作用模型呼应了这一概念，一个服务对应于一个键背后的接口。提供某个服务的组件是该服务的提供方，注入某个服务的

组件是其消费者。单个服务可以由多个提供方实现，而这种多重性可以以两种形式实现。(1) 独占绑定：多个实现共享一个接口，但同一时刻至多绑定一个；编排器选择绑定哪个实现，而在实现之间切换须卸载一个提供方再加载另一个，并瞬时扰动每个消费者的依赖。(2) 服务代理 (service broker)：一个作为该接口入口的中心服务，由各后端提供方与消费者共同注入，于是多个提供方并存，由代理在其间分派每个请求。与独占绑定相比，代理吸收了这种扰动：更新一个后端提供方时代理保持原位，因此消费者看不到其依赖的任何变化，也不会触发任何 reload。

服务代理支撑三项能力：负载均衡 (load balancing)、滚动更新 (rolling update) 与跨进程调用 (cross-process invocation)。

负载均衡。当多个提供方并存时，代理依据可配置的策略（如轮询、最小负载、延迟加权）或消费者指定的显式目标，在其间分派请求。由于提供方是普通组件，可以增删以扩缩容量；每个提供方通过一个可逆副作用向代理注册，因此卸载它会撤销该注册，并自动将其从代理的路由集合中移除。

滚动更新。在运行时升级一个服务实现，归结为一个受控的提供方迁移 [51, 52]。为执行迁移，新提供方作为附加的纤程 (fiber) 被加载并向代理注册；一旦它变为 ACTIVE，流量便逐渐从旧提供方转移到新提供方（例如通过调整选择权重），旧提供方在不再承载在途请求后被卸载。这一提供方迁移把传统上属于基础设施层面的操作（如容器编排、蓝绿部署）转变为应用层面的组合模式。

跨进程调用。服务代理还可以应用到进程边界之外 [53]。每个进程托管自己的 Cordis 上下文 (context) 及本地提供方；一个协调组件把它们连接起来，将每一个视为远程提供方。跨进程的服务访问由一个保持接口的 RPC 机制中介，使分布对消费者透明。须注意，跨进程调用会引入延迟且可能在途中失败，因此若将其同步暴露，会阻塞调用方。故打算跨进程暴露的接口必须按异步契约来设计。

6.3 访问控制与沙箱化

对于由独立组件装配而成的应用，保障其安全需要两种互补的机制：(1) 约束组件可以访问哪些依赖；(2) 将不可信代码与宿主环境沙箱化 (sandboxing) 隔离 (isolation)。Cordis 通过依赖声明与拦截 (interception) 支持前者；后者则需要外部沙箱。

基于能力的访问控制。依赖访问机制（第 5.1.4 节）已经构成一种针对经代理中介之属性的访问控制：组件只能访问其声明过的依赖，未声明的访问会引发错误。这在结构上类似于基于能力的安全 (capability-based security) [54, 55, 56]：权威由持有引用授予，而非来自环境权威。inject 声明充当能力请求，上下文代理充当能力中介。由于这些请求是静态声明的，组件所需的经代理中介的能力全集在它运行之前即为已知，使编排器能够在加载时审查并批准它们，而不是在访问发生时才被发现。

这一中介通过拦截机制推广为细粒度策略。访问控制元数据可以由上下文携带或由组件声明（定义 30），提供方在依赖被调用时查阅它，以决定一个请求是否被允许。例如，一个文件系统依赖可以携带声明组件可读写哪些路径的元数据，提供方对照该元数据检查每次调用。由于这一拦截位于上下文而非任一方的代码之中，编排器可以在不修改提供方的情况下调整它，以约束任一组件对某个依赖的访问，例如，授予社区组件只读的数据库访问，而核心组件保留完全访问。此外，由于拦截只影响依赖如何被调用，而不影响它是否被满足，它可以在运行时被安装、重配置或移除，而不触发任何 reload 或扰动依赖图。

不可信组件的沙箱化。当组件的代码不可信时，语言层面的访问控制并不足够：一个能访

问宿主运行时的恶意组件可以直接触及底层对象，使这类检查形同虚设。沙箱化需要语言层面手段触及之外的执行边界，例如软件故障隔离 [57]、独立的语言运行时、沙箱进程或虚拟化容器 [58]。无论机制为何，不可信组件都运行在自己的沙箱上下文中，经由一座桥触及宿主提供的依赖，这是第 6.2 节跨进程调用的推广：同一透明性论证使这种经桥接的访问与本地注入无从区分。在宿主一侧，桥是一个普通线程，其能力可被上述访问控制削弱。

6.4 语言无关性与语言选择

尽管 Cordis 以 TypeScript 实现，上下文范式却是语言无关的：时空可组合性（spatiotemporal composability）仅由其两个可组合性维度定义，因而可以在任何沿这两个维度满足若干要求的语言中实现。我们依次沿每个维度分析这些要求。

时间性可组合。在最基本的层面上，时间性可组合（temporal composability）要求闭包：一个可逆副作用把一个动作与一个逆变换配对，而该逆变换必须连同它所恢复的状态一起被捕获为一个值，以便在 teardown 时重放。除此之外，组件的代码及加载它的副作用必须能在运行时被引入和撤回。

一门语言如何满足第二个要求，取决于其执行模型。在托管运行时中，这采取可编程模块注册表的形式，已加载模块可以从注册表中被驱逐、并在失去引用后被垃圾回收；例如 Node.js 便暴露了这样一个注册表。¹本地代码没有模块注册表，因此引入与撤回采取显式动态链接与卸链的形式（如 Unix 上的 dlopen/dlclose、Windows 上的 LoadLibrary/FreeLibrary）[59]，亦即把目标代码载入运行中的进程、此后再将其分离。WebAssembly 依其嵌入者取两条路径之一：在托管嵌入者（如 JavaScript 宿主）之下，模块实例由宿主的回收器回收；在本地嵌入者（如 Wasmtime）之下，则在其被丢弃时释放。纵贯这些机制，可逆副作用模型把加载视为对上下文的一个副作用，其逆变换撤销该模块所引入的符号、类型或处理器的注册。

空间性可组合。空间性可组合（spatial composability）要求一种机制，让组件声明其依赖，并由运行时提供和注入这些依赖。这归结为依赖注入（dependency injection, DI）问题 [38]，它在两个层面上表现出来，且不同语言各不相同：依赖如何被赋予类型，以及它们的访问如何被中介。

在类型层面，语言应让开发者表达良类型的依赖访问。消费者通过从上下文读取其键来获得协副作用，因此上下文类型（第 3.2.1 节）必须记录每个键的协副作用。类型类（Haskell 的 typeclass）[60] 与 trait（Rust）[61] 通过让提供方从自己的模块经 instance 或 impl 扩展上下文类型来实现这一点 [62]。TypeScript 的模块扩充（module augmentation）[63] 同样让提供方模块把声明合并进上下文类型。

在运行时层面，依赖访问必须被动态中介：键背后的协副作用会随提供方的加载与卸载而变化，且在不同上下文中的解析可能不同。因此，语言需要一种透明地介入访问、而消费者代码保持不变的手段，例如经 JavaScript 的 Proxy 对象 [64] 或 Python 的描述符协议（__get__）[65]。若无这样的原语，运行时反射 [66, 67] 可以动态地中介访问，代价是类型安全与开发者体验。

纵贯这两个层面，元编程设施同时供给类型与中介。注解 [68] 与装饰器把元数据附加到声明上，由处理器展开为中介访问的访问器；编译期元编程（如 Rust 过程宏、Scala 宏 [69]、Zig comptime）为每个依赖生成一个带类型的声明及这样一个访问器，从而免除通用的拦截原语。

¹CommonJS 经 require.cache 暴露模块缓存；ES 模块没有公开的驱逐 API，不过仍可通过引擎内部接口管理模块。

6.5 互依赖与组件粒度

在反应式协副作用模型中，依赖环只是让涉事组件永久不活跃：给定两个组件 A 与 B ，若 A 需要一个由 B 提供的键，而 B 需要一个由 A 提供的键，则二者的满足谓词永远无法为真。与并发系统中取决于调度、必须在其发生之时才能检测的死锁不同，这一条件仅凭依赖声明即可预测，因此运行时可以在组件加载时报告它。

实践中，大多数表面上的互依赖都可以分解为消除环的更细粒度组件。考虑两个组件：服务器（提供网络接口）与访问控制器（执行授权策略）。二者双向交互：访问控制器中介到达服务器的请求，服务器暴露一个修改访问控制策略的端点。单体式设计会让每个组件依赖另一个。然而，两个交互方向在逻辑上是独立的关注点。分解它们得到四个组件：server-core、access-control-core、request-mediation（依赖两个核心，以便对到来的请求施加访问控制）与 policy-management（依赖两个核心，以便经服务器暴露策略修改）。经此途径，环被消除，因为两个核心互不依赖；只有整合组件依赖两者。

这一分解原则上总是可行的，因为每个双向交互都可以析因成独立的单向绑定，但它增加了组件数量：一般情形下，给定 n 个相互交互的组件，整合组件的数量可以随 n 二次增长，因为每对交互组件在每个交互方向上可能都需要一个单独的组件。这不影响正确性或运行时性能（组件是轻量的），且更细的粒度可能有益：用户获得只加载其所需整合绑定的能力，实际提高了系统的可组合性。但它可能影响开发者体验：更多组件需要更多配置、更多命名，以及理解依赖图时更多的认知开销。

缓解这一粒度成本是工程问题而非理论问题。实用策略包括包捆绑（即把相关的细粒度组件分组为单个可安装单元）、基于约定的装配（即自动连接名称或类型匹配某一模式的组件），以及脚手架工具（即从声明式规约生成样板整合组件）。这些策略保持了无环模型的形式保证，同时把编写负担降低到接近单体式情形的水平。

6.6 依赖类型与版本化

在形式模型中，依赖链接纯凭键同一性建立：提供键 k 的组件满足任何在其依赖集中声明 k 的组件。类型族 ν_k 保证单个编译单元内的类型层面一致，但当组件被独立开发与构建时——组件生态中的常见情形——这一保证便失效。这一失效引出两个不同的问题。

接口漂移。提供方可能在版本之间修改与 k 相关联的接口（增加字段、更改方法签名、改动行为契约），而针对较早接口编译的消费者继续声明同一个键 k 。依赖在协副作用层面被满足（ $k \in \text{dom}(\sigma)$ ），但运行时值不再符合消费者的期望，导致类型错误、方法未找到的失败，或无声的行为分歧 [70]。

键冲突。两个独立开发的提供方可能用同一个键名 k 指代全然无关的接口。由于仅凭键同一性建立链接，期待某一提供方接口的消费者会不加任何兼容性检查地接受另一方的值。与提供方和消费者至少共享共同来源的接口漂移不同，键冲突中期望类型与实际类型之间不存在任何关系，使由此引发的失败不可预测且难以诊断。

两个问题指向同一个缺口：协副作用模型只提供名义链接（按键名），而不提供带版本的或结构性的链接（按接口兼容性）[71]。我们讨论填补这一缺口的三种途径，从与基础设施耦合最深到最语言无关。

键命名空间化。把键空间从 K 扩展到 $K \times P$ ，其中 P 标识定义接口的包，便从构造上消除了键冲突：独立开发的同本地名接口占据不同的键。这是最直接的解决方案，也是耦合最深的：它把包命名空间嵌入形式模型本身，使系统依赖外部包注册表来确立键同一性。

同侪依赖。更轻的耦合是通过宿主语言的包管理器声明版本约束 [72]。这是 Cordis 目前采用的做法。组件依赖在语义上是同侪依赖 (peer dependency)：组件不在内部捆绑其依赖，而是期望运行时上下文来供给它们。支持同侪依赖的包管理器（如 npm）可以强制版本兼容：若提供某个键的包的版本落在消费者声明的同侪范围之外，不兼容性在安装时即被捕获，而不是以运行时失败的形式浮现。但该途径有两个局限：(1) 它依赖提供方忠实遵守语义化版本控制，而这是无法强制执行的约定；(2) 包管理器通常把每个依赖解析到单一版本，这阻碍了在同一应用内加载同一包的多个版本。

结构兼容性。完全语言无关的做法，是把隶属检查 $k \in \text{dom}(\sigma)$ 替换为一个兼容性谓词，验证提供方的实际接口在结构上蕴含消费者的期望。这类似于结构子类型化 (structural subtyping) [73]：若提供的接口是所需接口的子类型，则提供方满足消费者。挑战在于语言无关地定义这一谓词：对记录类型，结构兼容性是直接的（宽度子类型化），但对行为契约（如前/后条件 [74]、效应规约 [22]）变得复杂，而一旦参数多态引入有界量化，便不可判定 [75]。

这三种途径处理问题的不同方面。设计一个统一这些途径、同时保持协副作用模型之动态组合保证的依赖模型，仍是悬而未决的问题。

6.7 与语言和操作系统的协同设计

第 6.4 节确定了宿主语言为上下文范式必须供给的最小集合。本节讨论反问题：与该范式协同设计的语言或操作系统，能在这一最小集合之外提供什么。

与语言协同设计。围绕上下文范式设计的语言可以在两方面优于一个库：它赋予上下文的语义，以及它赋予副作用与协副作用的原语。

这样一门语言可以在保持第 3.6 节上下文语义的同时，让上下文重新隐式化。命令式语言本就让每条语句运行于一个隐式上下文，而那个单一上下文既不记录副作用，也不解析协副作用。上下文范式转而区分多个上下文，其中一个操作或者修改它运行所针对的上下文，或者从它派生出另一个上下文（定义 27）。原地实现修改环境上下文，正如命令式语言所为；派生实现则引入一个单独的上下文，语言必须为其提供一个构造。让上下文隐式化带来人机工程与安全两方面的收益。(1) 在库实现中，每个涉及副作用或协副作用的函数都把上下文当作普通参数或接受者，如第 5.1 节所示。在语言隐式供给上下文之处，函数不再需要接受它。(2) 每个上下文携带自己的生命周期状态与已提交视图（第 4.1 节）。库实现把上下文当作普通变量传递，因此组件可能经闭包或全局变量误触另一个组件的上下文。它在那里安装的副作用便泄漏出自己的生命周期，它在那里读取的协副作用也逃出其依赖规约。让上下文隐式化同时封堵这两者。

这样一门语言还可以让副作用与协副作用为其编译器所知。(1) 对副作用而言，效应迭代器（定义 51）每一步都分配一个闭包，来持有逆变换及其恢复的状态。有了执行副作用的语法，编译器可以为整个迭代生成单个状态机，并把那些逆变换保存在它的栈帧中。(2) 对协副作用而言，协副作用规约可以纳入类型系统，有两个收益。第一，依赖环在编译期即被报告，而不是留给运行时（第 6.5 节）。第二，依赖可以按其类型的结构而非仅按键同一性进行比较，如行类型 (row type) 所为 [28]，这是第 6.6 节结构兼容性的类型层面支持。

与操作系统协同设计。第 1.2.3 节观察到动态可组合性的一种粗粒度替代：操作系统以进程粒度供给时间性可组合，其上的容器编排器以服务粒度供给空间性可组合。与范式协同设计的操作系统将支持细粒度组合：让组件声明的协副作用规约成为它所能触及的全部，并把自身的资源作为协副作用提供。

这样的操作系统能供给第 6.3 节留给语言之外机制的沙箱。它通过把组件限定在其声明的

依赖之内来做到这一点：在组件加载时供给这些依赖，并使其他任何东西从组件内部都不可触及，正如 WebAssembly 模块在实例化时从其嵌入者接收其导入 [76]。它还可以把第 3.2.3 节的副作用隔离与拦截作为自身的能力提供，为每个组件不同地绑定键，并中介它所供给的访问。

这样的操作系统还可以把自身的资源作为副作用提供。位于边界之外的资源，在运行时把每次获取记录在获取它的组件名下之处被变为可逆（第 6.1 节），而每个运行时都保有一份记录。把资源作为副作用提供的操作系统只保留一份记录，因为它是交出资源的一方，能把它归因于请求它的组件。内存与文件描述符是最直接的候选者，且为恢复之目的跟踪它们，已在内核接口上实现过 [77, 78]。此外，操作系统可以把第 6.1 节只能扣留或补偿的一些操作变为可逆：以事务方式向持久存储写入的系统可以回滚该写入 [79]，建立在写时复制或不可变存储之上的系统通过移动一个指针到达较早状态 [80, 81]。

第 7 章 相关工作

动态可组合性与若干成熟的研究领域相交。我们考察其中最相关的工作路线，并把我们的贡献与它们逐一区分。

7.1 效应与协效应系统

第 2 节将效应 (effect) 与协效应 (coeffect) 作为本工作背后的理论支柱作了综述。我们首先定位如今在工业实践中常见的单子式效应系统，然后考察在与 Cordis 相关的方向上扩展效应与协效应的三条研究路线：把代数效应重铸为能力、给效应以可逆语义、以及在单次的分次纪律 (graded discipline) 之下统一效应与协效应。

单子式效应系统。一族库在既有通用语言的类型系统中编码效应，把效应表示为由运行时执行的单子 (monad) 值。Scala 的 ZIO [82] 把一个计算建模为 $ZIO[R, E, A]$ ，TypeScript 的 Effect-TS [83] 则建模为 $Effect<A, E, R>$ ，一个泛型，其参数描述计算的结果、其带类型的错误，以及其上下文必须供给的服务；fp-ts 库 [84] 通过基于 Reader 的单子变换子编码同样的错误与需求通道。两点特征把这些系统与 Cordis 区分开来。第一，追踪是以单子嵌入买来的：程序只有被写在效应类型之内才能获得追踪，而 Cordis 把效应追踪作为普通宿主代码之上的一个 overlay。第二，需求由解释来解除，即由一个安装起来供给其操作的服务来解除，而当该服务被撤走时，其操作所造成的后果仍然留在原处；Cordis 则为每个副作用 (effect) 配以逆变换，并随提供方 (provider) 的来去重新解析需求 (第 3.1 节、第 3.2 节)。

作为能力的代数效应。代数效应 (algebraic effect, 第 2.1 节) 使效应操作对类型系统可见。与本工作最接近的扩展是 Brachthäuser 等人的 Effekt 语言，它把效应类型重新解释为能力 (capability) [85, 86]：效应类型表达的是一个计算从其上下文 (context) 要求什么，而非它可能产生什么副作用。与我们的工作一样，这一视角把上下文视为能力的中介者。Cordis 与 Effekt 在两方面不同。(1) 在目的上，代数效应让效应可见是为了支持模块化解释，给一个操作多种处理器语义；而 Cordis 让它们可见是为了支持追踪与逆转，为每次上下文变换配以逆变换。(2) 在设定上，Effekt 在类型层面静态地约束效应，默认采用基于作用域的推理，其中能力是二等的、被限制在其词法作用域内，并通过装箱 (boxing) 恢复一等使用——装箱通过在类型中追踪被捕获的能力来解除该限制；Cordis 则在运行时约束效应，目标是在组件移除时完全回收资源；第 6.7 节讨论了在这个意义上把上下文二等化的语言能提供什么。

可逆效应语义。另一条平行的路线赋予效应可逆语义而非解释语义。Heunen 等人 [87] 通过把 Hughes 的箭头改造为匕首箭头 (dagger arrow) 与逆箭头 (inverse arrow)，在可逆设定中建模副作用，捕捉了序列化与可变存储这类其操作可容许逆变换的效应。这是与我们的可逆副作用 (reversible effect) 最接近的形式论述：两者都为每个效应配上撤销它的手段，而非通过处理器解除它。二者的差别在于可逆性居于何处，以及它们要求多少可逆性。Heunen 等人工作在指称的、范畴的设定中，可逆性是一个全局属性，由构造保证，因为每个计算都可逆，逆是双边的且从范畴结构中恢复。Cordis 在运行时追踪逆变换，且要求更少：不要求整个计算可逆，只要求每个原子效应容许一个单边逆，由调用方在实施之处供给而非推导出来，任意复合的逆由此经复合得出 (第 3.1 节)。

作为统一效应与协效应的分次类型。Orchard 等人 [88] 提出分次模态类型 (graded modal type) 作为涵盖效应推理 (经分次单子) 与协效应推理 (经分次余单子) 的伞形概念，并在

Granule 语言中实现，证明了单一的类型系统可以同时追踪一个计算做什么与它需要什么；更近的工作把协效应扩展到命令式的类 Java 语言 [89, 90] 与 call-by-push-value [91]。所有这些都在类型层面运作：效应与协效应是编译期在词法固定的作用域上检查的静态标注。我们的贡献与这一分析正交：我们把同样的两个概念提升为运行时机制，使 Cordis 得以处理动态组合。时间性撤回与空间性依赖随着已加载组件集合的演化而被重新解析，而不是在固定的程序文本上一次性敲定。

7.2 编程范式

第 3.3.3 节确立了上下文范式，即经由显式上下文中介副作用（effect）与协副作用（coeffect）的纪律。两个成熟范式值得显式比较：一个与我们共享术语，另一个与我们共享对横切关注点的处理。

面向上下文的编程。面向上下文编程（context-oriented programming, COP）[92, 93] 为语言装备层（layer）——部分的方法与类定义，依据执行上下文在运行时被激活与停用，使行为无须基代码点名其上下文依赖即可适应 [94]。COP 与 Cordis 在把上下文视为一等、运行时可变的实体，以及动态地激活与停用行为上相合，但相似只是名义上的。在 COP 中，“上下文”指环境执行情境（如位置、用户、模式），激活在动态定界的区间内改变方法分派；一个层既不追踪它引发的副作用，也不逆转它们，激活也不受依赖满足的支配。在 Cordis 中，上下文是中介副作用与协副作用的 Γ_∞ 实体：激活运行组件的可逆副作用并由反应式协副作用满足驱动（第 3.2 节），停用则完全逆转它们。COP 变化的是运行什么行为；Cordis 组合并逆转的是组件安装哪些效应与依赖。二者的差别是一种权衡。COP 把激活折叠进宿主语言的方法分派，以语言特定性为代价获得动态定界的层区间；而 Cordis 作为语言无关的 overlay，在一个共享的上下文之上反应式地解析激活。因此，Cordis 只能把 COP 的全局、值驱动的片段表达为协副作用：实现之间的上下文相关选择，而非动态定界的激活。

面向方面的编程。面向方面编程（aspect-oriented programming, AOP）[95, 96] 把一个横切关注点模块化为一个方面（aspect）：一个在基程序中选取连接点（join point）之上量化的切入点（pointcut），以及织入在每个连接点的通知（advice）。Cordis 处理的是同一个问题——否则会散布在各组件之间的上下文相关行为——但它对应于方面的角色是一个协副作用：一个许多组件共同声明依赖的共享中介点，使得横切行为可以在那里被重塑，而无需编辑其中任何一个。两个范式随后在两条轴上不同。(1) 声明对浑然不觉：AOP 切入点是浑然不觉且量化的，匹配任意连接点，其代码并不知道自己被通知；而 Cordis 把横切限定在每个组件声明的协副作用之内，因此其触及范围恰为所声明的表面。这带来了确定性与可追溯性：应用编排器可以在配置层检视并治理什么横切了一个组件，而无需阅读或分析其源码；而 AOP 关注点只有通过在其上量化的那些方面才是可读的。(2) 生命周期整合：Cordis 中的横切变更由组件的效应承载，在组件卸载时被逆转并反应式地传播到其依赖方，因此它是动态组合模型中的一步棋；动态 AOP 系统 [97, 98] 也可以在运行时织入与反织入，但那是独立的操作，既不绑定组件的生命周期，也不触发被通知代码间的重新解析。

7.3 时间性可组合

时间性可组合（temporal composability）关注在运行中的程序里替换或移除组件，同时恢复它安装的效应。既有方法按其对待离场组件之状态与效应的方式划分：把状态前向迁移到后继版本；经开发者手写的清理恢复效应；在一个预先固定的作用域内自动逆转效应；或者，从运行时在某接口上介入而积累的记录中回收资源。

有状态前向迁移。一大族系统通过跨版本携带状态前进，在运行中的程序里无停机地替换组件。它们都遵守同一时序纪律：组件只有到达安全的、无交互的点才可被替换。Kramer 与 Magee 把这一判据确立为静寂（quiescence）[51]，Vandewoude 等人后来将其放松为破坏性更小的静和（tranquility）[52]；我们的滚动更新模式（第 6.2 节）通过在卸载提供方前排空在途请求来实施它。动态软件更新（dynamic software updating, DSU）进而经手写的变换函数迁移状态：Hicks 等人为 C 设计的通用 DSU [99]、Stoye 等人经 con-freeness 分析得到的类型安全更新点 [100]，以及 Hayden 等人的 Kitsune [101]，都把旧版本数据映射到新版本的表示，原样继承堆对象、打开的文件与连接，同时重新初始化未被迁移的部分。同一纪律延伸到持久状态：Overeem 等人 [102] 经手写的升级操作，在保持系统可用的同时转换一个运行中事件存储在模式版本之间的数据。Erlang/OTP [15] 在进程层面采取同样立场，经 `code_change/3` 迁移状态，并通过重启受监管的进程而非逆转其效应来从故障中恢复；JavaScript 的热模块替换（hot module replacement, HMR，如 webpack [46]、Vite [47]）在模块层面亦然，经 `module.hot` 或 `import.meta.hot` API 在一次 reload 之间传递状态。与 Cordis 的模块替换（第 5.2 节）相比，这些方法更优雅地迁移内存中状态：Cordis 逆转旧组件被追踪的效应，并让新组件从干净状态重新施加自己的效应，因此组件自身的内存中状态除非放在更长寿命的依赖中，否则不能在 reload 之后幸存；把 DSU 式前向迁移叠加到可逆副作用之上是未来工作。尽管如此，Cordis 的方法在两方面更为一般：它不需要 DSU 与 HMR 所要求的那类手写迁移函数，而且它支持把组件整体卸载并回收其资源，而不只是在原地更新。

开发者手写的恢复。第二族经开发者手写的清理或补偿逻辑来恢复组件的效应。插件生命周期约定（如 OSGi [50]、Eclipse 的扩展点、IntelliJ 与 VSCode）把清理托付给开发者书写的卸载回调；命令模式（Command pattern）[103] 把操作连同撤销方法封装起来，供撤销/重做栈使用；saga 模型 [49] 把长寿命事务构造为每一步都配有补偿动作的步骤序列；代数效应处理器可以附加在 teardown 时运行的终结器 [104]；事件溯源（event sourcing）[105] 则通过追加补偿事件而非执行任何逆变换来撤回状态。在所有这些之中，逆都是一项不被强制履行、与操作脱钩的义务，因此被遗忘的逆会无声地泄漏资源（如第 1.2.1 节凭经验记录的那样）。React 的 `useEffect` hook [106] 在结构上最接近把效应与其逆配对，它返回一个由运行时在每次重新执行之前和卸载之时调用的清理函数。其短板是可组合性：hook 只能在组件或其他 hook 的顶层被调用，绝不能在条件、循环或嵌套函数之内，其效应体既不接受 `async` 函数也不接受迭代器。效应因而既不能由其他效应装配，也不能与控制流交错，从而没有任何可以从中导出复合逆的东西。Cordis 的效应没有这些限制：它们是自由复合、可以异步运行的普通操作，且只要求为每个原子效应手写一个逆，任何复合的逆都由此经复合导出，因此装配既有的效应完全不需要书写任何逆。这种每个效应与其逆的结构配对，使完全恢复成为系统的不变量，而非开发者纪律问题。

静态定界的逆转。第三族自动地、由构造保证地逆转效应，但把逆转限定在一个预先固定的作用域内。软件事务内存（software transactional memory, STM）[107, 108] 源自硬件事务内存 [109]，它记录读/写日志，使得一组内存操作或者提交或者中止，把内存回滚到事务前状态。可逆计算，从 Landauer 与 Bennett 的热力学分析 [110, 111] 到 Janus [112] 这类可逆语言，走得更远，使整个计算的每一步都全局可逆。可逆进程演算把回溯内建于语义本身：RCCS [113] 为每个进程携带一份记忆，当某步所通向的过去与当前因果等价时允许将该步收回；Phillips 与 Ulidowski [114] 为 CCS、ACP 与 CSP 一致地导出可逆算子，同时保持其前向操作语义。它们的因果一致性判据是 Cordis 恢复所遵循顺序的并发对应物——一个累加器（accumulator）按后进先出顺序施加组件自己的逆，且第 4.3.1 节的守卫把提供方的撤离（withdrawal）推迟到其消费者都已停用之后（定理 63）。然而，其触及范围由语义固定，每个执行过的动作都保持可撤销；

而 Cordis 的组件为每个原子效应供给一个逆，其累加器把上下文带回它的复合开始之处。线性类型 [115]、RAII [4] 与 Rust 的所有权系统 [61] 把资源的释放绑定到词法区域。这些都静态地固定了逆转的作用域与触及范围；相比之下，Cordis 不预先固定这样的作用域：它在组件的整个生命周期上逆转任意的上下文操作，并把词法资源管理视为互补手段，适于单个组件内部的局部资源。

介入式回收。第四族无须组件自己供给逆，而是通过在运行时控制的接口上记录其获取，来回收组件所获得的东西。Nooks [77] 包装每一笔跨越 Linux 内核与其可加载扩展之间边界的调用，使扩展触及的内核对象经过一个对象追踪器，其记录告知恢复管理器在扩展失败时释放什么；影子驱动 [78] 从另一侧搭接同样的调用，记录决定驱动状态的请求与配置，使重启后的实例能恢复到该状态。Akeso [116] 转而经编译器插桩获得记录，把内核执行划分为可嵌套的恢复域，记录其状态变更与跨线程依赖，并把出错的请求连同依赖它的每个域一起回滚。回收于是得自运行时维护的记录，而非开发者记得书写的清理，这使这一族成为可逆副作用在系统层面最接近的先例。它在词汇与触及范围上与 Cordis 不同。平台固定了什么可以被记录——无论是按内核对象类型给出的释放代码、每驱动类一个影子，还是每个被插桩分配器的一个逆——因此组件只能持有平台已知如何释放的资源；而 Cordis 的组件引入自己的效应并为每个原子效应供给逆（第 3.1 节）。回收同样受一次提交的请求或同一扩展的一次重启所限定，而 Cordis 在组件的整个生命周期上逆转，并把移除传播到其依赖方，后者再依次释放自己的效应（第 3.2 节）。

7.4 空间性可组合

空间性可组合（spatial composability）关注组件对其他组件的依赖如何被声明与绑定。既有机制按绑定如何回应变化划分：在初始化时一次性接线依赖；对整个组件的可用性作出反应；或在单个值的粒度假传播变化。

初始化时的依赖接线。两种成熟机制在初始化时把组件接线在一起。依赖注入框架 [38]（如 Spring [117]、Guice、Angular、Inversify）在初始化时把依赖注入组件，UI 框架上下文（如 Vue.js 的 provide/inject 与 React 的 Context API）沿组件树传递它们。其中一些支持动态作用域（如 Spring 的 prototype/request 作用域、Angular 的层级注入器），但都不能反应式地重新解析：当提供方在运行时被替换或移除时，既有的依赖方既不被停用，也不被重新初始化，而且没有一个提供我们组件状态机那种生命周期管理。Cordis 的反应式协副作用（reactive coeffect, 第 3.2 节）补上了这一点：通知机制在满足谓词变化时触发生命周期迁移。

可用性反应式的组件模型。与我们的反应式协副作用最接近的先例对服务可用性作出反应。OSGi 的 Declarative Services 与 iPOJO [118, 119] 让组件声明提供与需要的服务，运行时随服务的出现与消失自动激活与停用它们；iPOJO 的 Gravity 项目 [119] 明确面向对变化的服务可用性的自主运行时适应，其 provide/require 模型直接预示了 Cordis 的 ctx.provide/ctx.get 模式。R-OSGi [53] 经 RPC 把同一抽象透明地扩展到分布式环境，把网络故障映射为服务撤离事件——第 6.2 节把这一模式作为 Cordis 模型的扩展加以讨论。所有这些系统都经一个停用回调恢复，而该回调有两重局限。第一，回调是手写的，因此资源安全依赖于开发者纪律，被遗忘的回调会无声泄漏。第二，回调是同步的：若 teardown 需要与正在离场的依赖进行一次异步交互，这些框架没有等待它的协议，只能对一个可能已然失效的引用做阻塞等待。Cordis 的反应式协副作用同时封堵两个缺口：停用逆转依赖方积累的效应，而其惯性的 UNLOADING 状态（第 4.3.3 节）在处理进一步变化之前，把异步 teardown 运行至完成。

值层面的反应性。函数式反应式编程（functional reactive programming, FRP）[120] 及其现

代形态（如 SolidJS、Vue 的反应式系统、Angular Signals 中的信号 [121, 122]）以值层面的粒度传播变化：当信号变化时，派生计算被同步地或在调度器 [123] 之下重新求值。Cordis 的反应式协副作用在组件粒度上运作，附加以值层面传播所不建模的异步生命周期语义。同一粒度差异在一致性上反过来看也成立：在一个回合（turn）之内、按依赖图固定的顺序传播，使 FRP 可以要求没有派生计算读到更新与陈旧输入的混合，此即无故障（glitch freedom）[124]；而 Cordis 没有回合的对应物，编排动作一次到达一个，且只保证单个迁移不会横跨其协副作用的两次解析（定理 64）。二者互补而非竞争：Cordis 的协副作用自身可以携带反应式值，组件只在其真正消费的部分上更新，把组件层面的反应性细化为横跨两个层面的更细粒度反应式协副作用。

第 8 章 结论

我们通过把效应 (effect) 与协效应 (coeffect) 的经典概念提升为运行时机制, 为动态可组合性给出了一个形式基础。可逆副作用 (reversible effect) 解决局部的时间性可组合 (temporal composability): 每一次上下文 (context) 变换都携带一个由运行时追踪的逆变换, 且追踪与恢复都保持复合, 因此在组件移除时上下文得以恢复。反应式协副作用 (reactive coeffect) 解决局部的空间性可组合 (spatial composability): 每当上下文变化, 组件都会依据其协副作用规约得到通知, 每次变化被归类为激活、停用或中性, 协副作用隔离 (isolation) 改变一个被声明的键解析到什么, 协副作用拦截 (interception) 改变该绑定如何被使用。我们把副作用上下文 (effect context) 与协副作用上下文 (coeffect context) 统一为单一的上下文类型, 其中协副作用上的一个观测等价 (observational equivalence) 为副作用供给了独立性, 由此构成一种面向时空可组合性 (spatiotemporal composability) 的编程范式。再把这些机制组合为组件 (component) 的概念, 便得到一个动态组合演算 (calculus of dynamic composition), 其元理论 (metatheory) 把时空可组合性从单个组件推广到由交错组件构成的整个系统。我们把这一范式实现为 Cordis 元框架: 一个提供副作用追踪与协副作用解析的核心库, 以及一个具备配置调和 (configuration reconciliation) 与热模块替换 (hot module replacement) 的声明式组件加载器 (component loader)。Koishi 案例研究在一个拥有 4000 余个社区插件的生产系统中验证了 Cordis 的设计。

在人工策展的插件生态之外, 一个引人注目的未来验证方向是自我进化的智能体底座 (self-evolving agent harness, 第 1.2.2 节): AI 智能体 (agent) 在极少人工监督下持续生成并替换自己底座的组件。把 Cordis 应用于这样的场景, 将验证快速组件替换之下完全恢复的时间性保证, 以及频繁拓扑变化之下依赖协调的空间性保证。这样的验证将证明, 该范式作为智能体底座及其他自治系统中可恢复、可协调且连续的自我进化的基础, 具有切实的适用性。

参考文献

Bibliography

- [1] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972, doi: 10.1145/361598.361623.
- [2] D. Birsan, "On Plug-ins and Extensible Architectures," *ACM Queue*, vol. 3, no. 2, pp. 40–46, 2005, doi: 10.1145/1053331.1053345.
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.
- [4] B. Stroustrup, *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [5] S. Marlow, S. Peyton Jones, A. Moran, and J. Reppy, "Asynchronous Exceptions in Haskell," in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in PLDI '01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 274–285. doi: 10.1145/378795.378858.
- [6] L. Cardelli, "Program Fragments, Linking, and Modularization," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 1997)*, ACM Press, 1997, pp. 266–277. doi: 10.1145/263699.263735.
- [7] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison-Wesley, 2002.
- [8] R. Lopopolo, "Harness Engineering: Leveraging Codex in an Agent-First World." [Online]. Available: <https://openai.com/index/harness-engineering/>
- [9] Anthropic, "Harness Design for Long-Running Application Development." [Online]. Available: <https://www.anthropic.com/engineering/harness-design-longrunning-apps>
- [10] L. Wang et al., "A Survey on Large Language Model Based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024, doi: 10.1007/s11704-024-40231-1.
- [11] Y. Qin et al., "Tool Learning with Foundation Models," *ACM Computing Surveys*, 2025, doi: 10.1145/3704435.
- [12] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," *CoRR*, vol. abs/2310.08560, 2023.
- [13] T. Guo et al., "Large Language Model Based Multi-Agents: A Survey of Progress and Challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, in IJCAI 2024. 2024, pp. 8048–8057. doi: 10.24963/ijcai.2024/890.
- [14] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large Language Models as Tool Makers," in *Proceedings of the Twelfth International Conference on Learning Representations*, in ICLR 2024. 2024. [Online]. Available: <https://openreview.net/forum?id=qV83K9d5WB>
- [15] J. Armstrong, "Making Reliable Distributed Systems in the Presence of Software Errors," *Doctoral dissertation*, 2003. [Online]. Available: https://erlang.org/download/armstrong_thesis_2003.pdf
- [16] E. Moggi, "Notions of computation and monads," *Information and Computation*, vol. 93, no. 1, pp. 55–92, 1991, doi: 10.1016/0890-5401(91)90052-4.
- [17] G. Plotkin and J. Power, "Adequacy for Algebraic Effects," in *Foundations of Software Science and Computation Structures*, F. Honsell and M. Miculan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–24.
- [18] T. Petricek, D. Orchard, and A. Mycroft, "Coeffects: unified static analysis of contextdependence," in *Proceedings of the 40th International Conference on Automata, Languages, and Programming - Volume Part II*, in ICALP'13. Riga, Latvia: Springer-Verlag, 2013, pp. 385–397. doi: 10.1007/978-3-642-39212-2_35.

- [19] M. Gaboardi, S.-ya Katsumata, D. Orchard, F. Breuvar, and T. Uustalu, “Combining effects and coeffects via grading,” in *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, in ICFP 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 476–489. doi: 10.1145/2951913.2951939.
- [20] A. Church, “A Formulation of the Simple Theory of Types,” *The Journal of Symbolic Logic*, vol. 5, no. 2, pp. 56–68, 1940, doi: 10.2307/2266170.
- [21] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [22] J. M. Lucassen and D. K. Giford, “Polymorphic Effect Systems,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’88. San Diego, California, USA: Association for Computing Machinery, 1988, pp. 47–57. doi: 10.1145/73560.73564.
- [23] P. Wadler, “Monads for functional programming,” in *Program Design Calculi*, M. Broy, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 233–264.
- [24] G. Plotkin and J. Power, “Notions of Computation Determine Monads,” in *Foundations of Software Science and Computation Structures*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 342–356. doi: 10.1007/3-540-45931-6_24.
- [25] G. Plotkin and M. Pretnar, “Handlers of Algebraic Effects,” in *Programming Languages and Systems (ESOP)*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7.
- [26] M. Pretnar, “An Introduction to Algebraic Effects and Handlers. Invited tutorial paper,” *Electron. Notes Theor. Comput. Sci.*, vol. 319, no. C, pp. 19–35, Dec. 2015, doi: 10.1016/j.entcs.2015.12.003.
- [27] D. Leijen, “Koka: Programming with Row Polymorphic Effect Types,” *Electronic Proceedings in Theoretical Computer Science*, vol. 153, pp. 100–126, Jun. 2014, doi: 10.4204/eptcs.153.8.
- [28] D. Leijen, “Type directed compilation of row-typed algebraic effects,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, in POPL ’17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: 10.1145/3009837.3009872.
- [29] A. Bauer and M. Pretnar, “Programming with algebraic effects and handlers,” *Journal of Logical and Algebraic Methods in Programming*, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: 10.1016/j.jlamp.2014.02.001.
- [30] K. Sivaramakrishnan et al., “Retrofitting parallelism onto OCaml,” *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408995.
- [31] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: a calculus of context-dependent computation,” in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, in ICFP ’14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: 10.1145/2628136.2628160.
- [32] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” *Electronic Notes in Theoretical Computer Science*, vol. 203, no. 5, pp. 263–284, 2008, doi: 10.1016/j.entcs.2008.05.029.
- [33] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A Core Quantitative Coeffect Calculus,” in *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: 10.1007/978-3-642-54833-8_19.
- [34] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for differential privacy,” *SIGPLAN Not.*, vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: 10.1145/1932681.1863568.
- [35] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: 10.1145/292540.292555.
- [36] D. E. Denning, “A lattice model of secure information flow,” *Commun. ACM*, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

-
- [37] U. Dal Lago and F. Gavazzo, "A relational theory of effects and coefficients," *Proc. ACM Program. Lang.*, vol. 6, no. POPL, Jan. 2022, doi: 10.1145/3498692.
 - [38] M. Fowler, "Inversion of Control Containers and the Dependency Injection pattern." [Online]. Available: <https://martinfowler.com/articles/injection.html>
 - [39] A. M. Pitts and I. D. B. Stark, "Observable Properties of Higher Order Functions that Dynamically Create Local Names, or What's New?," in *Mathematical Foundations of Computer Science 1993 (MFCS 1993)*, in *Lecture Notes in Computer Science*, vol. 711. Springer, 1993, pp. 122–141. doi: 10.1007/3-540-57182-5_8.
 - [40] G. D. Plotkin, "LCF Considered as a Programming Language," *Theoretical Computer Science*, vol. 5, no. 3, pp. 223–255, 1977, doi: 10.1016/0304-3975(77)90044-5.
 - [41] D. R. Ghica, K. Muroya, and T. Waugh Ambridge, "A Robust Graph-Based Approach to Observational Equivalence," *Logical Methods in Computer Science*, vol. 21, no. 2, p. 8:1–8:95, 2025, doi: 10.46298/LMCS-21(2:8)2025.
 - [42] X. Leroy and S. Blazy, "Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations," *Journal of Automated Reasoning*, vol. 41, no. 1, pp. 1–31, 2008, doi: 10.1007/s10817-008-9099-0.
 - [43] R. P. James and A. Sabry, "Yield: Mainstream Delimited Continuations," in *First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011)*, 2011, pp. 20–32. [Online]. Available: <https://homes.luddy.indiana.edu/sabry/files/yield.pdf>
 - [44] A. W. Mazurkiewicz, "Trace Theory," in *Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II*, in *Lecture Notes in Computer Science*, vol. 255. Springer, 1986, pp. 279–324. doi: 10.1007/3-540-17906-2_30.
 - [45] U. A. Acar, G. E. Blelloch, and R. Harper, "Adaptive functional programming," *ACM Transactions on Programming Languages and Systems*, vol. 28, no. 6, pp. 990–1034, 2006, doi: 10.1145/1186632.1186634.
 - [46] webpack, "Hot Module Replacement." [Online]. Available: <https://webpack.js.org/api/hot-module-replacement>
 - [47] Vite, "HMR API." [Online]. Available: <https://vite.dev/guide/api-hmr>
 - [48] E. N. (M. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, "A Survey of Rollback-Recovery Protocols in Message-Passing Systems," *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002, doi: 10.1145/568522.568525.
 - [49] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, in *SIGMOD '87*. 1987, pp. 249–259. doi: 10.1145/38713.38742.
 - [50] OSGi Alliance, OSGi Core Release 8. OSGi Alliance, 2020. [Online]. Available: <https://docs.osgi.org/specification/osgi.core/8.0.0/>
 - [51] J. Kramer and J. Magee, "The Evolving Philosophers Problem: Dynamic Change Management," *IEEE Transactions on Software Engineering*, vol. 16, no. 11, pp. 1293–1306, 1990, doi: 10.1109/32.60317.
 - [52] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D'Hondt, "Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates," *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 856–868, 2007, doi: 10.1109/tse.2007.70733.
 - [53] J. S. Rellermeyer, G. Alonso, and T. Roscoe, "R-OSGi: Distributed Applications Through Software Modularization," in *Proceedings of the ACM/IFIP/USENIX 8th International Middleware Conference*, in *Middleware '07*. 2007, pp. 1–20. doi: 10.1007/978-3-540-76778-7_1.
 - [54] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.
 - [55] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," technical report SRL2003–2, 2003. [Online]. Available: <http://zesty.ca/capmyths/usenix.pdf>

- [56] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, “Capsicum: Practical Capabilities for UNIX,” in Proceedings of the 19th USENIX Security Symposium, 2010, pp. 29–46. [Online]. Available: https://www.usenix.org/legacy/events/sec10/tech/full_papers/Watson.pdf
- [57] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, “Efficient Software-Based Fault Isolation,” in Proceedings of the 14th ACM Symposium on Operating Systems Principles, in SOSP ’93. 1993, pp. 203–216. doi: 10.1145/168619.168635.
- [58] A. Barth, A. P. Felt, P. Saxena, and A. Boodman, “Protecting Browsers from Extension Vulnerabilities,” in Proceedings of the 17th Annual Network and Distributed System Security Symposium, in NDSS ’10. 2010. [Online]. Available: <https://www.ndss-symposium.org/ndss2010/protecting-browsers-extension-vulnerabilities/>
- [59] W. W. Ho and R. A. Olsson, “An Approach to Genuine Dynamic Linking,” *Software: Practice and Experience*, vol. 21, no. 4, pp. 375–390, 1991, doi: 10.1002/SPE.4380210404.
- [60] P. Wadler and S. Blott, “How to Make Ad-hoc Polymorphism Less Ad Hoc,” in Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’89. 1989, pp. 60–76. doi: 10.1145/75277.75283.
- [61] N. D. Matsakis and F. S. K. II, “The Rust Language and Type System,” in ACM SIGPLAN ML Family Workshop, Gothenburg, Sweden, Sep. 2014.
- [62] D. Dreyer, R. Harper, M. M. T. Chakravarty, and G. Keller, “Modular Type Classes,” in Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’07. 2007, pp. 63–70. doi: 10.1145/1190216.1190229.
- [63] Microsoft, “Declaration Merging.” [Online]. Available: <https://www.typescriptlang.org/docs/handbook/declaration-merging.html>
- [64] T. Van Cutsem and M. S. Miller, “Proxies: Design Principles for Robust Object-oriented Intercession APIs,” in Proceedings of the 6th Symposium on Dynamic Languages, in DLS ’10. 2010, pp. 59–72. doi: 10.1145/1869631.1869638.
- [65] R. Hettinger, “Descriptor HowTo Guide.” [Online]. Available: <https://docs.python.org/3/howto/descriptor.html>
- [66] P. Maes, “Concepts and Experiments in Computational Reflection,” in Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1987, pp. 147–155. doi: 10.1145/38765.38821.
- [67] G. Bracha and D. M. Ungar, “Mirrors: design principles for meta-level facilities of objectoriented programming languages,” in Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOP-SLA), 2004, pp. 331–344. doi: 10.1145/1028976.1029004.
- [68] R. Rouvoy and P. Merle, “Leveraging component-based software engineering with Fraclet,” *Annals of Telecommunications*, vol. 64, no. 1–2, pp. 65–79, 2009, doi: 10.1007/s12243-008-0072-z.
- [69] E. Burmako, “Scala Macros: Let Our Powers Combine!,” in Proceedings of the 4th Workshop on Scala, in SCALA@ECOOP ’13. 2013, p. 3:1–3:10. doi: 10.1145/2489837.2489840.
- [70] S. Raemaekers, A. van Deursen, and J. Visser, “Semantic Versioning and Impact of Breaking Changes in the Maven Repository,” *Journal of Systems and Software*, vol. 129, pp. 140–158, 2017, doi: 10.1016/j.jss.2016.04.008.
- [71] P. Lam, J. Dietrich, and D. J. Pearce, “Putting the Semantics into Semantic Versioning,” in Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, in Onward! ’20. 2020, pp. 157–179. doi: 10.1145/3426428.3426922.
- [72] P. Abate, R. Di Cosmo, R. Treinen, and S. Zacchiroli, “Dependency Solving: A Separate Concern in Component Evolution Management,” *Journal of Systems and Software*, vol. 85, no. 10, pp. 2228–2240, 2012, doi: 10.1016/j.jss.2012.02.018.
- [73] L. Cardelli, “Structural Subtyping and the Notion of Power Type,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’88. 1988, pp. 70–79. doi: 10.1145/73560.73566.

- [74] B. Meyer, “Applying ”Design by Contract”,”Computer, vol. 25, no. 10, pp. 40–51, 1992, doi: 10.1109/2.161279.
- [75] B. C. Pierce, “Bounded Quantification is Undecidable,”Information and Computation, vol. 112, no. 1, pp. 131–165, 1994, doi: 10.1006/inco.1994.1055.
- [76] A. Haas et al., “Bringing the web up to speed with WebAssembly,”in Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2017, pp. 185–200. doi: 10.1145/3062341.3062363.
- [77] M. M. Swift, B. N. Bershad, and H. M. Levy, “Improving the reliability of commodity operating systems,”in Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), ACM, 2003, pp. 207–222. doi: 10.1145/945445.945466.
- [78] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, “Recovering device drivers,”ACM Transactions on Computer Systems, vol. 24, no. 4, pp. 333–360, 2006, doi: 10.1145/1189256.1189257.
- [79] D. E. Porter, O. S. Hofmann, C. J. Rossbach, A. Benn, and E. Witchel, “Operating System Transactions,”in Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), ACM, 2009, pp. 161–176. doi: 10.1145/1629575.1629591.
- [80] O. Kiselyov and C.-chieh Shan, “Delimited Continuations in Operating Systems,”in Modeling and Using Context (CONTEXT 2007), in Lecture Notes in Computer Science, vol. 4635. Springer, 2007, pp. 291–302. doi: 10.1007/978-3-540-74255-5_22.
- [81] E. Dolstra and A. Löb, “NixOS: a purely functional Linux distribution,”in Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP), ACM, 2008, pp. 367–378. doi: 10.1145/1411204.1411255.
- [82] ZIO, “ZIO: Type-safe, composable asynchronous and concurrent programming for Scala.”[Online]. Available: <https://zio.dev/>
- [83] Effect, “Effect: A TypeScript library for building robust applications.”[Online]. Available: <https://effect.website/>
- [84] G. Canti, “fp-ts: Functional programming in TypeScript.”[Online]. Available: <https://github.com/gcanti/fp-ts>
- [85] J. I. Brachthäuser, P. Schuster, and K. Ostermann, “Effects as capabilities: effect handlers and lightweight effect polymorphism,”Proc. ACM Program. Lang., vol. 4, no. OOPSLA, 2020, doi: 10.1145/3428194.
- [86] J. I. Brachthäuser, P. Schuster, E. Lee, and A. Boruch-Gruszecki, “Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back,”Proc. ACM Program. Lang., vol. 6, no. OOPSLA1, 2022, doi: 10.1145/3527320.
- [87] C. Heunen, R. Kaarsgaard, and M. Karvonen, “Reversible Effects as Inverse Arrows,”in Proceedings of the Thirty-Fourth Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIV), in Electronic Notes in Theoretical Computer Science, vol. 341. 2018, pp. 179–199. doi: 10.1016/j.entcs.2018.11.009.
- [88] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,”Proc. ACM Program. Lang., vol. 3, no. ICFP, 2019, doi: 10.1145/3341714.
- [89] R. Bianchini, F. Dagnino, P. Giannini, E. Zucca, and M. Servetto, “Coeffects for sharing and mutation,”Proc. ACM Program. Lang., vol. 6, no. OOPSLA2, Oct. 2022, doi: 10.1145/3563319.
- [90] R. Bianchini, F. Dagnino, P. Giannini, and E. Zucca, “A Java-like calculus with heterogeneous coeffects,”Theoretical Computer Science, vol. 971, p. 114063, 2023, doi: <https://doi.org/10.1016/j.tcs.2023.114063>.
- [91] C. Torczon, E. Suárez Acevedo, S. Agrawal, J. Velez-Ginorio, and S. Weirich, “Effects and Coeffects in Call-by-Push-Value,”Proc. ACM Program. Lang., vol. 8, no. OOPSLA2, Oct. 2024, doi: 10.1145/3689750.
- [92] R. Hirschfeld, P. Costanza, and O. Nierstrasz, “Context-oriented Programming,”Journal of Object Technology, vol. 7, no. 3, pp. 125–151, 2008, doi: 10.5381/jot.2008.7.3.a4.

- [93] P. Costanza and R. Hirschfeld, “Language constructs for context-oriented programming: an overview of ContextL,” in *Proceedings of the 2005 Symposium on Dynamic Languages (DLS '05)*, ACM, 2005, pp. 1–10. doi: 10.1145/1146841.1146842.
- [94] G. Salvaneschi, C. Ghezzi, and M. Pradella, “Context-oriented programming: A software engineering perspective,” *Journal of Systems and Software*, vol. 85, no. 8, pp. 1801–1817, 2012, doi: 10.1016/j.jss.2012.03.024.
- [95] G. Kiczales et al., “Aspect-Oriented Programming,” in *ECOOP'97 —Object-Oriented Programming*, 11th European Conference, in *Lecture Notes in Computer Science*, vol. 1241. Springer, 1997, pp. 220–242. doi: 10.1007/BFb0053381.
- [96] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, “An Overview of AspectJ,” in *ECOOP 2001 —Object-Oriented Programming*, 15th European Conference, in *Lecture Notes in Computer Science*, vol. 2072. Springer, 2001, pp. 327–353. doi: 10.1007/3-540-45337-7_18.
- [97] A. Popovici, T. Gross, and G. Alonso, “Dynamic Weaving for Aspect-Oriented Programming,” in *Proceedings of the 1st International Conference on Aspect-Oriented Software Development (AOSD 2002)*, ACM, 2002, pp. 141–147. doi: 10.1145/508386.508404.
- [98] J. Bonér, “What Are the Key Issues for Commercial AOP Use: How Does AspectWerkz Address Them?,” in *Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD 2004)*, ACM, 2004, pp. 5–6. doi: 10.1145/976270.976273.
- [99] M. Hicks, J. T. Moore, and S. Nettles, “Dynamic Software Updating,” in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in *PLDI '01*. 2001, pp. 13–23. doi: 10.1145/378795.378798.
- [100] G. Stoyle, M. Hicks, G. Bierman, P. Sewell, and I. Neamtiu, “Mutatis Mutandis: Safe and Predictable Dynamic Software Updating,” in *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in *POPL '05*. 2005, pp. 183–194. doi: 10.1145/1040305.1040321.
- [101] C. M. Hayden, K. Saur, E. K. Smith, and M. Hicks, “Kitsune: Efficient, General-Purpose Dynamic Software Updating for C,” *ACM Trans. Program. Lang. Syst.*, vol. 36, no. 4, 2014, doi: 10.1145/2629460.
- [102] M. Overeem, M. Spoor, and S. Jansen, “The Dark Side of Event Sourcing: Managing Data Conversion,” in *IEEE 24th International Conference on Software Analysis, Evolution and Reengineering*, in *SANER '17*. 2017, pp. 193–204. doi: 10.1109/SANER.2017.7884621.
- [103] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994.
- [104] D. Leijen, “Algebraic Effect Handlers with Resources and Deep Finalization,” technical report MSR-TR-2018-10, Apr. 2018. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/algebraic-effect-handlers-resources-deep-finalization/>
- [105] M. Fowler, “Event Sourcing.” 2005.
- [106] J. Lee, J. Ahn, and K. Yi, “React-tRace: A Semantics for Understanding React Hooks,” *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA2, pp. 471–498, 2025, doi: 10.1145/3763067.
- [107] N. Shavit and D. Touitou, “Software Transactional Memory,” in *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing*, in *PODC '95*. 1995, pp. 204–213. doi: 10.1145/224964.224987.
- [108] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, “Composable Memory Transactions,” in *Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, in *PPoPP '05*. 2005, pp. 48–60. doi: 10.1145/1065944.1065952.
- [109] M. Herlihy and J. E. B. Moss, “Transactional Memory: Architectural Support for Lock-Free Data Structures,” in *Proceedings of the 20th Annual International Symposium on Computer Architecture*, in *ISCA '93*. 1993, pp. 289–300. doi: 10.1145/165123.165164.

- [110] R. Landauer, "Irreversibility and Heat Generation in the Computing Process," *IBM Journal of Research and Development*, vol. 5, no. 3, pp. 183–191, 1961, doi: 10.1147/rd.53.0183.
- [111] C. H. Bennett, "Logical Reversibility of Computation," *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973, doi: 10.1147/rd.176.0525.
- [112] T. Yokoyama and R. Glück, "A Reversible Programming Language and its Invertible Self-Interpreter," in *Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation*, in *PEPM '07*. 2007, pp. 144–153. doi: 10.1145/1244381.1244404.
- [113] V. Danos and J. Krivine, "Reversible Communicating Systems," in *CONCUR 2004 —Concurrency Theory*, 15th International Conference, in *Lecture Notes in Computer Science*, vol. 3170. Springer, 2004, pp. 292–307. doi: 10.1007/978-3-540-28644-8_19.
- [114] I. Phillips and I. Ulidowski, "Reversing Algebraic Process Calculi," in *Foundations of Software Science and Computation Structures*, 9th International Conference (FOSSACS 2006), in *Lecture Notes in Computer Science*, vol. 3921. Springer, 2006, pp. 246–260. doi: 10.1007/11690634_17.
- [115] P. Wadler, "Linear Types Can Change the World!," in *Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2/2.3 Working Conference*, North-Holland, 1990, pp. 561–581. [Online]. Available: <https://homepages.inf.ed.ac.uk/wadler/papers/linear/linear.ps>
- [116] A. Lenharth, V. S. Adve, and S. T. King, "Recovery domains: an organizing principle for recoverable operating systems," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2009, pp. 49–60. doi: 10.1145/1508244.1508251.
- [117] C. Walls, *Spring in Action*, 6th ed. Manning Publications, 2022. [Online]. Available: <https://www.manning.com/books/spring-in-action-sixth-edition>
- [118] C. Escofier, R. S. Hall, and P. Lalanda, "iPOJO: an Extensible Service-Oriented Component Framework," in *IEEE International Conference on Services Computing*, 2007, pp. 474–481. doi: 10.1109/SCC.2007.74.
- [119] H. Cervantes and R. S. Hall, "Autonomous Adaptation to Dynamic Availability Using a Service-Oriented Component Model," in *Proceedings of the 26th International Conference on Software Engineering*, in *ICSE '04*. 2004, pp. 614–623. doi: 10.1109/ICSE.2004.1317483.
- [120] C. Elliott and P. Hudak, "Functional Reactive Animation," in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, in *ICFP '97*. 1997, pp. 263–273. doi: 10.1145/258948.258973.
- [121] G. H. Cooper and S. Krishnamurthi, "Embedding Dynamic Dataflow in a Call-by-Value Language," in *Programming Languages and Systems (ESOP 2006)*, in *Lecture Notes in Computer Science*, vol. 3924. Springer, 2006, pp. 294–308. doi: 10.1007/11693024_20.
- [122] I. Maier and M. Odersky, "Deprecating the Observer Pattern with Scala.React," technical report EPFL-REPORT-176887, 2012. [Online]. Available: <https://infoscience.epfl.ch/record/176887>
- [123] E. Bainomugisha, A. L. Carreton, T. Van Cutsem, W. De Meuter, and others, "A Survey on Reactive Programming," *ACM Comput. Surv.*, vol. 45, no. 4, 2013, doi: 10.1145/2501654.2501666.
- [124] A. Margara and G. Salvaneschi, "On the Semantics of Distributed Reactive Programming: The Cost of Consistency," *IEEE Trans. Software Eng.*, vol. 44, no. 7, pp. 689–711, 2018, doi: 10.1109/TSE.2018.2833109.